

Peer-to-peer offline chat application based on CRDT

Master Thesis

Natural Science Faculty of the University of Basel
Department of Mathematics and Computer Science
Computer Networks Group
<https://cn.dmi.unibas.ch/>

Examiner: Prof. Dr. Christian Tschudin
Supervisor: Dr. Erick Lavoie

Abhilash Mendhe
abhilash.mendhe@stud.unibas.ch
20-067-674

04/10/2024

Acknowledgments

I am very thankful to Prof. Dr. Christian Tschudin and Dr. Erick Lavoie for giving me an opportunity to do thesis in your research group. I am very grateful to Dr. Erick Lavoie for providing me research papers that have helped me to work on this thesis. Moreover, I also thank to both Prof. Tschudin and Dr. Lavoie to help me improve in my MSc thesis during monthly milestones. I again thanks to Dr. Lavoie for the weekly meetings that helped me designing CRDTs for my thesis topic.

I thank Flavio Junqueira for having a discussion in an interview that raised important questions relevant to this work, and helped me to address these challenges.

ChatGPT and Google's Gemini was used for assistance with programming syntax checking and debugging support.

Grammarly was used to improve grammar, clarity, and readability of my thesis.

Abstract

In the current era, the usage of messaging applications is ever-growing, and with the support of real-time communication, one can instantly send or receive messages globally. Regardless of geographical constraints, people are now connected globally with these applications. However, these chat applications are heavily centralized, and a single point of failure can disrupt communication between users. Yet these systems do employ replication strategies that prevent failover and make data highly available. Centralized mechanisms are slowly fading away, and techs these days focus on a decentralized approach.

This thesis also focuses on building an offline P2P group chat application that is consensus-free using CRDT. CRDT is a conflict-free replicated data type, a type of data structure that ensures eventual consistency without the need for central coordination or consensus. CRDT allows multiple replicas to update independently and merge back together, guaranteeing that all the replicas converge to the same state.

Our primary goal is to design and implement abstractions for public group conversations in a messaging app, including support for adding and removing participants, that are fully decentralized, local-first, strongly eventually-consistent, and can prevent Byzantine non-members of a group from participating. We have employed Causal-Length CRDT, which is a state-based CRDT, that will efficiently perform all these group operations.

We've build the prototype of our group chat model and conducted two test experiments. In the first experiment, we tested the insertion cost, and in the second experiment, we tested the replication strategies among the peers in a P2P network architecture. In a single system, we added around 10,000 members inside a group that took a huge space of around ~ 240 Mb, but it was improved with periodic GC (Garbage collection), significantly reducing the storage size to around $\sim 4-9$ Mb. In a P2P network, we scaled around 255 peers, where push-based synchronization incurred higher total data transfer in a network, whereas pull-based synchronization incurred low total data transfer but with higher convergence rounds.

Further possible extension can be implemented, which is discussed in the future Work.

Table of Contents

Acknowledgments	ii
Abstract	iii
Abbreviations	vii
1 Introduction	1
2 Background	4
2.1 Popular Group Messaging Apps	4
2.1.1 WhatsApp	4
2.1.1.1 Group Membership model	4
2.1.2 Telegram	5
2.1.2.1 Group Membership model	5
2.1.3 Microsoft Teams	6
2.1.3.1 Group Membership model	6
2.1.4 GroupMe	6
2.1.4.1 Group Membership model	6
2.1.5 Signal	7
2.1.5.1 Group Membership model	7
2.1.6 WeChat	8
2.1.6.1 Group Membership model	8
2.1.7 iMessage	8
2.1.7.1 Group Membership model	8
2.2 Git	9
2.3 Conflict-Free Replicated Data Types	10
2.3.1 CvRDT - State-based CRDT: Convergent Replicated Data Type	11
2.3.2 Causal-Length CRDT	12
2.3.2.1 CL-CRDT: Use Case Scenario	13
2.4 Ordering	14
2.5 Epidemics and their ways...	15
2.5.1 Types of Nodes	15
2.5.2 Types of Communication	15
2.5.3 Strategies for Epidemics	15

3	Design - Group Chat App	16
3.1	Notations	16
3.1.1	Definitions	17
3.2	APIs	18
3.2.1	Group Operation Table	19
3.3	Pseudo-code	19
3.3.1	Creating a Group	19
3.3.2	Adding Members	20
3.3.3	Removing Members	21
3.3.4	Modifying the group name and description	22
3.3.5	Promoting Member	22
3.3.6	Demoting Admin	23
3.3.7	Merging Group States	24
3.3.8	Sending Messages	26
3.3.9	Get Group Information	26
3.4	Why repairing state in add_member is necessary?	27
3.5	Behavior Under Concurrent Operations	28
3.5.1	Caveat	29
3.6	End of the Chapter	30
4	Proofs	31
4.1	Ordering	31
4.1.1	Ordering For ν	31
4.1.1.1	Definitions	31
4.1.1.2	Proofs	31
4.1.2	Ordering For $\mathbb{X}$	32
4.1.2.1	Definitions	32
4.1.2.2	Proofs	32
4.1.3	Ordering For D	33
4.1.3.1	Definitions	33
4.1.3.2	Proofs	33
4.1.4	Proof for conjunction of partial orders over $\mathbb{G}$	34
4.2	Monotonicity	36
4.3	Least Upper Bound	38
4.3.1	LUB of ν and ν'	38
4.3.2	LUB of $\mathbb{X}$ and $\mathbb{X}'$	39
4.3.3	LUB of D and D'	39
5	Implementation - Group Messaging App	42
5.1	System Model and Assumptions	42
5.2	Setup of our Application	43
5.2.1	How to run?	43
5.2.2	Inside .author-cb folder	43
5.2.3	How peers are identified?	44

5.3	Core Group Messaging App	45
5.3.1	Attributes	45
5.3.2	Group State	46
5.3.2.1	Example of Group State	47
5.3.3	Commits For Group Chat App	47
5.3.4	Group Messaging Commit References	51
5.3.5	Merging Groups	52
5.3.6	Design Choice	52
5.4	Programming Languages Used	54
6	Evaluation	56
6.1	Experiment 1	56
6.1.1	Methodology	56
6.1.2	Results	57
6.1.2.1	Moving Average Plots for Insert and Push Latency	57
6.1.2.2	CDF of Inserts and Push	58
6.1.2.3	Bar Plots of Repository Size and Git Objects	59
6.1.2.4	Discussion	59
6.2	Experiment 2	60
6.2.1	Methodology	60
6.2.2	Results	63
6.2.2.1	Single Writer	64
6.2.2.2	Three Writer	65
6.2.2.3	Performance Tradeoff Scatter Plot	66
6.2.2.4	Discussion	67
6.3	End of Chapter	68
6.3.1	Limitations	68
7	Related Work	70
8	Conclusion	74
8.1	Future Work	75
	Bibliography	77
	Appendix A – Concurrency Diagrams	82
A.1	Add Same Member	82
A.2	Add Two Different Members	83
A.3	Remove Same Member	83
A.4	Remove Two Different Members	84
A.5	Remove Member + Member Exit	84
A.6	Promote Same Member	85
A.7	Promote Two Different Members	85

A.8 Demote Same Member	86
A.9 Demote Two Different Members	86
A.10 Demote Each Other + Members	87
A.11 Demote Each Other + No Members	87
A.12 Remove Admins + Members	88
A.13 Remove Admins + No Members	88
Appendix B – More Evaluation Plots	89
B.1 Experiment 1	89
B.1.1 Insert Latency vs Number of Inserts	89
B.1.2 Push Latency vs Number of Inserts	91
B.2 Experiment 2	92
B.2.1 Raw Residual Swift Plots	92
B.2.2 Cumulative Residual Plots - Single Writer	93
B.2.3 CDF Residual Plots - Single Writer	94
B.2.4 Cumulative Residual Plots - Three Writers	95
B.2.5 CDF Residual Plots - Three Writers	96
Appendix C – Securing Group State and Operations	97
C.1 Approach I	97
C.1.1 Encrypting Blob Values	98
C.2 Approach II	99
C.2.1 Example i	99
C.2.2 Example ii	100
C.3 Approach III	101
C.4 Conclusion	101

1

Introduction

A conflict is a combination of concurrent updates, which may be individually correct, but that, taken together, would violate some invariant. - *Marc Shapiro, Nuno Preguiça, Carlos Baquero, Marek Zawirski.*

In this modern era, everyone is connected with technology, and the way of communication has changed a lot and is much simpler. We have software messaging applications that enable people to connect or communicate with one another on various devices, thanks to the current emerging technologies. There are well-known chat applications like WhatsApp, Facebook Messenger, Telegram, Discord, Signal, etc. [21] that everyone mostly uses. All of them provide common features like private messaging, group messaging where a user can broadcast their messages to the members of a particular group, phone/video calling, sharing files, etc.

These chat applications' design follows a conventional approach, which is centralization [11]. This implies that all the user's information, like contacts, messages, call logs, and shared data, is stored on the central server. Nevertheless, centralized chat apps have a few downsides that a user could potentially face, and these are discussed further. The most common and well-known downside of a chat application is *single point of failure*; if the central server fails, all the data is lost, leading to communication disruption. However, *to mitigate the issues of data loss* or *to make it failure resilience* [10], a few replication strategies are implemented, leveraging the concepts of distributed systems, where real-time data is now stored across multiple replicas of databases and also having multiple regional deployments to achieve high availability and fault tolerance [65]. To achieve this, it uses a distributed system concept known as *quorum consensus* [35], where a minimum number of nodes (quorum) must agree on a data state or a series of operations to achieve consistency, and a controller to manage operations like read/write requests, etc., across the nodes. Popular *quorum-based consensus* protocols like Paxos and Raft [55] were designed to maintain consistency and safety [51] between the replicas. In distributed chat applications, quorum-based consensus

is a must requirement to achieve fault tolerance. A few examples like *first*, having multiple copies of service registries that are responsible for service discovery, such as which chat servers are active or which chat servers handle messages, so the users can communicate. *Second*, sharing copies of group chat membership metadata across nodes. Group metadata is the data about the number of groups, metadata about each group, the name, and the description of the group. When performing group operations like adding or removing users from a group or updating the group chat information, all the replicas of the chat app need to have the same consistent state.

The distributed chat application assures that users will not face any downtime and that the service will be up and running 24/7. However, the distributed chat application itself has a major drawback. We need an infrastructure to deploy several replicas/nodes, moreover we need a bunch of servers to operate the backend of the chat application; this would incur costs. With many replicas/nodes, it becomes difficult to manage, and there would arise some latency issues due to inter-network node communication to achieve quorum-based consistency. The infrastructure supporting the distributed chat application should keep running; otherwise, users won't be able to access any chat features.

Other challenges that users might face are [9], *lack of privacy*; central servers have the potential to get hacked and can be vulnerable to data breaches. Lack of privacy also leads to data monitoring by the government or other authorities. There is a *lack of transparency and limited control over the data*, and we users entrust power to a central authority within the system, granting it the authority to make and enforce decisions, having no idea about how the data is being governed or used by these app developers. For example, in 2010 [45], Facebook user data was sold to the Cambridge Analytica firm for political advertisement. Although these challenges are not addressed in our P2P offline group messaging app.

Making clients independent of the server would enable cheaper operations, removing the need for distributed centralized servers, and becoming completely decentralized. Decentralization offers strong control over users' data; there is less risk of censorship, and it offers strong privacy. The management of nodes is reduced since all the nodes are now independent.

Chat applications built using blockchains are not ideal because of some limitations. Blockchains are not scalable; they have limited capacity and speed. Blockchain requires a lot of processing power and electricity to secure the network since it needs to process larger volumes of transactions, depending on the consensus protocol or network size.

Considering the drawbacks of both distributed chat apps and decentralized chat apps built on blockchains, there is a common drawback between them, which is consensus. Consensus can be time-consuming, especially in scenarios where there are latency issues or when a large number of nodes are involved. Achieving consensus typically involves multiple rounds of communication and agreement among distributed nodes to reach a consistent state or decision. In addition, running apps on blockchain is not free. One has to pay a small gas fee to run the application. Gas fee could be applicable for deploying your code on a blockchain, or when a user interacts with an app.

The idea of this thesis is to build an offline P2P group messaging application for small community groups, where the app is independent of central servers and doesn't require any infrastructure to keep running 24/7. Our target users are those who live in a region where

it is difficult to deploy servers. A few examples we would like to highlight about the people living in such regions where this offline group messaging app would be considered. For e.g;

Small Island nations[41] named *Kiribati, Tuvalu, Antigua, Maldives, etc*, have limited power supply and are vulnerable to climate change and rising sea level. It would be difficult and an irrational decision to deploy infrastructure to such regions. Hence, this app would be helpful to the people living in such regions, or a small group of tourists can make use of such an application to interact among themselves.

Geopolitically Sensitive regions[18] like *Ukraine or West Bank, and Gaza*, where there is an ongoing wars potentially have destroyed infrastructure, and it is difficult to deploy servers. Even if everything stabilizes, it would cost them to deploy servers. In such cases, they could use this app to connect with their friends and family.

The functional requirement of this thesis is to design and implement a peer-to-peer offline chat application focusing on group messaging. Every peer is a user who doesn't need to be online all the time, guaranteeing consistency of the state of the messaging groups across the remote peers, delivering the messages in total order, and having restrictive Byzantine fault tolerance, where the members of the groups are only allowed to send the messages.

In *chapter 2*, we present a detailed discussion of the background knowledge that was required to design and build this group messaging app.

The *chapters 3, & 5* cover the design and implementation of our chat application model.

To prove the app system correctness, we provided the detailed proofs of our application design in *chapter 4*.

The non-functional requirements of this thesis are to show how it scales in terms of the group size (number of members in a group), and how much storage/space it takes when n number of members are added inside the group. Moreover, we also tested how our group chat model replicates its state across different peers and presented how much traffic is generated, the total bytes transfer, and how the convergence takes place. In *chapter 6*, we have presented the experiment results of our group chat model.

In *chapter 7*, we discuss the related work of our thesis and the approaches and methods based on our application.

Finally, in *chapter 8*, we present the conclusion about our detailed research on our P2P offline group messaging app presented in the thesis and talk about future extensions that can be apply in our application.

2

Background

In this chapter, we discuss the background knowledge required for the design and implementation of the group messaging app.

2.1 Popular Group Messaging Apps

A group messaging app is a communication platform that allows broadcasting messages to members associated with a particular group. In this section, we discuss 7 popular messaging apps that support group messaging. In addition, we also present the features of the group membership model that are within the scope of our thesis. We also discuss the admin roles of each messaging app and mention the size of the group members.

2.1.1 WhatsApp

WhatsApp [48], a popular messaging platform, was founded by *Brian* and *Jan* in the year 2009, later acquired by *Meta(Facebook)*. WhatsApp supports one-to-one and group messaging, phone and video calling, etc. In the year 2022, they also added a new feature called *channels* which is designed for one-way broadcasts to potentially massive audiences. The app is available on both mobile phones(Android and iOS platforms), and on PCs. To create a WhatsApp account, one needs a valid mobile phone number.

2.1.1.1 Group Membership model

1. **Group operations:** It supports creating, adding/remove member, demoting/promote member or admins, change description of the group. Both admins and members can leave the group. Users can also delete messages, but it leaves a mark after the message is deleted. Users can also edit sent messages, but there is a time limit to edit sent messages; one can only edit the sent message within 15 minutes after sending. Edited messages are labeled with "Edited" text.
2. **Group size:** 1024 members.
3. **Admin roles:** A WhatsApp group can have N number of admins, there is no limit.

When there is only one admin and exits the group, randomly, a normal member of the group is promoted to an admin. Admins can remove and/or demote (or remove admin rights) from other admins, except they cannot demote/remove the original creator of the group. The creator of the group remains the admin until they leave or self-demote.

4. **Group names and description:** WhatsApp supports multiple groups with similar names and descriptions. Groups with similar names are distinguished by using a unique group ID. However, it is not mentioned anywhere on the WhatsApp documents site, but it is often discussed by experienced software engineers [11] who design the system of such group chat applications. Unique IDs [58], like UUIDs, Thread IDs, etc., are often used as an identifier for the groups.

2.1.2 Telegram

Telegram [43] is a Russian-based messaging app founded by *Pavel* and *Nikolai Durov* in 2013. It became popular because of privacy features like *secret chats*. In *secret chats*, conversation between one-to-one users are device-specific and are encrypted. Data is not stored on the cloud. Extra privacy features, like messages can't be forwarded, and screenshots are not available. However, secret chats are not available in group conversations and only meant for one-to-one conversation. The app is available on both mobile phones (Android and iOS platforms), and on PCs. To create an account on Telegram, one needs a valid mobile phone number.

2.1.2.1 Group Membership model

1. **Group operations:** It also supports creating, adding/remove member, demoting/promote member or admins, change description of the group. Both admins and members can leave the group. Users can also delete messages, but only own messages. Admin has full control over deleting everyone's messages. Users can too edit messages, but the messages should be edited within 48 hrs. Edited messages are labeled with "Edited" text.
2. **Group size:** 200,000 members.
3. **Admin roles:** A group in Telegram can have N number of admins; there is no limit. When there is only one admin and exits the group, the group becomes orphaned. Before exiting the group, the admin should promote a trustworthy member of the group. Admins can remove and/or demote other admins of the group, including the creator of the group.
4. **Group names and description:** Similar to WhatsApp, Telegram supports multiple groups with similar group names and description and they are distinguished by a unique group ID.

2.1.3 Microsoft Teams

Microsoft Teams [30] is not just a powerful group messaging application; it is more than that. It was developed for organizing group meetings, giving Zoom-like features like video calling, etc. The app is available on both mobile phones(Android and iOS platforms), and on PCs. To use Microsoft Teams, one needs to create an account using valid email address. One can also use their existing social accounts, like Google, Microsoft, or Apple for sign-up.

2.1.3.1 Group Membership model

1. **Group operations:** It also supports creating, adding/remove member, demoting/promote member or admins, change description of the group. Both admins and members can leave the group. Users can perform both delete and edit operations over the sent messages. There is no time limit to do that.
2. **Group size:** 250 for free version. For the enterprise version it can support up to 25000 members.
3. **Admin roles:** In Microsoft Teams, only 100 admins are allowed. When there is only one admin and exits the group, the group becomes orphaned similar to what happens in Telegram. Before exiting the group, the admin should promote a trustworthy member of the group. Admins can remove and/or demote other admins of the group, including the creator of the group.
4. **Group names and description:** Similar to other group messaging apps discussed above, Microsoft Teams too supports multiple groups with similar group names and description and they are distinguished by a unique group ID.

2.1.4 GroupMe

GroupMe[22] is another group messaging app owned by Microsoft. GroupMe was launched in May, 2010 by the company named GroupMe. It was created at a hackathon at the TechCrunch Disrupt conference. It was then quickly acquired by Skype in August 2011, making it a subsidiary of Microsoft later that same year. The app is available on both mobile phones(Android and iOS platforms), and on PCs. To create an account in GroupMe, one can use either a valid mobile phone number or a valid email address. Same like Microsoft Teams, one can also use their existing social accounts like Google, Facebook, Microsoft, or Apple for sign up.

2.1.4.1 Group Membership model

1. **Group operations:** It also supports creating, adding/remove member, demoting/promote member or admins, change description of the group. Both admins and members can leave the group. Users can only delete their own previously sent messages. But the sent messages are still visible to other members of the group. There is no feature to delete a sent message for everyone. Users can edit the sent messages, but there is a

time limit, and it's 15 minutes. Admins can delete messages for everyone. Admins can edit messages too, but the time-frame to edit messages is only 30 days.

2. **Group size:** Initially supported 100 members, but later in the year 2019, it now supports at-most 5000 members per group.
3. **Admin roles:** There is no limit in the number of admins present in a group in *GroupMe* app. When there is only one admin and exits the group, the group becomes orphaned similar to what happens in Telegram and Microsoft Teams. Before exiting the group, the admin should promote to a trustworthy member of the group. Similar to WhatsApp, admins can remove and/or demote (or remove admin rights) from other admins, except they cannot demote/remove the original creator of the group. The creator of the group remains the admin until they leave or self-demote.
4. **Group names and description:** Similar to other group messaging apps discussed above, GroupMe too supports multiple groups with similar group names and description and are distinguished by a unique group ID.

2.1.5 Signal

Another popular messaging app named *Signal* [40]. It is free, non-profit, open-source, and widely recognized for its robust privacy and security features. The app was developed by Open Whisper Systems in 2014. This app can be used on mobile phones and PCs, and it needs a valid mobile number to sign up.

2.1.5.1 Group Membership model

1. **Group operations:** It also supports creating, adding/remove member, demoting/promote member or admins, change description of the group. Both admins and members can leave the group. Users can only delete their own previously sent messages for everyone, and it should be done within the 3 Hrs of sent message. Users can also edit the message, but it should be edited within 24 Hrs of the sent message.
2. **Group size:** 1000 members.
3. **Admin roles:** There is no limit in the number of admins present in a group in *Signal* app. When there is only one admin and exits the group, the group becomes orphaned similar to what happens in Telegram, Microsoft Teams, and GroupMe. Before exiting the group, the admin should promote a trustworthy member of the group. Admins can remove and/or demote other admins. Creator or owner of the group doesn't have any special protected privileges.
4. **Group names and description:** Similar to other group messaging apps discussed above, Signal Teams too supports multiple groups with similar group names and description and are distinguished by a unique group ID.

2.1.6 WeChat

WeChat [47], another popular Chinese messaging app developed by *tencent* and first released in 2011. This app can be used on mobile phones and PCs, and it needs a valid mobile number to start messaging with WeChat.

2.1.6.1 Group Membership model

1. **Group operations:** It also supports creating, adding/remove member, demoting/promote member or admins, change description of the group. Both admins and members can leave the group. Users can't edit any previously sent messages, but they can only delete messages for everyone. The time-frame to delete a message for everyone is short and is only up-to 2 minutes.
2. **Group size:** 500 members.
3. **Admin roles:** There is a limit on the number of admins, and it's 3. When there is only one admin and exits the group, the group becomes orphaned similar to what happens in Telegram, Microsoft Teams, GroupMe, and Signal. Ownership doesn't automatically transfer, nor do admin roles; they must be re-assigned by the owner. Before exiting the group, the admin should promote to a trustworthy member of the group. Similar to WhatsApp and GroupMe, admins can remove and/or demote other admins except the creator of the group.
4. **Group names and description:** Similar to other group messaging apps discussed above, WeChat too supports multiple groups with similar group names and description and are distinguished by a unique group ID.

2.1.7 iMessage

iMessage [23] is another popular instant messaging app designed and developed by *Apple Inc.*. The apps can only be used by users who owns an Apple device like an iPhone, iPad, Mac, etc. One can only be use Apple ID to use this messaging app.

2.1.7.1 Group Membership model

1. **Group operations:** It also supports creating, adding/remove member, and change description of the group. Users can leave/exit the group. Users can edit a sent message for everyone within a time-frame of 15 minutes and can undo a sent message within time-frame of 2 minutes.
2. **Group size:** 30 members.
3. **Admin roles:** There is no admin concept in iMessage group chat. By default, all the members are admin in a group.
4. **Group names and description:** Similar to other group messaging apps discussed above, iMessage too supports multiple groups with similar group names and description and are distinguished by a unique group ID.

We would like to highlight one more important feature that we missed out is the *ordering of messages* in a group chat. Well, it's a hard problem to design such a system where all the members/admins view the same total order 2.4 of these messages. There might be a case of missing and/or out-of-order messages. But there are multiple ways how messages are delivered [12, 6.2 Message delivery guarantees] using server-side-timestamp where each message gets an authoritative timestamp by a server or messages can be ordered by the client side.

However, all the above-mentioned apps support group phone and/or video calling, but with a limited number of group members. Moreover, group members can also share files like text, images, GIFs, videos, etc. Each app supports different file sizes that are shared. Although these additional features are out of scope of our thesis.

One more feature we would like to discuss about these messaging apps is that a few of them also support channels. Channels are like a PUB-SUB model or a one-way broadcast tool for sending messages to a vast number of members present inside the channel. There is only a single creator of the channel who is responsible to manage the channel. Normal users/members can't publish anything inside the channel, but can react to the posts that were published by the creator of the channel. However, this feature is also out of the scope of our thesis.

In the above discussed apps, all the information, like chats and group information are stored in a central server, which means the data is managed and controlled by a single authority. And, there are a few downsides of this architecture as discussed in *chapter 1*.

2.2 Git

Git[19] is a version control system that tracks file changes inside a computer or server. It was initially designed and developed by Linus Torvalds. In general, it is a source code management tool used in software development. In brief, git creates a snapshot of your current repository that contains source code files, folders, and sub-folders. Each time a file is created or modified inside the repository, the developers use git to create a new snapshot. In addition, git stores the history of all the snapshots created by the git user. The developers can refer to the history to switch to a different snapshot or merge with their current repository. Online platforms like GitHub, GitLab, Bitbucket, etc., use git software that developers can use to store, manage, and share their code. They can also push their local repository on these platforms, where other developers can collaborate or contribute to software development. These platforms are also used for CI/CD workflow [14] to automate tasks. To automate tasks, these platforms use an advanced git tool called *hooks*, which is covered later in this section.

The git repository is like a database that stores the metadata and git db objects in a *.git* [19, p. 418] folder. The metadata includes the configuration of the repository, such as who is the author of the repository, who can commit, etc., and information about the remote git repositories. The git objects are the core part of the git internals and can be found in the *.git/objects* folder. These objects are stored in Zlib compressed format and are identified by their SHA-1 hash value. The hash values are also known as object IDs(OID) and are used

to look up the content of the git objects. The hash values are simply the computation of the content of the git objects (before Zlib compression). There are three types of git objects, and those are *blobs*, *trees*, and *commits*. The *blob* [19, p. 419] object only stores the content of the file, not the metadata of the file like name or permissions. The *tree* [19, p. 421] object is a sorted list of path entries used to store the directory structures. The information in the tree object includes the object types (blobs or trees), file modes (permissions), OID of the objects, and the names of the objects. Sub-directories are also referred to as tree objects, and they can recursively be pointed to by their root tree object. The *commit* [19, p. 424] objects are the snapshots of your git repository. Every time a commit is created, it points to the latest tree objects. The tree objects have files and directories that were staged (prepared to commit). In addition, commits store a list of parent commits (OIDs) corresponding to the previous snapshots. A commit with no parent is the root commit, and a commit with multiple parents is a merge commit. Commits also store metadata, such as the author and committer, including their name, email address, and date. At last, commits also require a commit message to describe the purpose of the snapshot. A git repository could have multiple commit logs, and it's impossible to remember all the OIDs of the latest commit object of those logs. Therefore, git has *references*. *References*, or *refs* [19, p. 429], are an alias to git commit OIDs and are an indirect way to refer to the latest git commit objects. It is an internal mechanism to represent the branches, which are stored in the *.git/refs* folder.

Git Hooks [19, p. 354] are powerful tools for automating your tasks. They are stored in the *.git/hooks* folder. These scripts are run when an event occurs either on the client side or server side. On the client side, when you are trying to create a commit message or want to push your latest snapshot to the server side, these scripts can be used to check if it is safe to do such operations. On the server side, when a push is received from the client side, these scripts can help to check whether it is safe to perform a merge operation on the server side.

We will use the git low-level APIs (plumbing commands) to build our chat applications on top of the git, which is explained in detail in the implementation chapter.

2.3 Conflict-Free Replicated Data Types

In distributed systems, achieving data consistency is not a trivial task. Multiple nodes/servers try to modify/update the same data concurrently. Without the coordination of these replicas, data inconsistency would be possible. To maintain data integrity and consistency between replicas, protocols like Raft and Paxos [55] were designed. But there is a catch: these protocols have communication overhead and high latency due to coordination since multiple replicas are involved. To resolve conflicts, real-time consensus is required, allowing at most $n/2-1$ nodes to be down. These protocols are unsuitable for offline operations. These protocols are unfit for collaborative apps where multiple users modify the data concurrently. We need a data structure where users not only modify data concurrently, but at the same time, they can also work offline on these applications, allowing them to access from different devices. Well, we have CRDTs. They are well-suited for use cases where concurrent modifications are common.

CRDT (Conflict-Free Replicated Data Type) [67] is a data structure used for replicating

data objects to multiple servers/systems over a network without requiring a consensus. Unlike centralized systems that rely on global coordination and complex conflict resolution protocols, CRDT allows clients to make updates concurrently and independently. CRDT ensures *SEC* (Strong Eventual Consistency) [67, p. 5]. *SEC* is *Eventual Consistency* + *Strong Convergence* [51], which is a consistency model in Distributed Systems that guarantees both *liveness* and *safety* properties. *Eventual Consistency*, updates are made first locally and then propagated to all the replicas. Data in eventual consistency is not recent, client might read a stale value. Not all the correct replicas will have the latest value, but eventually get the same updated value. *Strong Convergence*, all the correct replicas that receive updates (*unordered*) will eventually converged to an equivalent state. With *SEC*, the replicas don't require consensus, and updates can be in any order because *SEC* eliminates conflicts. With no conflicts, then no need to rollback, because all the states will eventually converge. If we loosen SC [51], *SEC* consistency model allows $n-1$ nodes to be temporarily down without preventing operations from completing.

2.3.1 CvRDT - State-based CRDT: Convergent Replicated Data Type

State-based CvRDT is *passive replication*. Here, the updates first occur locally, then these states are propagated to the replicas, which then perform a *merge* operation to get a final state. Propagation does not happen immediately, but eventually whenever a push is made from the local to the remote state. [67, p. 10]

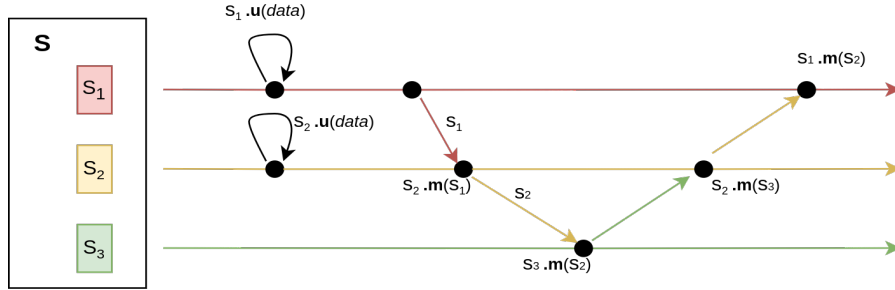


Figure 2.1: State-Based Replication [67]

In state-based CRDT, each replica is a tuple object that is (S, s^0, q, u, m) [67, p. 4]. S is a set of the states of replicas, s^0 is the initial state of a replica which is $\{\}$, q is the query function that queries for the state of a replica, u is the update function that updates the current local state of a replica to a newer state, and m is the merge function that merges the remote state to its local state that was received from remote replica.

In CvRDT, a state-based object when equipped with partial order $\preceq$ is called a monotonic semilattice object [67, p. 6]. The possible payloads in set S forms a semilattice, i.e, ordered by *partial order* $\preceq$. When two states merge, the state-based CRDT computes LUB (Least Upper Bound) or the supremum bound, that is, $\text{merge}(s, s') = s \sqcup s'$. *Because* it is the LUB of two values, the merge function automatically provides the following properties:

1. **Commutative** - $x \sqcup_v y = y \sqcup_v x$, doesn't matter which one is the source and remote

replica when merging.

2. **Associative** - $(x \sqcup_v y) \sqcup_v z = x \sqcup_v (y \sqcup_v z)$, doesn't matter the sequence of states while merging.
3. **Idempotent** - $x \sqcup_v x = x$, merging the same value multiple times will eventually give the same state.

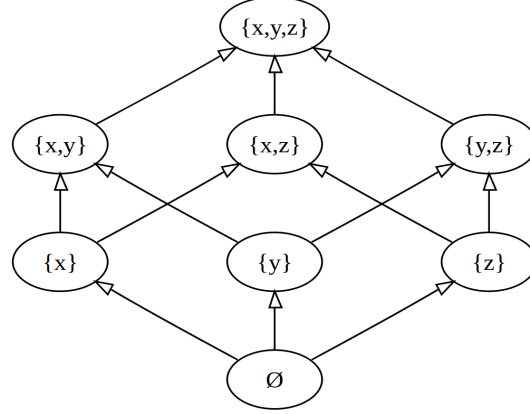


Figure 2.2: Semi-Lattice [67]

Updates are monotonically increasing, i.e., $s \preceq \text{update}(s, u)$. The semi-lattice property ensures that all the replicas will eventually converge without worrying about delays, or messages lost during transmission.

2.3.2 Causal-Length CRDT

Causal-Length CRDT [62, 68] is a state-based CRDT used for adding and removing elements from a set efficiently. It supports infinite additions and removal of elements. This CRDT is an extension of 2P-SET [63, p. 22], where every element in CL-CRDT is associated with metadata (*causal value*). The metadata is simply an integer value that helps to track the presence of an element in a set. *But what makes CL-CRDT better?*

There are different flavors of state-based CRDT that support insertion and deletions of elements. For e.g., in 2P-Set [63, p. 22], there is an *add* set and *remove* set. When an element is inserted, it is added to the add set, and when an element is removed, it is added to the remove set. The remove set is like a tombstone that helps to track the deleted element. To query the elements, we simply take the difference between the add and remove sets. Downside of the 2P-Set is that the removed element cannot be added again.

OR-Set (*Observed-Remove Set*) [52], is another type of state-based CRDT for addition and removal of elements. Here, a previously removed element can be added again. OR-Set is an extension of 2P-Set. It has both an add and remove set; the only difference is that when an element is inserted, the element is added inside the add set with a token value. This token value helps to track the presence of an element inside a set. When an element is deleted, the element with the token value is added inside the remove set (*tombstone*). The

removed element can be added again inside with a different token value. The downside of this CRDT is that it increases the storage size of the remove set (*tombstone*) because it contains old elements that were previously removed.

CL-CRDT is better because it only requires a single set to store the elements, no additional remove set (*tombstone*). The causal value associated with the elements is sufficient to track the existence of element. The *odd* causal value indicates that the elements is present inside set, and the even value indicates that the element is absent. Below we explain the data structure and the operations of CL-CRDT.

Following the presentation of the equivalent ∞ P-Set [62], we may view the CL-CRDT as a dictionary D , where the keys of the dictionary are the elements that can be stored in a set, and the value of each key is an integer (*causal length*). The notation to access the value of a key is $D[e]$, which returns the integer value.

1. $D' = \text{add}(D, e)$ - This function adds the element in D and returns a new dictionary D' . If $e \notin D$, then $D[e] \leftarrow 1$. If $e \in D$ and $D[e]$ is even, we increment the key value by 1 to make it odd. If $e \in D$ and $D[e]$ is odd, no change in the key value, and $D = D'$.
2. $D' = \text{remove}(D, e)$ - This function removes the element from D and returns a new dictionary D' . If $e \notin D$, nothing to do, hence $D = D'$. If $e \in D$ and $D[e]$ is odd, we increment the key value by 1 to make it even. If $e \in D$ and $D[e]$ is even, no change in the key value, and $D = D'$.
3. $D'' = \text{merge}(D, D')$ - In merge, we set the keys of D'' by taking the union of $\text{keys}(D)$ and $\text{keys}(D')$. We then loop through the keys of D'' and compare the key values of both the dictionary D and D' by taking the max value and setting it to the $D''[e]$.
4. $S = \text{query}(D)$ - This function returns all the elements present inside the dictionary. It only returns those elements whose causal-length is an odd integer.

The above function satisfies *ordering*, *monotonicity*, and *Least-Upper Bound* properties. For more details about the proofs, refer *State-Based ∞ P-Set Conflict-Free Replicated Data Type* by Erick Lavoie [62].

2.3.2.1 CL-CRDT: Use Case Scenario

In this section, we illustrate a use case scenario where CL-CRDT can be implemented. Let us assume a user wants to purchase grocery from an online store, but before making the final payment, the user is adding the list of items to the shopping cart.

The shopping website has 3 replicas that will store the names of the items.

Each item is associated with the causal-length value, and we show them as a superscript on the name of the item, e.g., $\text{item}^{\text{integer}}$.

Initially, all the replicas have the same empty state. The user first adds eggs in the shopping cart, where the first replica (R1) was triggered to perform an add operation by adding eggs with the metadata value of 1 in its local state and then sending its states to all the replicas.

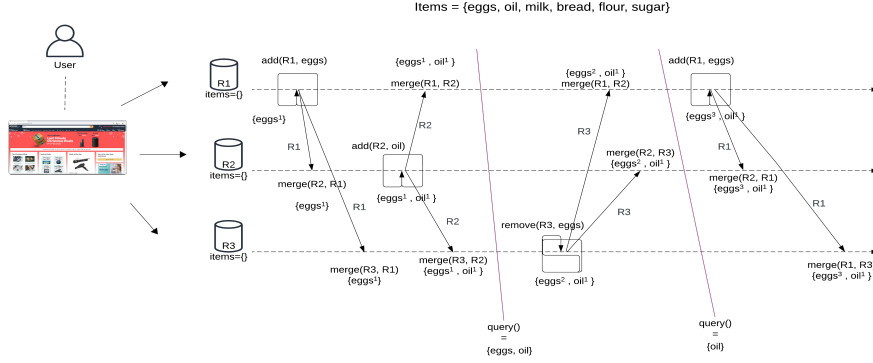


Figure 2.3: Flow Diagram of Shopping Cart Using CL-CRDT

Then user adds oil, and the first operation was performed in replica (R2) and was propagated to other replicas. The shopping cart contains $\{eggs^1, oil^1\}$.

The user now wants to remove the egg, and the removal operation was performed by replica (R3). The replica R3 will increment the metadata of eggs, which is now $\{eggs^2, oil^1\}$.

When querying the items, the replica will check for the odd metadata value and will return the list of items. After removal of eggs, if the query is performed the returned list contains only {oil}.

2.4 Ordering

Ordering [59] in distributed systems tells us the execution of events/processes in a correct order. Ordering of events are crucial because it ensures correctness and consistency. Events may occurs across different nodes where there is no shared clock, and this can be challenging to ensure the correct sequence of events.

There are two types of ordering:

1. **Total Ordering** - Events occurred in a specific order, which is based on what time they've occurred. In distributed systems, if all the nodes agree upon a specific order, then it is a total order.
2. **Partial Ordering** - Partial order is a more relaxed way than total order. Events are only ordered when there is a causal relationship. It means that if an event A happens before event B, then everyone should agree on that order. It is denoted as $a \rightarrow b$. In a partial order, some of the events are independent and are incomparable. In a partial order, events can occur concurrently, denoted as $a || b$. It means, Either $a \rightarrow b$ or $b \rightarrow a$.

There is a challenge in capturing causal relationship between the events. Therefore, Lamport introduced, Lamport timestamps. In distributed systems, to know the causal relationship, every node/site is associated with a positive integer clock value. Whenever an event has occurred in a node, the node increments its causal value by 1. We compare the events between the nodes by taking the maximum value of the counter clock to achieve causal ordering.

2.5 Epidemics and their ways...

In this section [53], we discuss how data is replicated or updated among several different nodes/sites and also present the types of communication between sites. We also discuss the types of nodes/sites, i.e., the behavior of sites when receiving an update.

2.5.1 Types of Nodes

[53, p09]

1. **Susceptible** - Node that has not yet received any updates or data objects from other sites.
2. **Infective** - Node has received an update and is willing to share it with other sites/nodes.
3. **Removed** - Node has updates but is no longer willing to share the updates with other sites/nodes.

2.5.2 Types of Communication

1. **Direct Email** - Once a node/site receives an update, it will then send the update to each and every site it knows. This is a deterministic way to send updates to all the sites. Replication method is fast, but there is a huge network load due to disseminating updates to every site. This approach is not always reliable since there is no fixed quorum of nodes/sites. It is not very scalable because of its time complexity, which is $O(n^2)$, since each site updating to every other site.
2. **Anti-Entropy** - This is a randomized approach where a site randomly chooses another site and exchange the database contents. It not only exchanges the database contents, but it also tries to resolve the differences between the database contents. The approach is very expensive to run, because it tries to resolve conflicts. Although there are ways to optimize. This approach is an SI model where, a node is always susceptible or infective.
3. **Rumor mongering** - This is also a randomized approach where the site randomly chooses another site for exchange. But it's a little different from anti-entropy. When the site tries to update database contents with a different site, and if the site has already seen the update, then it will no longer try to exchange the data. This approach is a SIR model where, a node can be susceptible, infective, or removed.

2.5.3 Strategies for Epidemics

1. **Push** - A site only pushes its DB contents. Infective nodes are the ones infective susceptible to infection. Efficient when there are few updates.
2. **Pull** - A site only pulls data from other sites. Efficient when there are many updates.
3. **Pull-Push** - A site does both pull and push operations.

3

Design - Group Chat App

This chapter discusses the design of a peer-to-peer offline chat application. We focus on group messaging, where members of a particular group can send or receive messages simultaneously. In *chapter 2*, we have presented a brief knowledge about the group messaging applications 2.1 and their group membership model. They have a few limitations discussed in *Chapter 1*. One of the major drawbacks is that the systems/servers of these apps should keep running even if the internet is down or censored. The deployment of such a chat application needs an infrastructure and multiple resources that would incur costs. Our goal is to design a low-cost decentralized group messaging app that the client doesn't rely on the central servers that are online 24/7. In our group messaging app, we employ CL-CRDT (2.3.2) data structure to manage the state of the messaging group and the members of the group need not be online all the time and still manage to get the entire state of the modified group (latest messages in total order [59, p. 3], newly added members or removed old members, etc.).

The functional requirements of our group chat model closely resemble those of these chat applications 2.1, and that is, a group should have at least one admin. An admin can add or remove members from a group. Members can exit the group. Admin has the capabilities to make other members admin and vice-versa. Multiple groups can have the same name, and/or members can be in multiple groups with the same name. However, as discussed in 2.1, that messages don't guarantee ordering, but in our design, we will present our idea about the messages appearing in a total order [59, p. 3].

In the upcoming section, we present a brief overview of the APIs that match the functional requirements of our group messaging application. In the end, we give a detailed explanation of these APIs, explaining the pseudo-code and how we implement the CL-CRDT[68] data structure.

3.1 Notations

Before heading towards the API and pseudo-code section, we first describe the notation and the definitions that are used in order to make the context easier to understand.

Notation	Meaning
<i>name</i>	Name of the group
<i>description</i>	Description of the group
<i>m</i>	Member to add/remove from group
<i>a</i>	Member that performs group operations
$\mathbb{M}$	Set of members
$\mathbb{A}$	Set of admins
ν	A positive counter
D	A dictionary representing CL-CRDT [68]
$\mathbb{X}$	A grow-only set[63] to store messages
$\mathbb{G}$	A group object/state

Table 3.1: Notations for APIs

3.1.1 Definitions

1. D - A dictionary representing CL-CRDT[68] that maps to a tuple of size 2 that stores positive counter values, whereas the keys are of *string* data-type. These counters are referred to as causal values [68]. The first tuple value indicates the presence of a member inside a group, and the second tuple value indicates whether the member is an admin or not. To fetch the tuple value, we need to pass the key to the dictionary, and it returns $(\mathbb{Z}_+, \mathbb{Z}_+)$. To access the values of a tuple from a corresponding key, it is illustrated as $D[key]_0$ and $D[key]_1$. The $D[key]_0$ fetches the first value of the tuple, whereas the $D[key]_1$ fetches the second value of the tuple.
2. $\mathbb{X}$ - A set which is a grow-only set[63] that stores messages. The values of set is a tuple of size 2. The first value of tuple is a 64-bit integer datatype that stores the Unix time in nanoseconds [46], whereas the second value of tuple is of a string datatype. The format of the second tuple value which is a string is $\langle name \rangle: \langle message \rangle$. The *name* is the member of a group and *message* is a piece of information exchanged between group members. The idea behind storing the Unix timestamp with messages is to identify the ordering of messages. However, Unix timestamp does not always guarantee a correct or causal ordering [31, 32] because client clocks are not perfectly synced. The time may drift or jump, and network latency can reorder message delivery. The syntax to fetch values from set is X_i . It returns a tuple. To access the first value of tuple is depicted as X_{i0} , and for the second value is X_{i1} .
3. $\mathbb{G}$ - It is the group state/object that stores information of a group which is a tuple of 5 values. The tuple consists *name*, *description*, ν , D and $\mathbb{X}$. The first two values are of *string* data-type that are *name* and *description*. The *name* represents the name of a group, and *description* describes the role of the group. The *third* value is the Lamport clock counter [59] that stores a positive integer value. Whenever a group is created, the initial value of ν will always be 1. This counter is incremented when an admin modifies the *name* and *description*. This counter value helps to track the changes made in the description of a group. The *fourth* value dictionary D is the CL-CRDT [68]. The *final* value is the set ($\mathbb{X}$) that stores the messages. These **five** values are stored inside a tuple $\mathbb{G}$.

3.2 APIs

This section describes the APIs that are required to function in our group chat application.

- $\mathbb{G} \leftarrow \text{create_group}(\text{name}, \text{description})$
The API takes two inputs, *name* and *description*. The API creates a group object and returns $\mathbb{G}$.
- $\mathbb{G}' \leftarrow \text{add_member}(\mathbb{G}, a, m)$
The API takes three inputs, $\mathbb{G}$, *a*, and *m*. The API first checks if the current member (*a*) is present inside the group ($\mathbb{G}$) and has privileges to perform group operations. If this condition is satisfied, the API modifies the group state by adding a new member *m* inside a group and returns the modified group state $\mathbb{G}'$.
- $\mathbb{G}' \leftarrow \text{remove_member}(\mathbb{G}, a, m)$
The API takes three inputs, $\mathbb{G}$, *a*, and *m*. The API first checks if the current member (*a*) is present inside the group ($\mathbb{G}$) and has privileges to perform group operations. If this condition is satisfied, the API modifies the group state by removing the existing member *m* (if present) and returns the modified group state $\mathbb{G}'$. This API is also used for exiting the group.
- $\mathbb{G}' \leftarrow \text{rename_group}(\mathbb{G}, \text{new-name}, \text{new-description})$
The API is used to rename the group. It takes three inputs, $\mathbb{G}$, *name'*, and *description'*. It first checks if the current member (*a*) is present inside the group ($\mathbb{G}$) and has privileges to perform group operations. If this condition is satisfied, the API modifies the group state by setting up the new name and new description of the group and returns a modified group state $\mathbb{G}'$.
- $\mathbb{G}' \leftarrow \text{promote}(\mathbb{G}, a, m)$
The API takes three inputs, $\mathbb{G}$, *a*, and *m*. The API first checks if the current member (*a*) is present inside the group ($\mathbb{G}$) and has privileges to perform group operations. It then checks if the member (*m*) is present inside the group ($\mathbb{G}$). If both conditions are satisfied, then the API modifies the group state by promoting *m* to an admin and returns the modified group state $\mathbb{G}'$.
- $\mathbb{G}' \leftarrow \text{demote}(\mathbb{G}, a, m)$
The API takes three inputs, $\mathbb{G}$, *a*, and *m*. It first checks if the current member (*a*) is present inside the group ($\mathbb{G}$) and has privileges to perform group operations. It then checks if the member (*m*) is present inside the group ($\mathbb{G}$). If both conditions are satisfied, then the API modifies the group state by demoting *m* to a normal member of the group and returns the modified group state $\mathbb{G}'$. This API can also be used to self-demote, dropping all the admin privileges.
- $\mathbb{G}' \leftarrow \text{send_grp_msg}(\mathbb{G}, \text{message}, m)$
The API is used to send a message to a group by taking three inputs, $\mathbb{G}$, *message*, and *m*. First, it checks if *m* is present inside the group ($\mathbb{G}$). If the member *m* is present in

the group, then the message is added inside the set $\mathbb{X}$ that stores messages with the latest Unix timestamp value. Finally, returns a modified group state $\mathbb{G}'$.

- $(\mathbb{M}, \mathbb{A}, \mathcal{X}) \leftarrow \text{get_group_info}(\mathbb{G}, a)$

The API is used to get the group information. It takes only one input $\mathbb{G}$ and returns a tuple. The tuple consists of two string objects, two sets $\mathbb{M}$, and $\mathbb{A}$, and one list $\mathcal{X}$.

3.2.1 Group Operation Table

Before proceeding to the pseudo-code section, we present a high-level overview of the APIs that a user can perform. A user can be an admin or a group member or a non-group member.

Table 3.2: Group Operations Table

APIs	Admin	Group Member	Non-group Member
add_member	✓	✗	✗
remove_member	✓	✗	✗
remove_member (exit)	✓	✓	✗
rename_group	✓	✗	✗
promote	✓	✗	✗
demote	✓	✗	✗
demote (exit)	✓	✓	✗
get_group_info	✓	✓	✓
send_grp_msg	✓	✓	✗

Table 3.2 is self-explanatory; the *admin* of a particular group has the authority to perform or call all group operation APIs. The *normal member* of a group can send group messages, get information about the group, and exit the group. The *non-group member* that was removed previously/exited can only get the information about the group.

3.3 Pseudo-code

In this section, we present a detailed explanation of our APIs and how they internally work with the help of pseudocode.

3.3.1 Creating a Group

The create group function takes two inputs *name* and *description*. Internally, it creates an empty dictionary D , set the counter value ν to zero, and creates an empty set $\mathbb{X}$. This function returns a tuple of 5 values, which is the group state/object $\mathbb{G}$.

Algorithm 1 Group Creation

```

1: function CREATE_GROUP(name, description)
2:    $D \leftarrow \{\}$  ▷ Dictionary to store members
3:    $\nu \leftarrow 0$  ▷ Counter
4:    $\mathbb{X} \leftarrow []$  ▷ set of messages
5:   return (name, description,  $\nu$ ,  $D$ ,  $\mathbb{X}$ )
6: end function

```

3.3.2 Adding Members

The add function takes three inputs: *first* is the group state $\mathbb{G}$, which is a tuple of five values, *second* is the current member (a) of the group, and *third* is a member m to add. The function first checks if the current member (a) is present inside the existing group that wants to perform group operations, and also whether the current member is the admin of the group or not. If the first condition fails, we return an unchanged group state.

Algorithm 2 Add Members

```

1: function ADD_MEMBER((name, description,  $\nu$ ,  $D$ ,  $\mathbb{X}$ ), a, m)
2:   if  $a \notin \text{keys}(D)$  or  $D[a]_0$  is even or  $D[a]_1$  is even then ▷  $a$  is not in  $D$  and not an admin
3:     return (name, description,  $\nu$ ,  $D$ ,  $\mathbb{X}$ ) ▷ No change in state
4:   end if
5:    $D' \leftarrow \text{copy}(D)$ 
6:   if  $m \notin \text{keys}(D')$  then
7:      $D'[m] \leftarrow (1, 0)$ 
8:   else if  $D'[m]_0$  is even then
9:      $D'[m]_0 \leftarrow D'[m]_0 + 1$ 
10:    if  $D'[m]_1$  is odd then
11:       $D'[m]_1 \leftarrow D'[m]_1 + 1$ 
12:    end if
13:  end if
14:  return (name, description,  $\nu$ ,  $D'$ ,  $\mathbb{X}$ )
15: end function

```

If the admin (a) is present, we further proceed to add the m inside the group. We create a copy of the dictionary D and store it in D' . We then check if the key member (m) is present in the D' ; if not, a new key is inserted in the dictionary D' , setting the tuple (size=2) value to (1, 0). The first value of the tuple is a positive odd counter that indicates the member is present inside a group, and the second value is a positive even counter that indicates the member is a regular member of a group (not an admin). In the other case, if the key member (m) was already present in the group but its counter value is even, it indicates that the member was previously removed from the group. We fetch the counter value and increment it by 1 to make it odd. This indicates that the member is re-added to the group. With that, we also check to see if the previously removed member's admin value (2nd tuple value) is odd or even. This condition was necessary, and we have presented the exact reason in the section 3.4. Make sure to go through all the APIs' logic and then refer to this section 3.4. Finally, we return the modified group state, which is a tuple of *name*, *description*, ν ,

D' , and $\mathbb{X}$ with non-primed values indicating they did not change.

3.3.3 Removing Members

The remove function has two roles: either you can exit the group, or you can remove members from a group. The function takes three inputs: *first* is the group state $\mathbb{G}$, which is a tuple of five values, *second* is the current member (a) of the group, and *third* is the member (m) to remove. Initially, we make a copy of our dictionary to D' .

Algorithm 3 Remove Members

```

1: function REMOVE_MEMBER( $(name, description, \nu, D, \mathbb{X}), a, m$ )
2:    $D' \leftarrow copy(D)$ 
3:   if  $a \notin keys(D)$  or  $D[a]_0$  is even then                                 $\triangleright a$  is not in  $D$ 
4:     return  $(name, description, \nu, D, \mathbb{X})$ 
5:   end if
6:   if  $a = m$  then                                                         $\triangleright$  Self-exiting
7:      $D'[a]_0 \leftarrow D'[a]_0 + 1$ 
8:     if  $D'[a]_1$  is odd then
9:        $D'[a]_1 \leftarrow D'[a]_1 + 1$ 
10:    end if
11:  else                                                                     $\triangleright$  Removing other member
12:    if  $D[a]_1$  is even then                                                 $\triangleright a$  is not an admin
13:      return  $(name, description, \nu, D, \mathbb{X})$ 
14:    end if
15:    if  $m \in keys(D')$  and  $D'[m]_0$  is odd then                             $\triangleright m$  is currently in the group
16:       $D'[m]_0 \leftarrow D'[m]_0 + 1$ 
17:      if  $D'[m]_1$  is odd then                                                 $\triangleright m$  is an admin, demote
18:         $D'[m]_1 \leftarrow D'[m]_1 + 1$ 
19:      end if
20:    else                                                                     $\triangleright m$  not in the group
21:      return  $(name, description, \nu, D, \mathbb{X})$ 
22:    end if
23:  end if
24:  return  $(name, description, \nu, D', \mathbb{X})$ 
25: end function

```

The function first pre-checks if the current member (a) is present inside the existing group that wants to perform group operations. If the condition fails, we return an unchanged group state. The first condition of the remove function checks if a and m are equal or not. If they are equal, then the current member a wants to exit the group. We fetch the first tuple value from the dictionary D' for the key a , and increment it by 1 to make it even. In this first condition, we have a nested condition that checks if the member is an admin or not. If it is an admin, then we also increment the second tuple value by 1 to make it even. This condition is mandatory because if the admin exits the group, their privileges are taken, and they cannot perform any group operations.

In the second condition of the remove function, a and m are not equal, it indicates that

the current member a wants to remove the member m from a group. Inside the second condition, we first check if a has the privileges to remove the member from a group. If not, we return the unmodified group state. If the current member a is in fact an admin, we proceed further removing the member m from the group. Before removing, we check if the member m is present inside the group; if not we return the unchanged group state. If the member m exists inside the group, we fetch the first tuple value from D' and increment by 1 making it even. We also check if the member is an admin or not. If it is an admin, we fetch the second tuple value from D' and increment it by 1, making it an even value. Finally, we return the modified group state, which is a tuple of $name$, $description$, ν , D' , and $\mathbb{X}$ with non-primed values indicating they did not change.

3.3.4 Modifying the group name and description

The rename group function take four inputs, *first* is the group state $\mathbb{G}$ which is a tuple of five values, *second* is the new name of the group, *third* is the new description of the group, and *fourth* is the current member (a) of the group. The function first checks if the current member is present inside the group and is an admin. If not, we return an unchanged group state. If the condition is satisfied, we set the new name to the $name'$ variable and the new description to the $description'$ variable. We also increment the ν value by 1 and set it to ν' .

Algorithm 4 Renaming Group

```

1: function RENAME_GROUP( $(name, description, \nu, D, \mathbb{X})$ ,  $new-name$ ,  $new-description$ ,  $a$ )
2:   if  $a \notin keys(D)$  or  $D[a]_0$  is even or  $D[a]_1$  is even then  $\triangleright a$  is not in  $D$ , and not an admin
3:     return  $(name, description, \nu, D, \mathbb{X})$ 
4:   end if
5:    $name' \leftarrow new-name$ 
6:    $description' \leftarrow new-description$ 
7:    $\nu' \leftarrow \nu + 1$ 
8:   return  $(name, description, \nu, D', \mathbb{X})$ 
9: end function

```

Finally, we return the modified group state, which is a tuple of $name$, $description$, ν , D' , and $\mathbb{X}$ with non-primed values indicating they did not change.

3.3.5 Promoting Member

The functions take three inputs, *first* is the group state $\mathbb{G}$ which is a tuple of five values, *second* *second* is the current member (a) of the group, and *third* is a member m . The function first checks if the current member (a) is present inside the existing group, and if the current member a is the group admin who wants to promote the member m to an admin. If the first condition fails, we return an unmodified group state. If a is present inside the group and is an admin, we further promote the member m . We create a copy of the dictionary D and store it in D' . We then check if the key member (m) is present in the D' and is not an

admin. If the condition is satisfied, we fetch the second counter value of the tuple from D' and increment it by 1 to make it odd. This indicates that the member m is now an admin of the group.

Algorithm 5 Promote to Admin

```

1: function PROMOTE( $(name, description, \nu, D, \mathbb{X}), a, m$ )
2:   if  $a \notin keys(D)$  or  $D[a]_0$  is even or  $D[a]_1$  is even then  $\triangleright a$  is not in  $D$ , and not an
   admin
3:     return  $(name, description, \nu, D, \mathbb{X})$ 
4:   end if
5:    $D' \leftarrow copy(D)$ 
6:   if  $m \in keys(D')$  and  $D'[m]_0$  is odd and  $D'[m]_1$  is even then  $\triangleright m$  is in  $D$ , and not
   an admin
7:      $D'[m]_1 \leftarrow D'[m]_1 + 1$ 
8:   end if
9:   return  $(name, description, \nu, D', \mathbb{X})$ 
10: end function

```

Finally, we return the modified group state, which is a tuple of $name$, $description$, ν , D' , and $\mathbb{X}$ with non-primed values indicating they did not change.

3.3.6 Demoting Admin

This function has two roles: either you can self-demote and become a normal member of a group, or you can demote admins to become normal members of the group. The functions take three inputs, *first* is the group state $\mathbb{G}$, which is a tuple of five values, *second* is the current member (a) of the group, and *third* is the member (m). Initially, we make a copy of our dictionary to D' . The function first checks if the current member (a) is present inside the existing group that wants to perform group operations. If the condition fails, we return an unmodified group state.

In the first condition of the demote function, if a is equal to m , then a is an admin and wants to self-demote. We fetch the second value of the tuple from D' and increment it by 1 to make an even value. This indicates that the current member a is no longer an admin of the group.

In the second condition, current member a is demoting member m . Before demoting, we check if the current member is an admin of the group; if not, we return an unchanged group state. If the current member a is an admin, we further demote member m . We check if the member m is present inside the group and is an admin. If the condition is satisfied, we fetch the second tuple value for the member m and increment it by 1, making it an even number. This indicates that the member m is no longer an admin. Finally, we return the modified group state, which is a tuple of $name$, $description$, ν , D' , and $\mathbb{X}$ with non-primed values indicating they did not change.

Algorithm 6 Demote Admin

```

1: function DEMOTE( $(name, description, \nu, D, \mathbb{X}), a, m$ )
2:    $D' \leftarrow copy(D)$ 
3:   if  $a \notin keys(D)$  or  $D[a]_0$  is even then  $\triangleright a$  is not in  $D$ 
4:     return  $(name, description, \nu, D, \mathbb{X})$ 
5:   end if
6:   if  $a = m$  then  $\triangleright$  Self-Demote
7:      $D'[a]_1 \leftarrow D'[a]_1 + 1$ 
8:   else  $\triangleright$  Demoting other
9:     if  $D'[a]_1$  is even then  $\triangleright a$  not an admin
10:      return return  $(name, description, \nu, D, \mathbb{X})$ 
11:    end if
12:    if  $m \in keys(D')$  and  $D'[m]_0$  is odd and  $D'[m]_1$  is odd then  $\triangleright m$  is in  $D$ , and
       $m$  is an admin
13:       $D'[m]_1 \leftarrow D'[m]_1 + 1$ 
14:    end if
15:  end if
16:  return  $(name, description, \nu, D', \mathbb{X})$ 
17: end function

```

3.3.7 Merging Group States

This merge function is part of our gossip and anti-entropy algorithm 2 for reconciliation of the CRDT data objects. The functions take two input parameters: the group state objects G and G' . The first group state (G) is the current local state of the client, and the second group state (G') is the received group state from other peers. We initialize the empty string to $name''$ and $description''$, set the counter value ν'' to 0, and an empty set $\mathbb{X}''$. We first merge the group name and description by comparing the counter values ν and ν' . If ν is greater than ν' , we pick the $name$, $description$, and ν from the first group, else we choose $name'$, $description'$ and ν' from the second group. In other cases, it could be possible that both the counter values are equal (concurrent). In such a case, we concatenate $name$ and $description$ of the first group and $name'$ and $description'$ of the second group, and compute their hash (SHA1) value. We now compare the hash values in lexicographical order. If the first hash value is greater than the second hash value, we choose the $name$, $description$, and ν from the first group, else we choose $name'$, $description'$, and ν' from the second group. In the case where both the hash values are the same, they are equal; therefore, we arbitrarily choose the receiver's side $name'$ and $description'$.

Now we merge the messages by taking the union of the local set of messages $\mathbb{X}$ and the received set of messages $\mathbb{X}'$. At last we merge the group members. We initialize a new empty dictionary D'' and a set K'' that is the union of the members of the first and the second group. For each member (m) in K'' , if m is present in both the groups, we compare their tuple values by taking the max of both the first and the second values, and setting it to $D''[m]$. If m is only in the first group (D), we set the tuple value of the first group (D) to D'' ; else if it is only present in the second group (D'), we set the tuple value of the second group (D') to D'' .

Algorithm 7 Merge Group State

```

1: function MERGE_GROUP_STATE(
    (name, description,  $\nu$ , D,  $\mathbb{X}$ ),
    (name', description',  $\nu'$ , D',  $\mathbb{X}'$ ) )
2:   name''  $\leftarrow$  ""
3:   description''  $\leftarrow$  ""
4:    $\nu'' \leftarrow 0$ 
5:    $\mathbb{X}'' \leftarrow \{\}$ 
6:   if  $\nu > \nu'$  then ▷ Merging group name and description
7:     name''  $\leftarrow$  name
8:     description''  $\leftarrow$  description
9:      $\nu'' \leftarrow \nu$ 
10:  else if  $\nu < \nu'$  then
11:    name''  $\leftarrow$  name'
12:    description''  $\leftarrow$  description'
13:     $\nu'' \leftarrow \nu'$ 
14:  else
15:    if  $\text{sha1}(\text{concat}(\text{name}, \text{description})) >_{\text{lexicographic}} \text{sha1}(\text{concat}(\text{name}', \text{description}'))$  then
16:      name''  $\leftarrow$  name
17:      description''  $\leftarrow$  description
18:       $\nu'' \leftarrow \nu$ 
19:    else
20:      name''  $\leftarrow$  name'
21:      description''  $\leftarrow$  description'
22:       $\nu'' \leftarrow \nu'$ 
23:    end if
24:  end if
25:   $\mathbb{X}'' \leftarrow \mathbb{X} \cup \mathbb{X}'$  ▷ Merging Messages
26:  D''  $\leftarrow \{\}$ 
27:  K''  $\leftarrow \text{keys}(D) \cup \text{keys}(D')$ 
28:  for  $m \in K''$  do ▷ Merging group members
29:    if  $m \in \text{keys}(D) \wedge m \in \text{keys}(D')$  then
30:      D''[m]  $\leftarrow$  ( $\max(D[m]_0, D'[m]_0), \max(D[m]_1, D'[m]_1)$ )
31:    else if  $m \in \text{keys}(D)$  then
32:      D''[m]  $\leftarrow$  D[m]
33:    else if  $m \in \text{keys}(D')$  then
34:      D''[m]  $\leftarrow$  D'[m]
35:    end if
36:  end for
37:  if  $\forall (m, D''[m]_0, D''[m]_1) \in D : D''[m]_0 \text{ is odd} \wedge D''[m]_1 \text{ is even}$  then ▷ Check for
    admins
38:    D''  $\leftarrow \{(m, D''[m]_0, D''[m]_1 + 1) : (m, D''[m]_0, D''[m]_1) \in D''\}$ 
39:  end if
40:  return (name'', description'',  $\nu''$ , D'',  $\mathbb{X}''$ )
41: end function

```

Before, setting the new values for the new group state $\mathbb{G}''$ and returning it, we perform a check whether there exists at least one admin in the group. If there are no admins, we make everyone admin of the group by incrementing the second tuple value of the group D'' .

Finally, we return our merged group state, which is a tuple of $name''$, $description''$, ν'' , D'' , and $\mathbb{X}''$.

3.3.8 Sending Messages

The send message function takes three inputs, *first* is the group state $\mathbb{G}$, which is a tuple of five values, *second* is the current member (a) of the group, and *third* is the *message*. The function first checks if the current member is present inside the group. If not, we return an unchanged group state. We define our algorithm such that only members present inside the group can send messages. If the condition is satisfied, a tuple of the latest *message* with a UNIX-timestamp is added inside the set $\mathbb{X}$, setting it to new set of messages $\mathbb{X}'$.

Algorithm 8 Sending Messages

```

1: function SEND_GRP_MSG( $(name, description, \nu, D, \mathbb{X}), a, message$ )
2:   if  $a \notin keys(D)$  or  $D[a]_0$  is even then  $\triangleright a$  is not in  $D$ 
3:     return  $(name, description, \nu, D, \mathbb{X})$ 
4:   end if
5:    $\mathbb{X}' \leftarrow (<unix-timestamp>, message) \cup \mathbb{X}$ 
6:   return  $(name, description, \nu, D, \mathbb{X}')$ 
7: end function

```

Finally, we return a modified group state which is a tuple of $name$, $description$, ν , D , and $\mathbb{X}'$ with non-primed values indicating they did not change.

3.3.9 Get Group Information

The get group information function takes two inputs, the *first* input is the group state $\mathbb{G}$, and the *second* input is the current member a of the group. The goal of the function returns a set of members present inside a group, a set of admins present inside the group, and a list of messages associated with the group. First, we check if the member a is/was part of the group. If not, we return an empty tuple. There is no strict requirement that a has to be present inside D . If the condition is satisfied, we initialize three empty sets, i.e., $\mathbb{M}$, $\mathbb{A}$, and $\mathcal{X}$. We then fetch all the members and add them to the set $\mathbb{M}$, and we fetch all the admins present inside the group ($\mathbb{G}$) and add them to the set $\mathbb{A}$.

To add the messages in list $\mathcal{X}$, we sort all the values of $\mathbb{X}$ by the Unix timestamp that is associated with every message. After sorting the messages, we then add the tuple value in $\mathcal{X}$. Finally, we return the members $\mathbb{M}$, admins $\mathbb{A}$, and a list of messages $\mathcal{X}$.

Algorithm 9 Get Group Information

```

1: function GET_GROUP_INFO( $(name, description, \nu, D, \mathbb{X}), a$ )
2:   if  $a \notin keys(D)$  then                                      $\triangleright a$  is not in  $D$ 
3:     return  $(\{\}, \{\}, [])$ 
4:   end if
5:    $\mathbb{M} \leftarrow \{\}$ 
6:    $\mathbb{A} \leftarrow \{\}$ 
7:    $\mathcal{X} \leftarrow []$ 
8:   for  $m \in keys(D)$  do
9:     if  $D[m]_0$  is odd then
10:       $\mathbb{M} \leftarrow D[m]_0 \cup \mathbb{M}$ 
11:     if  $D[m]_1$  is odd then
12:       $\mathbb{A} \leftarrow D[m]_1 \cup \mathbb{A}$ 
13:     end if
14:   end if
15: end for
16:    $\forall (x_0, x_1) \in \mathbb{X} : sort(x_0)$ 
17:    $\mathcal{X} \leftarrow (x_0, x_1) \cup \mathcal{X}$ 
18:   return  $(\mathbb{M}, \mathbb{A}, \mathcal{X})$ 
19: end function

```

3.4 Why repairing state in `add_member` is necessary?

Now you have the idea about how our CRDTs work on our group membership model. In this section, we will give an example and mention the exact reason for adding this condition in the `add_member` API (3.3.2).

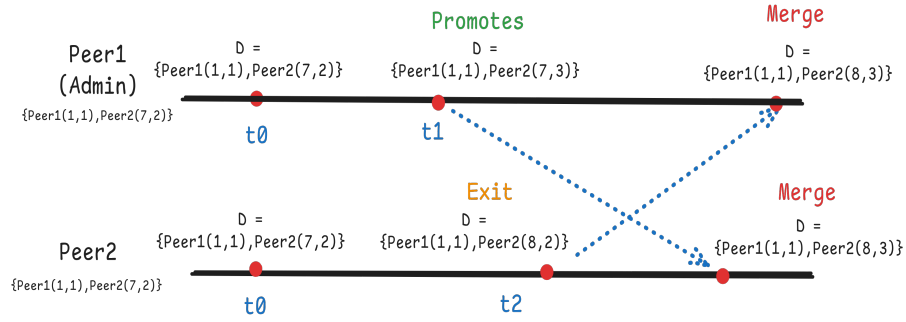


Figure 3.1: Self-exiting and promoting to admin.

From fig. 3.1, there is one admin *peer1* and a normal member *peer2* in a group. You can see the dictionary values of each member. At $t=0$, *peer1* and *peer2* have the same state. At $t=1$, *peer1* (admin) promotes *peer2*. At $t=2$, *peer2* exits the group. You can see the updated state values. Now the dictionary values are exchanged among the peers, and these states are now merged by taking the maximum values. After merging, for *peer2* the values are (8,3). These values indicates that the *peer2* is no longer a member of the group, but at the same time it is an admin of the group. This depicts an abnormal behavior of our group membership model. Therefore, in our algorithm (2), we check if the re-added member's second tuple value is odd or not. If it is odd we increment it and make it even.

Because adding a removed member again can't be an admin. To make an admin, we need to explicitly call the *promote* (5) API. Therefore, the repairing of the state is necessary. Now you must be wondering, how do we handle the logic while viewing *admins* and *members* of the group when such state exists. We already have a condition in the *get_group_group* algorithm (9). If a *member* is already inside a group, we then check if it is an *admin* or not, and we then add the member's name to the *admins* set. Please go through (9).

Can we apply this logic (repairing of states) at the merge_group_state function? Let's refer to the above example from the fig. 3.1 at the merge point. After merging we get the state (8,3). Now in the merge function, we observe that the first tuple value (member value) is an even number, and the second tuple value (admin value), it is an odd number. Depending on the first tuple value, which is an even number, we can increment the second tuple value by making it an even number. Because if the member is not in the group, then it should also not be an admin of the group. The latest state become (8,4). *Do we see a problem?* Yes, and this logic does not abide with the *idempotent* property. A peer/node can receive duplicate messages. From the above example, a peer can receive the state (8,3) twice or thrice. From the *idempotent* property, $\text{merge}((8,4), (8,3)) \neq (8,4)$.

3.5 Behavior Under Concurrent Operations

While drawing concurrent diagrams, which we have listed in (A), we stumbled upon a case of both adding and removing a member by multiple admins. We illustrate the scenario in the diagram below.

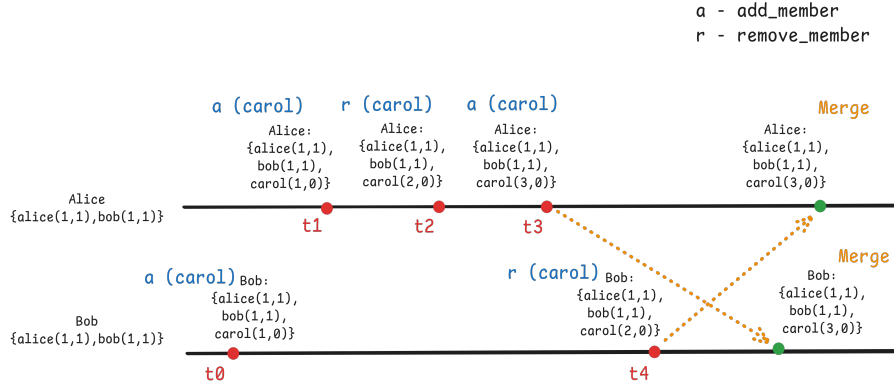


Figure 3.2: Concurrent Add & Remove of a member

In the fig. (3.2), we have 2 admins *alice* and *bob*. Both are trying to add *carol* in the group. At t_0 , *bob* adds *carol* locally and updates the *D* by setting *carol*'s value to (1,0). Not at t_1 , which is the even happens after t_0 , *alice* adds *carol* too locally. But at t_2 , *alice* thinks of removing *carol* and updates its *D* value to (2,0). Not at some point, in t_3 , *alice* again adds *carol* to the group locally and updates *carol*'s *D* value to (3,0). At t_4 , *bob* removes *carol* and updates *carol*'s *D* value to (2,0). After sometime, both *alice* and *bob* exchange their state, and a merge is performed over the exchanged state. We get *Carol*'s final value to be (3,0). Below we have also presented a table of the above scenario.

Time	Alice's Event	Bob's Event
t=0		Bob adds Carol Carol = (1,0)
t=1	Alice adds Carol Carol = (1,0)	
t=2	Alice removes Carol Carol = (2,0)	
t=3	Alice adds Carol Carol = (3,0)	
t=4		Bob removes Carol Carol = (2,0)

Table 3.3: Concurrent operations.

We observe that the CRDT state is growing monotonically, but there is no order between the events. When the entire group state (3) is exchanged between the peers, we have *carol* to be present inside the group, but the latest message in the group we see is '*Bob removes Carol*'. There are multiple ways to design, like Add-wins, Remove-wins, Last-Writer wins, etc. For our design, we have decided to solely rely on the larger tuple values of the key(members). If we choose *Last Writer Win*(LWW) strategy, then we have to explicitly set the tuple values to (2,0), and we don't want to go back. This will break the monotonic property of the CRDT. In our case, it is neither add-wins nor remove-wins. Sometimes, add wins and/or sometimes remove wins depending on the higher tuple value.

The same case might occur while promoting and demoting a member/admin from the group, and for this design, we again opt for larger tuple values.

3.5.1 Caveat

We illustrate a Byzantine scenario related to the above design choice.

Imagine a scenario where both *Alice* (*Malicious*) and *Bob* are in a group. *Alice* knows the internal working of add and remove operations while *Bob* is oblivious to these operation mechanisms. Let's refer to the figure below.

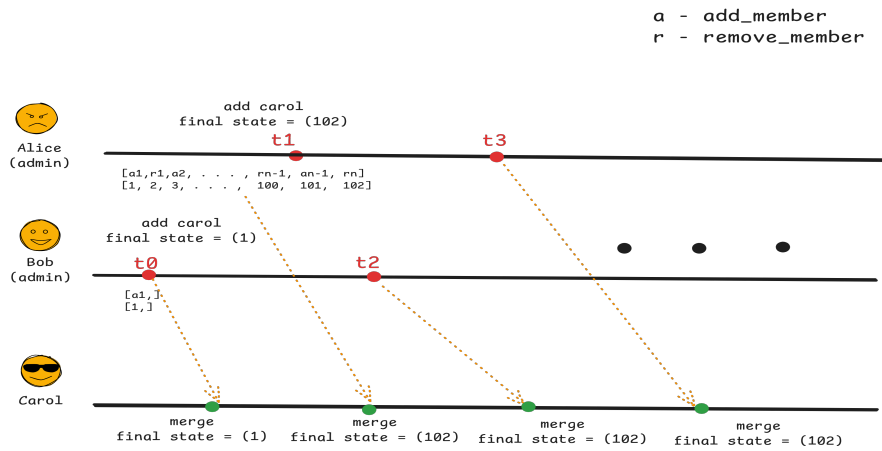


Figure 3.3: Concurrent Add Member

From the fig. (3.3), *Bob* now wants to add *Carol* to the group and pushes the updated state to *Carol*. *Alice*, who is a Byzantine peer, doesn't want *Carol* to be part of the group. *Alice* performs a bunch of add and remove operations for the key value *Carol*, having the final value to be 102. Now *Alice* pushes its state to *Carol* and a merge is now performed, indicating that now *Carol* is no more in the group, and *Alice* never pushes the final state values to *Bob*. At this point, *Carol* asks *Bob* to add again, but at the *Bob* side, *Carol* is already in the group. *Bob* retries and pushes its current state to *Carol*, and after merge, *Carol* is still not in the group. This is a complete byzantine behavior by *Alice*. Even if *Bob* pushes the current state multiple times, still *Carol* won't be part of the group. *Bob* can synchronize the state with *Carol* if *Carol* pushes its local state to *Bob*.

We have already stated in the assumption in *Chapter 5* about all peers being non-byzantine, and we leave it for future extension.

3.6 End of the Chapter

We presented a few concurrent diagrams in A about our design, but not all of them cover all the test cases. Therefore, in the next chapter 4 we present proofs of our design.

4

Proofs

The chapter discusses the process of verifying the design of the group chat application to ensure its completeness and consistency. The proofs are like a tool that will suffice to convince the behavior of CRDTs that are employed in the group chat model. The three properties of CRDT, that is *ordering*, *least upper bound*, and *monotonicity* must be proven so that the group state ($\mathbb{G}$) of our chat application eventually converge to a consistent state.

$\mathbb{G}$, which is the group state/object that stores information of a group which is a tuple of 5 values. The tuple consists *name*, *description*, ν , D and $\mathbb{X}$. In our group state tuple object, we have three CRDTs. *First*, ν which is the +ve counter value. The counter value exactly behave like the grow-only counters. *Second*, the D is the CL-CRDT. *Third*, is the set of messages, and the set of messages are like a grow-only set.

4.1 Ordering

To prove that the group state $\mathbb{G}$ is partial order, we need to first prove that ν , D , and $\mathbb{X}$ are partial orders.

4.1.1 Ordering For ν

In this section, we will prove that all possible states for counter value ν is a partial order.

4.1.1.1 Definitions

1. $\mathbb{C}$ - A counter value.
2. $[0, \infty)$ - Set of all possible states for counter values.

4.1.1.2 Proofs

PROVE: $c_i \leq c_j : c_i \in [0, \infty), c_j \in [0, \infty)$ is a partial order.

PROOF: The relation $\leq$ on $[0, \infty)$, defined as

$$c_i \leq c_j \quad \text{for } c_i, c_j \in [0, \infty),$$

is a partial order. We further prove that the ordering relation $\leq$ on the set of all possible states of $\mathbb{C}$ forms a partial order.

$\langle 1 \rangle 1$. $\mathbb{C}$ is a set of positive counter values, *i.e.* $\mathbb{C} = \mathbb{N}$

By definitions $\boxed{1}$, and $\boxed{2}$

$\langle 1 \rangle 2$. All positive integers are partially ordered by $\leq$.

$\langle 2 \rangle 1$. It is *reflexive* because $c_i \leq c_j$.

$\langle 2 \rangle 2$. It is *transitive* because $c_i \leq c_j \leq c_k \Rightarrow c_i \leq c_k$.

$\langle 2 \rangle 3$. It is *anti-symmetry* because $c_i \leq c_j$ and $c_j \leq c_i \Rightarrow c_i = c_j$

$\langle 1 \rangle 3$. Q.E.D

By $\boxed{\langle 1 \rangle 1}$, and $\boxed{\langle 1 \rangle 2}$.

4.1.2 Ordering For $\mathbb{X}$

In this section, we will prove that all possible states for set of messages $\mathbb{X}$ is a partial order.

4.1.2.1 Definitions

1. $\mathbb{X}$ - A set of messages.
2. $\mathbb{X}$ - Set of all possible states for set of messages is the power-set of set of messages $\mathbb{X}$, *i.e.*, $\mathbb{X} = \mathcal{P}(\mathbb{X})$.

4.1.2.2 Proofs

PROVE: $X_i \subseteq X_j : X_i \in \mathbb{X}, X_j \in \mathbb{X}$ is a partial order.

PROOF: The relation $\subseteq$ on $\mathbb{X}$, defined as

$$x_i \subseteq x_j \quad \text{for } x_i, x_j \in \mathbb{X},$$

is a partial order. We further prove that the ordering relation $\subseteq$ on the set of all possible states of $\mathbb{X}$ forms a partial order.

$\langle 1 \rangle 1$. $\mathbb{X}$ is power-set of messages, *i.e.* $\mathbb{X} = \mathcal{P}(\mathbb{X})$

By definitions $\boxed{1}$, and $\boxed{2}$

$\langle 1 \rangle 2$. The set of messages are in partial ordered by $\subseteq$.

$\langle 2 \rangle 1$. It is *reflexive* because $X_i \subseteq X_j$.

$\langle 2 \rangle 2$. It is *transitive* because $X_i \subseteq X_j \subseteq X_k \Rightarrow X_i \subseteq X_k$.

$\langle 2 \rangle 3$. It is *anti-symmetry* because $X_i \subseteq X_j$ and $X_j \subseteq X_i \Rightarrow X_i = X_j$

$\langle 1 \rangle 3$. Q.E.D

By $\langle 1 \rangle 1$, and $\langle 1 \rangle 2$.

4.1.3 Ordering For D

In this section, we will prove that all possible states for members and admins that are inside D are partial order.

4.1.3.1 Definitions

1. **Members** ($\mathbb{M}$) - Set of possible members in a group.
2. **Positive Integers** (Z_+) - $\{n \mid n \geq 0\}$, set of integers ≥ 0 .
3. **All Possible States** - The state of the group is a set D of tuples (m, p, q) s.t. $m \in \mathbb{M}$ and the integer values $p \in Z_+$ and $q \in Z_+$, whereas D is a dictionary. p indicates the presence of a member in a group, and q indicates the presence of an admin in a group. Let $\mathbb{S}$ be the space of all possible dictionaries D , i.e. $(\mathbb{S} = \mathcal{P}(\mathbb{M} \times Z_+ \times Z_+))$. The space of all possible dictionaries is defined as the power-set of the Cartesian product of $\mathbb{M}$, Z_+ , and Z_+ , s.t. a key $m \in D$ appears at most once in any $\mathcal{S}_i \in \mathbb{S}$.
4. **Keys** - The keys of D is a set of members, i.e. $keys(D) = \{m : (m, p, q) \in D\}$
5. **Counters** - The counters p , and q are stored in a tuple (of size 2) and are the dictionary values of D , i.e. $(m, p, q) \in D$, i.e. $D[m] = (p, q) \Leftrightarrow (m, p, q) \in D$.
6. **Dictionary Assignment** - The counter values of (p, q) are assigned to key m is written as $D[m] = (p, q) \Leftrightarrow (m, p, q) \in D$. To access a single value from the tuple from a corresponding key (m), it is illustrated as $D[m]_0$ and $D[m]_1$. The $D[m]_0$ fetches the first value of the tuple, whereas the $D[m]_1$ fetches the second value of the tuple.
7. **Possible keys** - All possible keys $\mathbb{K} = \{keys(D) : D \in \mathbb{S}\}$.
8. **Possible counters** - All possible counters $\mathbb{P}$ and $\mathbb{Q}$, where $\mathbb{P} = \{D[m]_0 : D \in \mathbb{S} \wedge m \in keys(D)\}$, and $\mathbb{Q} = \{D[m]_1 : D \in \mathbb{S} \wedge m \in keys(D)\}$

4.1.3.2 Proofs

PROVE: $\mathcal{S}_i \subseteq \mathcal{S}_j : keys(\mathcal{S}_i) \subseteq keys(\mathcal{S}_j)$ and for each $\mathbf{m} \in keys(\mathcal{S}_i)$, $\mathcal{S}_i[m]_0 \leq \mathcal{S}_j[m]_0$, and $\mathcal{S}_i[m]_1 \leq \mathcal{S}_j[m]_1$ is a partial order.

PROOF:

$\langle 1 \rangle$ All possible keys $\mathbb{K}$ of dictionary $\mathbf{m}$ are partially ordered by $\subseteq$, i.e. $keys(\mathcal{S}_i) \subseteq keys(\mathcal{S}_j)$ is a partial order.

$\langle 1 \rangle$ 1. $\mathbb{K}$ is powerset of elements, i.e. $\mathbb{K} = \mathcal{P}(\mathbb{M})$.

By definitions 3, and 4

$\langle 2 \rangle$ 2. The sets are partially ordered by $\subseteq$, because the subset relation satisfies the property of *reflexive*, *transitive*, and *anti-symmetric* [62, p. 11, A.1.1, $\langle 1 \rangle 3$].

$\langle 3 \rangle$ 3. Q.E.D

By $\langle 2 \rangle$ 1 and $\langle 2 \rangle$ 2.

$\langle 2 \rangle$ All possible counter values $\mathbb{P}$ and $\mathbb{Q}$ are partially ordered by $\leq$, i.e. $\mathcal{S}_i[m]_0 \leq \mathcal{S}_j[m]_0$, and $\mathcal{S}_i[m]_1 \leq \mathcal{S}_j[m]_1$ is a partial order.

$\langle 1 \rangle$ 1. $\mathbb{P}$ and $\mathbb{Q}$ are a set of positive integers, i.e. $\mathbb{P} = \mathbb{Q} = \mathbb{Z}_+$.

By definitions [3](#), and [5](#)

$\langle 2 \rangle$ 2. The positive integers $\mathbb{Z}_+$ are partially ordered by $\leq$, because $\leq$ satisfies the property of being *reflexive*, *transitive*, and *anti-symmetric* [62, p. 11, A.1.1, $\langle 1 \rangle 3$].

$\langle 3 \rangle$ 3. Q.E.D

By $\langle 2 \rangle$ 1 and $\langle 2 \rangle$ 2.

$\langle 3 \rangle$ From the proof [62, p. 11, A.1.1, $\langle 1 \rangle 4$, and $\langle 1 \rangle 5$], already proves that the conjunction ($\wedge$) of partial orders for all possible states of $\mathbb{D}$ is also a partial order.

From 4.1.1, 4.1.2, and 4.1.3 we have proved that the sub-state of the group object $\mathbb{G}$ i.e, ν , $\mathbb{X}$, and D are in partial order. But we now need to prove that the conjunction of the partial order of the group state $\mathbb{G}$ is also a partial order.

4.1.4 Proof for conjunction of partial orders over $\mathbb{G}$

ASSUME: Let the relations $\leq$, $\subseteq$, and $\sqsubseteq$ over set S .

Where the state space is,

$$S = \nu \times \mathbb{X} \times D$$

DEFINE: Let τ be the conjunction of above relations over S such that,

$$(\nu_1, \mathbb{X}_1, D_1) \tau (\nu_2, \mathbb{X}_2, D_2) = (\nu_1 \leq \nu_2) \wedge (\mathbb{X}_1 \subseteq \mathbb{X}_2) \wedge (D_1 \sqsubseteq D_2)$$

PROVE: τ over S .

PROOF SKETCH: We need to prove that the relation τ is *reflexive*, *antisymmetric*, and *transitive*.

$\langle 1 \rangle 1$. **Reflexive:**

Let $s = (\nu, \mathbb{X}, D) \in S$.

$\langle 2 \rangle 1$. From 4.1.1.2, we proved that ν is a partial order. Therefore, $\nu \leq \nu$ by the reflexivity of the relation $\leq$.

$\langle 2 \rangle 2$. From 4.1.2.2, we proved that $\mathbb{X}$ is a partial order. Therefore, $\mathbb{X} \subseteq \mathbb{X}$ by the reflexivity of the relation $\subseteq$.

$\langle 2 \rangle 3$. From 4.1.3.2, we proved that D is a partial order. Therefore, $D \sqsubseteq D$ by the reflexivity of the relation $\sqsubseteq$.

$\langle 2 \rangle 4$. From $\langle 2 \rangle 1$, $\langle 2 \rangle 2$, and $\langle 2 \rangle 3$,

$$(\nu, \mathbb{X}, D) \tau (\nu, \mathbb{X}, D)$$

Hence, τ is **reflexive**.

⟨1⟩2. **Antisymmetry:**

Let $s_1 = (\nu_1, \mathbb{X}_1, D_1)$, and $s_2 = (\nu_2, \mathbb{X}_2, D_2)$, and assume

$$s_1 \tau s_2 \text{ and } s_2 \tau s_1$$

⟨2⟩1. From 4.1.1.2, we proved that ν is a partial order and stated the *antisymmetry* property.

Then, $\nu_1 \leq \nu_2$ and $\nu_2 \leq \nu_1$

Therefore, $\nu_1 = \nu_2$.

⟨2⟩2. From 4.1.2.2, we proved that $\mathbb{X}$ is a partial order and stated the *antisymmetry* property.

Then, $\mathbb{X}_1 \subseteq \mathbb{X}_2$ and $\mathbb{X}_2 \subseteq \mathbb{X}_1$

Therefore, $\mathbb{X}_1 = \mathbb{X}_2$.

⟨2⟩3. From 4.1.3.2, we proved that D is a partial order and stated the *antisymmetry* property.

Then, $D_1 \subseteq D_2$ and $D_2 \subseteq D_1$

Therefore, $D_1 = D_2$.

⟨2⟩4. From ⟨2⟩1, ⟨2⟩2, and ⟨2⟩3,

$$s_1 = s_2$$

Hence, τ is **antisymmetry**.

⟨1⟩3. **Transitive:**

Let $s_1 = (\nu_1, \mathbb{X}_1, D_1)$, $s_2 = (\nu_2, \mathbb{X}_2, D_2)$, and $s_3 = (\nu_3, \mathbb{X}_3, D_3)$ and assume

$$s_1 \tau s_2 \text{ and } s_2 \tau s_3$$

⟨2⟩1. From 4.1.1.2, we proved that ν is a partial order. With *transitive* property,

Then, $\nu_1 \leq \nu_2$ and $\nu_2 \leq \nu_3$

Therefore, $\nu_1 \leq \nu_3$.

⟨2⟩2. From 4.1.2.2, we proved that $\mathbb{X}$ is a partial order. With *transitive* property,

Then, $\mathbb{X}_1 \subseteq \mathbb{X}_2$ and $\mathbb{X}_2 \subseteq \mathbb{X}_3$

Therefore, $\mathbb{X}_1 \subseteq \mathbb{X}_3$.

⟨2⟩3. From 4.1.3.2, we proved that D is a partial order. With *transitive* property,

Then, $D_1 \subseteq D_2$ and $D_2 \subseteq D_3$

Therefore, $D_1 \subseteq D_3$.

⟨2⟩4. From ⟨2⟩1, ⟨2⟩2, and ⟨2⟩3,

$$s_1 \tau s_3$$

Hence, τ is **transitive**.

⟨1⟩4. *Q.E.D*

By ⟨1⟩1, ⟨1⟩2, and ⟨1⟩3, τ is *reflexive*, *antisymmetry*, and *transitive*. τ is a partial order over S . Hence, we proved that the conjunction of partial orders over $\mathbb{G}$ is a partial order.

4.2 Monotonicity

In our group chat design, we have $\mathbb{G}$ (3), the group state, which is a tuple of *five* values.

ASSUME: $\mathbb{G}$ is the current state.

PROVE: All group operations with possible arguments will generate a new state when applied on $\mathbb{G}$. The group state's result will be equal to or larger than $\mathbb{G}$.

$\langle 1 \rangle 1$. CASE: $\mathbb{G} = \text{create_group}(\text{name}, \text{description}, \nu, D, \mathbb{X})$

This operation creates a new group state, so monotonicity does not apply.

$\langle 1 \rangle 2$. CASE: $\mathbb{G}' = \text{add_member}((\text{name}, \text{description}, \nu, D, \mathbb{X}), a, m) \geq \mathbb{G}$

$\langle 2 \rangle 1$. CASE: $a \notin \text{keys}(D) \vee D[a]_0$ is even $\vee D[a]_1$ is even

Current member a is not in D and is not an admin. Can't add m to D . No keys or counters are modified. $D' = D$, therefore $\mathbb{G} = \mathbb{G}'$.

$\langle 2 \rangle 2$. CASE: $m \notin \text{keys}(D)$

The key m is added to D , setting the tuple value $(1, 0)$. D' keys are a superset of D 's keys, therefore $\mathbb{G}' > \mathbb{G}$.

$\langle 2 \rangle 3$. CASE: $m \in \text{keys}(D) \wedge D[m]_0$ is even $\wedge D[m]_1$ is even

The counter $D[m]_0$ is incremented by 1. $D' > D$, therefore $\mathbb{G}' > \mathbb{G}$.

$\langle 2 \rangle 4$. CASE: $m \in \text{keys}(D) \wedge D[m]_0$ is even $\wedge D[m]_1$ is odd

Both the counters $D[m]_0$ and $D[m]_1$ is incremented by 1. $D' > D$, therefore $\mathbb{G}' > \mathbb{G}$.

$\langle 2 \rangle 5$. CASE: $m \in \text{keys}(D) \wedge D[m]_0$ is odd

No keys or counters are modified. $D' = D$, therefore $\mathbb{G}' = \mathbb{G}$.

$\langle 2 \rangle 6$. From $\langle 2 \rangle 2$., $\langle 2 \rangle 3$., and $\langle 2 \rangle 4$. we get a new dictionary D' . Now taking the conjunction of D' with the group state $\mathbb{G}$, resulting a new group state $\mathbb{G}'$ (a new partial order). Hence, $\mathbb{G}' > \mathbb{G}$.

$\langle 1 \rangle 3$. CASE: $\mathbb{G}' = \text{remove_member}((\text{name}, \text{description}, \nu, D, \mathbb{X}), a, m) \geq \mathbb{G}$

$\langle 2 \rangle 1$. CASE: $a \notin \text{keys}(D) \vee D[a]_0$ is even $\vee D[a]_1$ is even

Current member a is not in D and is not an admin. Can't remove m from D . No keys or counters are modified. $D' = D$, therefore $\mathbb{G} = \mathbb{G}'$.

$\langle 2 \rangle 2$. CASE: $m \notin \text{keys}(D)$

Key m not found in $\text{keys}(D)$. No keys or counters are modified. $D' = D$, therefore $\mathbb{G}' = \mathbb{G}$.

$\langle 2 \rangle 3$. CASE: $m \in \text{keys}(D) \wedge D[m]_0$ is odd $\wedge D[m]_1$ is even

The counter $D[m]_0$ is incremented by 1. $D' > D$, therefore $\mathbb{G}' > \mathbb{G}$.

$\langle 2 \rangle 4$. CASE: $m \in \text{keys}(D) \wedge D[m]_0$ is odd $\wedge D[m]_1$ is odd

Both the counters $D[m]_0$ and $D[m]_1$ is incremented by 1. $D' > D$, therefore $\mathbb{G}' > \mathbb{G}$.

$\langle 2 \rangle 5$. CASE: $m \in \text{keys}(D) \wedge D[m]_0$ is even $\wedge D[m]_1$ is even

No keys or counters are modified. $D' = D$, therefore $\mathbb{G}' = \mathbb{G}$.

$\langle 2 \rangle 6$. From $\langle 2 \rangle 3$., and $\langle 2 \rangle 4$., We get a new dictionary D' . Now taking the conjunction of D' with the group state $\mathbb{G}$, resulting in a new group state $\mathbb{G}'$ (a new partial order). Hence, $\mathbb{G}' > \mathbb{G}$.

- ⟨1⟩4. CASE: $\mathbb{G}' = \text{promote}((\text{name}, \text{description}, \nu, D), a, m) \geq \mathbb{G}$
- ⟨2⟩1. CASE: $a \notin \text{keys}(D) \vee D[a]_0$ is even $\vee D[a]_1$ is even
Current member a is not in D and is not an admin. Can't remove m from D .
No keys or counters are modified. $D' = D$, therefore $\mathbb{G} = \mathbb{G}'$.
 - ⟨2⟩2. CASE: $m \notin \text{keys}(D)$
Key m not found in $\text{keys}(D)$. No keys or counters are modified. $D' = D$,
therefore $\mathbb{G}' = \mathbb{G}$.
 - ⟨2⟩3. CASE: $m \in \text{keys}(D) \wedge D[m]_0$ is odd $\wedge D[m]_1$ is even
The counter $D[m]_1$ is incremented by 1. $D' > D$, therefore $\mathbb{G}' > \mathbb{G}$.
 - ⟨2⟩4. CASE: $m \in \text{keys}(D) \wedge D[m]_0$ is odd $\wedge D[m]_1$ is odd
No keys or counters are modified. $D' = D$, therefore $\mathbb{G}' = \mathbb{G}$.
 - ⟨2⟩5. CASE: $m \in \text{keys}(D) \wedge D[m]_0$ is even $\wedge D[m]_1$ is even
No keys or counters are modified. $D' = D$, therefore $\mathbb{G}' = \mathbb{G}$.
 - ⟨2⟩6. From ⟨2⟩3., We get a new dictionary D' . Now taking the conjunction of D' with
the group state $\mathbb{G}$, resulting in a new group state $\mathbb{G}'$ (a new partial order). Hence,
 $\mathbb{G}' > \mathbb{G}$.
- ⟨1⟩5. CASE: $\mathbb{G}' = \text{demote}((\text{name}, \text{description}, \nu, D), a, m) \geq \mathbb{G}$
- ⟨2⟩1. CASE: $a \notin \text{keys}(D) \vee D[a]_0$ is even $\vee D[a]_1$ is even
Current member a is not in D and is not an admin. Can't remove m from D .
No keys or counters are modified. $D' = D$, therefore $\mathbb{G} = \mathbb{G}'$.
 - ⟨2⟩2. CASE: $m \notin \text{keys}(D)$
Key m not found in $\text{keys}(D)$. No keys or counters are modified. $D' = D$,
therefore $\mathbb{G}' = \mathbb{G}$.
 - ⟨2⟩3. CASE: $m \in \text{keys}(D) \wedge D[m]_0$ is odd $\wedge D[m]_1$ is even
No keys or counters are modified. $D' = D$, therefore $\mathbb{G}' = \mathbb{G}$.
 - ⟨2⟩4. CASE: $m \in \text{keys}(D) \wedge D[m]_0$ is odd $\wedge D[m]_1$ is odd
The counter $D[m]_1$ is incremented by 1. $D' > D$, therefore $\mathbb{G}' > \mathbb{G}$.
 - ⟨2⟩5. CASE: $m \in \text{keys}(D) \wedge D[m]_0$ is even $\wedge D[m]_1$ is even
No keys or counters are modified. $D' = D$, therefore $\mathbb{G}' = \mathbb{G}$.
 - ⟨2⟩6. From ⟨2⟩4., We get a new dictionary D' . Now taking the conjunction of D' with
the group state $\mathbb{G}$, resulting a new group state $\mathbb{G}'$ (a new partial order). Hence,
 $\mathbb{G}' > \mathbb{G}$.
- ⟨1⟩6. CASE: $\mathbb{G}' = \text{rename_group}((\text{name}, \text{description}, \nu, D), \text{name}', \text{description}', a) \geq \mathbb{G}$
- ⟨2⟩1. CASE: $a \notin \text{keys}(D) \vee D[a]_0$ is even $\vee D[a]_1$ is even
Can't modify the group name and description, and ν remains unchanged. $\nu' = \nu$,
therefore $\mathbb{G}' = \mathbb{G}$.
 - ⟨2⟩2. CASE: $a \in \text{keys}(D) \wedge D[a]_0$ is odd $\wedge D[a]_1$ is odd
Modifying *name* and *description* of the group, we increase the counter value of
 ν . $\nu' > \nu$, therefore, $\mathbb{G}' \geq \mathbb{G}$.
 - ⟨2⟩3. From ⟨2⟩2., We get a new dictionary D' . Now taking the conjunction of D' with
the group state $\mathbb{G}$, resulting in a new group state $\mathbb{G}'$ (a new partial order). Hence,
 $\mathbb{G}' > \mathbb{G}$.

- ⟨1⟩7. CASE: $\mathbb{G}' = \text{send_grp_msg}((name, description, \nu, D, \mathbb{X}), a, message)$
- ⟨2⟩1. CASE: $a \notin keys(D) \vee D[a]_0$ is even
 Current member a is not in D . Can't send messages.
 The set of messages $\mathbb{X}$ is unchanged. $\mathbb{X}' = \mathbb{X}$, therefore $\mathbb{G} = \mathbb{G}'$.
- ⟨2⟩2. CASE: $a \in keys(D) \wedge D[a]_0$ is odd
 The message is added to set $\mathbb{X}$. $\mathbb{X}' > \mathbb{X}$, therefore $\mathbb{G}' > \mathbb{G}$.
- ⟨2⟩3. From ⟨2⟩2., We get a new dictionary D' . Now taking the conjunction of D' with the group state $\mathbb{G}$, resulting in a new group state $\mathbb{G}'$ (a new partial order). Hence, $\mathbb{G}' > \mathbb{G}$.
- ⟨1⟩8. CASE: $(\mathbb{M}, \mathbb{A}, \mathbb{X}) \leftarrow \text{get_group_info}(\mathbb{G}, a)$
 This operation fetches group information and doesn't modify anything in the $\mathbb{G}$, therefore monotonicity does not apply.

4.3 Least Upper Bound

In this section, we will prove the merge operation (3.3.7) of the two group states $\mathbb{G}$ and $\mathbb{G}'$, which results in a new $\mathbb{G}'$ that forms LUB in semi-lattice $\mathbb{L}$. From (3) we know that the group state $\mathbb{G}$ is tuple of 5 objects. From the tuple, we are merging 3 objects that are ν with ν' , $\mathbb{X}$ with $\mathbb{X}'$, and D with D' . Merge over the 2 group states is the conjunction over the states, i.e., taking max values for ν , union over $\mathbb{X}$, and performing union on the keys of D and taking max of the tuple values of the D .

First we will prove the merging of these 3 objects in order to prove merging of $\mathbb{G}$ with $\mathbb{G}'$.

4.3.1 LUB of ν and ν'

ASSUME: $\nu \in Z_+$ and $\nu' \in Z_+$, $\leq$ is both total order and partial order on Z_+ forming a semi-lattice $\mathbb{L}_{Z_+}$

PROVE: $\nu'' \leftarrow \max(\nu, \nu')$ computes LUB of ν, ν' .

PROOF:

- ⟨1⟩1. $\nu'' \geq \nu$
 Because $\nu'' = \max(\nu, \nu') \geq \nu$ (by cases on $\nu < \nu', \nu = \nu', \nu > \nu'$)
- ⟨1⟩2. $\nu'' \geq \nu'$
 Because $\nu'' = \max(\nu, \nu') \geq \nu'$ (by cases on $\nu < \nu', \nu = \nu', \nu > \nu'$)
- ⟨1⟩3. $\nexists \nu''' \in Z_+$ s.t. $\nu''' \geq \nu \wedge \nu''' \geq \nu' \wedge \nu''' < \nu''$
 Let us assume that there exists ν''' . By (⟨1⟩3) case, $\nu''' < \nu''$, therefore ν''' is smaller than both ν and ν' . But this contradicts $\nu''' \geq \nu, \nu''' \geq \nu'$, or both. Therefore, ν'' is the smallest +ve integer that is greater than both ν and ν' in $\mathbb{L}_{Z_+}$.

The conjunction of ⟨1⟩1, ⟨1⟩2, and ⟨1⟩3 is the definition of least upper bound.

4.3.2 LUB of $\mathbb{X}$ and $\mathbb{X}'$

ASSUME:

1. Let x be in $\mathbb{X}$ where x is a single message.
2. $\mathbb{X} = \mathcal{P}(\mathbb{X})$.
3. $\mathbb{X} \in \mathbb{X}$ and $\mathbb{X}' \in \mathbb{X}$.
4. $\subseteq$ is a partial order on $\mathbb{X}$ forming a semi-lattice $\mathbb{L}_{\mathbb{X}}$.

PROVE: $\mathbb{X}'' = \mathbb{X} \cup \mathbb{X}'$ computes LUB of $\mathbb{X}$, $\mathbb{X}'$.

PROOF:

$\langle 1 \rangle 1.$ $\mathbb{X} \subseteq \mathbb{X}''$

Because $\mathbb{X} \subseteq (\mathbb{X} \cup \mathbb{X}') = \mathbb{X}''$ (by cases on $\mathbb{X} \subset \mathbb{X}'$, $\mathbb{X} = \mathbb{X}'$, $\mathbb{X} \supset \mathbb{X}'$)

$\langle 1 \rangle 2.$ $\mathbb{X}' \subseteq \mathbb{X}''$

Because $\mathbb{X}' \subseteq (\mathbb{X} \cup \mathbb{X}') = \mathbb{X}''$ (by cases on $\mathbb{X} \subset \mathbb{X}'$, $\mathbb{X} = \mathbb{X}'$, $\mathbb{X} \supset \mathbb{X}'$)

$\langle 1 \rangle 3.$ $\nexists \mathbb{X}''' \in \mathbb{X}$ s.t. $\mathbb{X}''' \subset \mathbb{X}'' \wedge \mathbb{X} \subseteq \mathbb{X}''' \wedge \mathbb{X}' \subseteq \mathbb{X}'''$

Let us assume that there exists $\mathbb{X}'''$. We know that there is a message $x \in \mathbb{X}''$ and $x \notin \mathbb{X}'''$ because $\mathbb{X}''' \subset \mathbb{X}''$ and $\mathbb{X}'''$ is smaller. Since $\mathbb{X}'' = \mathbb{X} \cup \mathbb{X}'$, either $x \in \mathbb{X}$, $x \in \mathbb{X}'$ or in both. In that case, $\mathbb{X}'''$ cannot be superset of both $\mathbb{X}$ and $\mathbb{X}'$, because there is no x in $\mathbb{X}'''$. Therefore, $\mathbb{X}''$ must be the smallest superset of both $\mathbb{X}$ and $\mathbb{X}'$ in $\mathbb{L}_{\mathbb{X}}$.

The conjunction of $\langle 1 \rangle 1$, $\langle 1 \rangle 2$, and $\langle 1 \rangle 3$ is the definition of least upper bound.

4.3.3 LUB of D and D'

Merging two dictionaries D and D' , we are performing a union over the keys of the dictionaries and taking the maximum values of the counters.

ASSUME:

1. m is member inside D .
2. $\mathbb{S}$ is all possible state. From (3).
3. $D \in \mathbb{S}$ and $D' \in \mathbb{S}$.

PROVE: Merge of D and D' computes their LUB D'' in $\mathbb{L}$.

$\langle 1 \rangle 1.$ First we prove, $keys(D'')$ is the LUB of $keys(D) \cup keys(D')$.

ASSUME:

1. $\mathcal{P}(\mathbb{M})$ is power set of members in $\mathbb{M}$.
2. $M \in \mathcal{P}(\mathbb{M})$ and $M' \in \mathcal{P}(\mathbb{M})$
3. $\subseteq$ is a partial order on $\mathcal{P}(\mathbb{M})$ forming a semi-lattice $\mathbb{L}_{\mathcal{P}(\mathbb{M})}$

PROVE: $M'' = M \cup M'$ computes LUB of M , M' .

$\langle 2 \rangle 1.$ $M \subseteq M''$

Because $M \subseteq (M \cup M') = M''$ (by cases on $M \subset M'$, $M = M'$, $M \supset M'$)

⟨2⟩2. $M' \subseteq M''$

Because $M' \subseteq (M \cup M') = M''$ (by cases on $M \subset M', M = M', M \supset M'$)

⟨2⟩3. $\nexists M''' \in \mathcal{P}(\mathbb{M})$ s.t. $M''' \subset M'' \wedge M \subseteq M''' \wedge M' \subseteq M'''$

Let us assume that there exists M''' . We know that there is a member $m \in M''$ and $m \notin M'''$ because $M''' \subset M''$ and M''' is smaller. Since $M'' = M \cup M'$, either $m \in M$, $m \in M'$ or in both. In that case, M''' cannot be a superset of both M and M' , because there is no m in M''' . Therefore, M'' must be the smallest superset of both M and M' in $\mathbb{L}_{\mathcal{P}(\mathbb{M})}$.

⟨2⟩4. Q.E.D.

The conjunction of ⟨2⟩1, ⟨2⟩2, and ⟨2⟩3 is the definition of least upper bound.

⟨1⟩2. *Second* we prove, for every $m \in \text{keys}(D'')$ s.t. $m \in \text{keys}(D) \wedge m \in \text{keys}(D')$, $D''[m]_0$ is LUB of $D[m]_0$ and $D'[m]_0$, and $D''[m]_1$ is LUB of $D[m]_1$ and $D'[m]_1$.

NOTE: $c = D[m]_0 = D[m]_1 = D'[m]_0 = D'[m]_1$

The tuple values are the counter values, and we will only prove the LUB of the counter values, which suffice the proof of LUB for both the tuple values.

ASSUME:

1. $c \in Z_+$ and $c' \in Z_+$.
2. $\leq$ is both total order and partial order on Z_+ forming a semi-lattice $\mathbb{L}_{Z_+}$

PROVE: $c'' \leftarrow \max(c, c')$ computes LUB of c, c' .

PROOF:

⟨3⟩1. $c'' \geq c$

Because $c'' = \max(c, c') \geq c$ (by cases on $c < c', c = c', c > c'$)

⟨3⟩2. $c'' \geq c'$

Because $c'' = \max(c, c') \geq c'$ (by cases on $c < c', c = c', c > c'$)

⟨3⟩3. $\nexists c''' \in Z_+$ s.t. $c''' \geq c \wedge c''' \geq c' \wedge c''' < c''$

Let us assume that there exists c''' . By (⟨3⟩3) case, $c''' < c''$, therefore c''' is smaller than both c and c' . But this contradicts $c''' \geq c$, $c''' \geq c'$, or both. Therefore, c'' is the smallest +ve integer that is greater than both c and c' in $\mathbb{L}_{Z_+}$.

⟨3⟩4. Q.E.D.

The conjunction of ⟨3⟩1, ⟨3⟩2, and ⟨3⟩3 is the definition of least upper bound.

⟨1⟩3. $\text{keys}(D), \text{keys}(D'), \text{keys}(D'')$ are sets; with $\text{keys}(D'') = \text{keys}(D) \cup \text{keys}(D')$, and ⟨1⟩1, both imply that $\text{keys}(D'')$ is LUB of $\text{keys}(D)$ and $\text{keys}(D')$ in $\mathbb{L}_{\mathcal{P}(\mathbb{M})}$.

⟨1⟩4. $D[m]_0, D'[m]_0, D''[m]_0, D[m]_1, D'[m]_1, D''[m]_1$ are positive numbers, $D''[m] = (\max(D[m]_0, D'[m]_0), \max(D[m]_1, D'[m]_1))$, and ⟨1⟩2, both imply that $D''[m]_0$ is LUB of $D[m]_0$ and $D'[m]_0$, and $D''[m]_1$ is LUB of $D[m]_1$ and $D'[m]_1$.

⟨1⟩5. From ⟨1⟩1, ⟨1⟩2, ⟨1⟩3, and ⟨1⟩4 covered the conjunction on the conditions of both keys and the counter values of the tuple. And, there cannot be such D''' . Therefore D'' is the LUB of D and D' in $\mathbb{L}$.

For each group state $\mathbb{G}$ and $\mathbb{G}'$, we have proved the LUB for ν (4.3.1), $\mathbb{X}$ (4.3.2), and D (4.3.3). Now taking the conjunction over the two group states, which is the tuple values of 5 objects, computes the LUB and gives the next higher group state object, i.e., $\mathbb{G}''$ is LUB of $\mathbb{G}$ and $\mathbb{G}'$.

5

Implementation - Group Messaging App

In this chapter, we discuss the detailed implementation of our group messaging app. We have already covered 7 popular group messaging apps 2.1 in *chapter 2*, talking about their group membership model and different group operations that can be performed by a member or an admin of the group. The aim of this thesis is to mimic the characteristics and behavior of these chat applications, focusing on group messaging.

In *chapter 3*, we have already illustrated the design of our group messaging application and demonstrated the APIs and pseudocode of the group operations required in the group messaging app. In this chapter, we talk about the detailed implementation of our group chat application and how we map over *git* [19, p. 418]. The distributed nature of *git* gives us flexibility to work offline without a central server. Multiple git repositories can be set up in a P2P architecture, and owners of the git repository can directly push their local offline updates to remotes without a central coordinator. *Git* in-built has communication protocols like HTTP/HTTPS, SSH, and the git protocol to transfer data objects between remote git repositories. Therefore, leveraging these concepts of *git*, we are building our P2P offline group messaging app on top of git tool. We are using *git* low-level plumbing commands to create git objects and references, and for looking at the data stored in git objects. At the end of this chapter, we have given a detailed explanation of the git plumbing commands. To execute group operations, we have wrote a few *bash* scripts, where each bash script is the apis that were discussed in *chapter 3*. The script is composed of low-level git plumbing commands, that will help to simulate the behavior of these group operations. We have also implemented all the apis in Rust using the libgit2 library [20] for evaluation purposes, which is discussed in the following chapter in detail. Both Bash and Rust implementations can be found on GitHub [34].

5.1 System Model and Assumptions

Our implementation is designed for an asynchronous decentralized system where large number of peers can communicate within a group. Each *peer* is an *author* of a git repository that stores the metadata, groups, public/private keys, and other replicas' network addresses. Each peer is identified by their public key, and the private key is used to sign the messages

that are exchanged between peers in a group chat to ensure authenticity. Peer/user who creates a group is always an admin of that created group. Peers that are part of a group do not need to be online all the time, and can reconcile later. It is also assumed that the peers (members) that are inside a group can only send messages in a group. It is also assume that all the members of the group are non-byzantine. It is also assumed that a group supports multiple admins. A peer/user can also create a group with the same name multiple times, or the peers/members can be in multiple group with the same name.

5.2 Setup of our Application

Before giving a detailed internal explanation of our group messaging app, we would like to describe how to set up and/or start with the application. As we have mentioned above, our group messaging app is built on top of *git* tool, so we need a git repository. We not only require a basic local git repository, but we also need some additional files and configuration to setup our group messaging app. Therefore, we've written a script that creates a git repository for our group messaging app.

5.2.1 How to run?

```
$ repo-init "name" "email"
```

Listing 5.1: Init git repo with name and email

Our script name is *repo-init*, which takes 2 arguments name, and email to initialize git repository for our chat application. The above command is used to create the git repository for our messaging app.

When this script is run, it creates a bunch of files inside a secret folder named *.author-cb*. This secret folder is found inside *.git* folder. The script also generates a public-private key pair. The algorithm choice we chose is *ed25519* [17] because it is much faster to generate key pairs. Moreover, the key size required to generate pairs is much smaller compared to other algorithms. For e.g, in *RSA*, for better security, one needs a key size of 2048 bits. But it takes a lot of time to generate key pairs, whereas in *ed25519* it only requires a key size of 256 bits and provide better security compared to others. It makes authentication faster and is more efficient.

5.2.2 Inside *.author-cb* folder

Below are the files that are created inside the *.author-cb* folder.

1. **git-cb**: This file only stores the name of the author or/and/or committer of the repository. This file is a reference to fetch the name of the committer when making git commits.
2. **git-cb-email**: This file only stores the email of the author and/or committer. This file is a reference to fetch the email address of the committer when making git commits.

3. **key**: This file only stores the public key of the owner of this repository.
4. **key.pub**: This file only stores the private key of the owner of this repository.
5. **allowed_signers**: This file stores a list of signer's identity (typically an email address) and their authorized SSH public key.

5.2.3 How peers are identified?

Our peers are identified by their public and private keys. We generate a public-private key pair using *ssh-keygen* command. These keys are then stored in *key* and *key.pub* file as described above. These keys are useful for authentication purposes. Whenever an author creates commits, these commits are then signed by their own private key. These commits can be verified by their own public key.

We want our commits to be verified by other peers, and therefore, the public key is shared among all the peers in the network. *How are keys shared among the peers?*

When the *repo-init* script is run, it internally creates a single commit after the public-private key is generated. This commit only stores a plain-text public key of an author as a commit message. This commit is then labeled by a git reference, which acts as a pointer to this commit message.

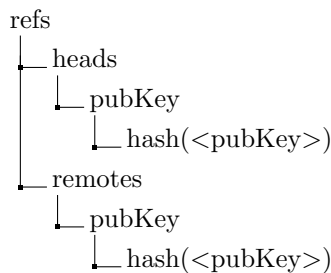


Figure 5.1: Git Public Key Reference

In the above figure 5.1, the git reference structure of the public keys is shown. The prefix *refs/heads/pubKey* means that this is the key belongs to the local git repository. There is only one local key, which was generated by the *repo-init* script. The prefix *refs/remotes/pubKey* belongs to the remote peer's public key. There can exist multiple remote keys. The suffix for both local and remote is a hash value of the public key. The hash is computed by passing the string to the **SHA-1** function, which generates a 40-alphanumeric long hash value.

These references can be viewed as a branch in a git repository, so that peers can now perform push operations on git branch to multiple peer git repositories.

One more thing we would like to highlight is that, when the *repo-init* script is run, it also copies some pre-defined hooks which are necessary to perform certain operations when an event occurs. Events like a push received from remote peers or a local commit operations is performed.

When a public key is received from remote peers, a hook name *update* is run, which then extracts the contents, like the public key and email of the remote peer, from the received public key branch. These contents are then appended to *allowed_signers* file. Now, when

Git checks a signature on a commit, it finds the signer's email in the commit, looks up that email in the *allowed_signers* file, and attempts to verify the signature using the corresponding public key.

In short, whenever a remote/friend is added to the git repository, we also share their public key that is inside the commit object, and these commit objects are pointed by *refs/remotes/pubKey/hash(<pubKey>)*. These commits are always signed by the peer's private key. This ensures that a valid remote peer is added to the local git repository. However, these peers are not always genuine or cannot be trusted. Security is not the primary goal of this thesis. This could be implemented as a future extension to this project.

However, our app does not require any personal phone numbers or email ids to create or log in to your account before using this application. We only require an email id which is a mandatory metadata to create commit messages. Our app is completely command-line based, UI could be a future extension of this project.

One more thing, our app only runs on a local private network. Once the git repositories(peers) are initialized, the peers need to add the remote with the full path of the other existing repository using the *git remote* command. Before adding the remote peers, the peers are needed to share their SSH public keys and disable SSH login permission for security reasons. However, if multiple git repos are initialized on a local machine then there is no need to share the SSH keys. Although we did not focus on the communication part, it can be a part of future extension.

5.3 Core Group Messaging App

In this section, we now discuss the implementation of our core group messaging app and how we map over git.

5.3.1 Attributes

Chat applications [11] that support group messaging require information/metadata to identify a group and are stored in a database. Some common data are group id, names, and descriptions. Our implementation model of the group chat application mimics the state of the group chat applications that store the information about the group, and the following attributes are,

1. **Group Id** - It is a 40-character unique immutable hash value generated by concatenating the *group name* with *time seconds since epoch* and then applying the SHA-1 hash function over the concatenated string. The group ID is not visible to the users/peers; it is stored internally inside the git tree object and in git references. With a unique group id, it helps to distinguish groups with similar name or it is possible to create multiple groups with similar names.

e.g. **169f5a9b392711d6855f8629071092fcb998afcd**

2. **Group name** - It is a mutable string that contains alphanumeric values with no spaces and can be modified by an admin to a different name. The *name* of the group

is visible to all the members of the group. Members can call (3.2) API that returns a tuple which includes the group name too.

e.g. **Hiking, Meeting, Picnic** etc.

3. **Group description** - It is a mutable string that describes the role of a group. The group description can also be modified by an admin. The *description* of the group is visible to all the members of the group. Members can call (3.2) API that returns a tuple that includes the group descriptions too.

e.g. **Hiking with PhD students.**

These three attributes are sufficient to identify a group of chat applications.

5.3.2 Group State

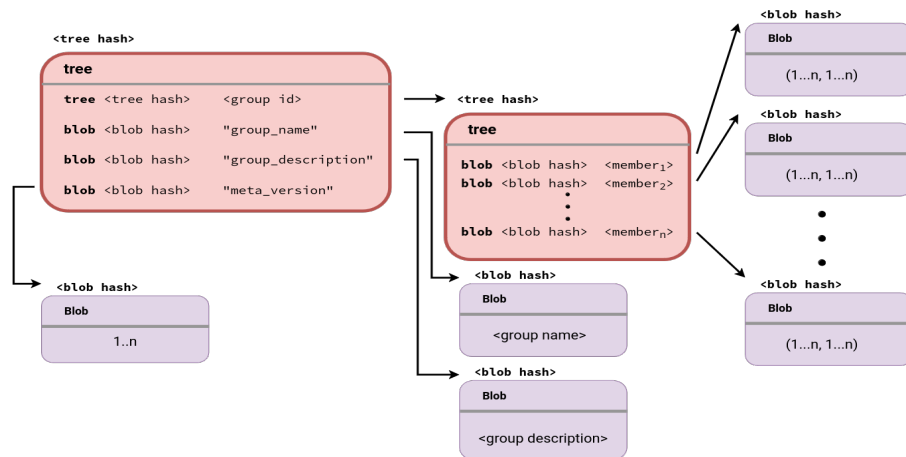


Figure 5.2: Chat Group State

The *group state* is the core part of the group chat applications that stores the information/metadata of the group, i.e., name, and description. In addition it also stores both members and admins that are present inside the group. This section talks about the detailed explanation of our group state/object and how we map them over the structure of git objects (*trees*, and *blobs*) (2.2). The *tree* object is like a *C struct* that groups several different variables, whereas *blob* can be imagined as the values that are stored inside a variable.

Fig 5.2 is a depiction of a group state model that is mapped over nested git objects, and the flow of the diagram goes from left to right. The top tree object (left side) consists of 4 variables, and three of them are *blobs* and one is the *tree* object. Remember, a *tree* object can nest multiple *tree* objects (2.2). The names of the blobs(variables) are static, but the values stored inside the blobs are dynamic, depending on what information a user wants to store. The first blob is *group_name*, which stores the name of the group, the second blob is *group_description*, which stores the description about the group, and the third blob is *meta_version*, which is the counter clock that stores a single positive integer value. The role of the *meta_version* is already described in 3.

The fourth variable is the *tree* object, which is CL-CRDT[68]. The name of the tree object is dynamic, which is the unique *group.id* created internally by the chat application (1). This tree object stores a limited *n* number of blobs(variables), and the names of the blobs are dynamic because it stores the names of the members that will be added inside the group. The values inside the blobs are a tuple of two integers. The integer values are the metadata of the causal-length CRDT (2.3.2). The *first* value of the tuple indicates the presence of a member inside a group, and the *second* value of the tuple indicates whether the member is an admin or not.

5.3.2.1 Example of Group State



Figure 5.3: Example of Group State on Git

Fig 5.3, illustrates the example of group state model. The group name is *hiking* and the description of the group is *With Friends*, which was created by *alice*. The meta value is 1 and will start with value 1 whenever a group is created. The tuple value for *alice* is **(1,1)**, and it indicates that she is inside the group and also an admin of the group. Whoever creates a group automatically becomes the admin of the group (5.1). *alice* then adds *bob* and *carol* to the group *hiking*. Their tuple values are **(1,0)**, and it indicates that they are only present inside the group as a normal member.

5.3.3 Commits For Group Chat App

Peers/users that make API (3.2) calls to generate or modify the group state are implemented as a git commit object. Fig. 5.4 is a depiction of the commit object associated with the group state of the chat application.

The commit object has various attributes such as *tree* that stores the SHA1 value of tree object, then the names of the *author* and *committer* who created the git snapshot, *date* that stores the time when the commit object was created, *parent* that stores the hash value/values of previous commit/commits, *commit-message* that stores the purpose of the commit object, and *signature* that stores the private-key of the author. Further in the table (5.1) shows the purpose of the attributes that we are storing in our git commits of the group chat application.

Each function/API call of the group chat applications is referred to as an event, where the event is about a group operation performed by users, or it could be the messages that

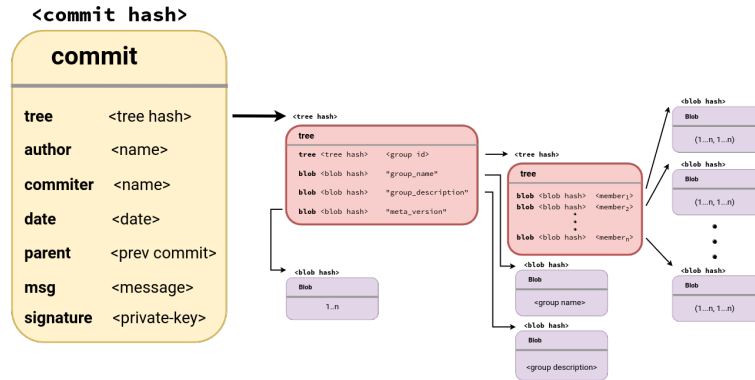


Figure 5.4: Git commit used in our group chat app

Table 5.1: Commit attributes

Attributes	Purpose
Commit tree	Hash of the tree referencing our group state model.
Author name	Member of a group that performs group operations.
Committer name	Same as author name.
Commit Date	The time at which the group operation API was called. Git internally creates date and time for the commit objects.
Parents	Referencing to the previous commits created by the members of the group.
Commit Message	Messages sent by the group members or the messages created by the APIs.
Signature	Signed commit created by the members of the group.

are exchanged between the members of the group. We want these events to be in a timely order. To ascertain the order of events, git commit objects are used. The *date* attribute of git commit ensures the ordering of commits. The format of the date is `<unix_timestamp> <time-zone-offset>`, where `<unix_timestamp>` is the number of seconds since UNIX epoch, whereas `<time-zone-offset>` is the positive or negative offset from UTC. Here the *unix_timestamp* of every commit is compared, which gives us an order of the git commit objects. With the property of the date attribute, we then don't have to worry about the ordering of events.

We have also implemented the message format for each API call that are stored in the commit message attribute of git commit objects. These messages will help the group members to know what group operation was performed.

In the table 5.2, show the message format of each API call that is stored inside git commit message attribute. The *send_grp_msg* API doesn't have a format because it is conversation between the members of group that doesn't require any message format. The *merge_group_state* doesn't have any format because it doesn't store any messages, and this API is internally called by our system, which is the chat application.

The message format of these API are not enough for users to clearly distinguish between the conversation messages and group operation messages. Chat applications like WhatsApp, Signal, etc., have set specific UI characteristics like fonts, font-size, and color for these group

Table 5.2: Message Format for each API

API	Message Format
create_group	'<group-name>' group created by '<user>' e.g., 'hiking' group created by 'user'
add_member	'<member>' added to group '<group-name>' e.g., 'bob' added to group 'hiking'
remove_member	'<member>' removed from group '<group-name>' e.g., 'carol' removed from group 'hiking'
exit_group	'<member>' exited the group '<group-name>' e.g., 'alice' exited the group 'hiking'
rename_group	'<old-group-name>' changed to '<new-group-name>' e.g., 'hiking' changed to 'picnic'
promote	'<member>' is now admin of the group '<group-name>' e.g., 'bob' is now admin of the group 'hiking'
demote	'<member>' is no more admin of the group '<group-name>' e.g., 'bob' is no more admin of the group 'hiking'
send_grp_msg	'<anything>' e.g., "Hi, how are you?", "Welcome to the group," etc.
merge_group_state	No format

operation APIs messages and for conversational messages when they are displayed on the UI interface.

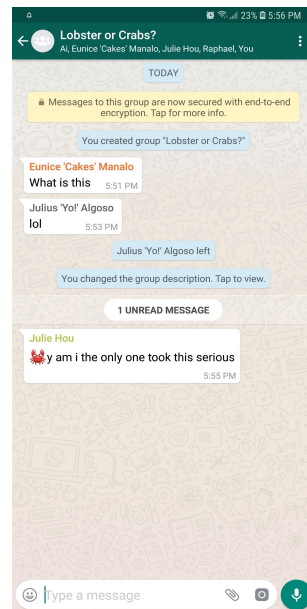


Figure 5.5: Screenshot of a group chat [29]

Fig 5.5 is a screenshot of a WhatsApp group. You can see that the group operation messages have specific UI characteristics, and the conversation messages have different UI characteristics. Various UI components for these messages give users better clarity in dis-

tinguishing between conversation messages and messages created by group operations.

But how does the UI distinguish between these messages? When messages are fetched from the database, each message is associated with some metadata [11, 13] that helps the UI to distinguish between the messages. In our implementation, we don't have metadata associated with each message but instead wrap the messages in a specific string, which is depicted in the table (5.3).

Table 5.3: Messages Wrapped in a String for UI interface

Message Type	Message Format
Group Operations Messages	GROUP_OPS(<message>, <operation-type>) e.g. GROUP_OPS('bob' added to group 'hiking', add) e.g. GROUP_OPS('carol' removed from the group 'hiking', remove)
Conversational Messages	MSG(<message>) e.g. MSG(Hi, How are you?) e.g. MSG>Welcome to the group...)

The message format shown in the table (5.3) is the final message that is stored in the commit message attribute of the git commit object. Now, with this messaging format, our UI API can distinguish between the message types and can display messages with different colors, so that it will be easy for users to distinguish between the messages.

The chats of a group are like a sequence/series of git commit objects called commit-graph where each commit object is a group operation (3.2). The whole commit graph is the group chat. The latest commit object in the commit-graph is pointed to by a git reference (5.3.4). Reference is also known as branch. In a git repository, we can create multiple branches. This implies that we can have many group chats as long as the branch names are unique.

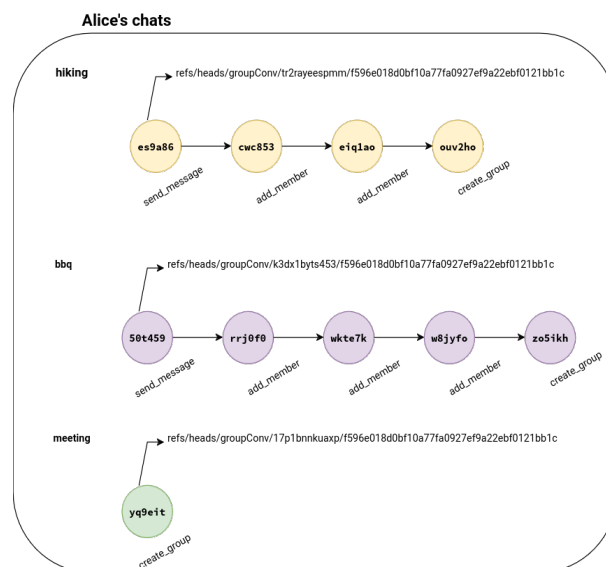


Figure 5.6: Multiple groups as commit graph

Fig. 5.6 is a trivial representation of group chats in a git repository. An author *alice* who

currently has 3 group chats in her git repository named *hiking*, *bbq*, and *meeting*. Each group is a commit graph that contains a sequence of git commit objects. The first commit object of the group will always be the *create_group* operation with no parents. Whenever a new group operation is performed, it will create a new commit object setting its parent attribute to the hash of the previous commit object, and the reference will point to the latest commit object. In reality, a group chat commit-graph contains multiple merged branches depending on the number of members present in a group.

5.3.4 Group Messaging Commit References

In this section, we discuss in detail the references that point to the latest commits made by the member/peer of a group. The structure of the group chat commit *reference* is designed to be unique and helps to distinguish between the groups in our chat application.

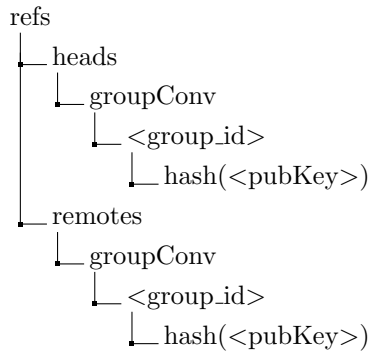


Figure 5.7: Group messaging commit references

In fig. 5.7, there are two references. The *first* reference with the prefix *refs/heads* is referred to a local reference created by the APIs (3.2) locally in the local repository. The *second* reference with the prefix *refs/remotes* is referred to as a remote reference that was pushed by the remote peers. The following suffix to *refs/heads* and *refs/remotes* is *groupConv*.

It indicates that the reference is pointing to the commit-graph (sequence of commits) of a group chat. The suffix of *refs/heads/groupConv* and *refs/remotes/groupConv* is always dynamic because the following suffix is *<group_id>* and it is the id of the group. Whenever a user creates a group with a different or the same name, the API (3.2) will always generate a new ID for the group (1). This ID is then set as a suffix to *refs/heads/groupConv* and *refs/remotes/groupConv*. The suffix of *refs/heads/groupConv/<group_id>* and *refs/remotes/groupConv/<group_id>* is *hash(<pubKey>)* which is always different. It is the SHA1 hash of the public key of the author/peer of their respective git repository. From our systems' assumption 5.1, we have mentioned that each peer is identified by their public key. To make a clear distinction between the members of a group, we have the hash of their public-key as a suffix to *refs/heads/groupConv/<group_id>*.

Depending on the number of members of the group, there are always N references shared among peers when all peers have eventually synchronized their history of messages from a particular group. For e.g. there is a group id *1a2bc21xf911*, and in this group, there are 10 members; then every peer will have a local group reference *refs/heads/groupConv/1a2bc21xf911/hash(<pubKey>)*.

Once these are shared among peers, their suffix will change from *refs/heads/* $\rightarrow$ *refs/remotes/*.

The following suffixes to *refs/heads* and *refs/remotes* can also be set to something different, which depends on the future extension of this project.

5.3.5 Merging Groups

Merging groups guarantees strong convergence and eventual consistency. We have implemented the *merge_group_state* API (7) inside git hook script [19, p. 353]. The hook acts as a listener; whenever a git push is received from the remote git repository, these hooks are executed inside the local git repository. There are many predefined hooks, and we are only using the *post-receive* hook for merging the two group states.

The *post-receive* hook takes two inputs as arguments. The first argument is the hash ID of the newest/latest commit object, and the second argument is the remote reference. In our case, the remote reference that is received is *refs/remotes/groupConv/<group_id>/hash(<pubKey>)*. Whenever a push is made from the remote repository, the local repository receives the whole commit-graph with the remote reference. These remote references and the first commit object of the commit-graph are passed as arguments to the *post-receive* hook script. From the received commit object, we extract the tree object (5.3.2), and from the tree object, we extract all the attributes to compare with local group state attributes. But, before making the comparison, we first check if the local repository has the git branch for the respective group ID. We first go through all the git references and look for the specific group ID in the reference. If the reference is not found, we create a new commit setting its tree ID equal to the received group state tree ID from the remote, setting its parent to the received commit graph, and setting the commit message "MERGE_OPS". After creating a new commit object, we create a new local reference that points to the latest commit object.

In other cases, if a local reference is found concerning the group ID, we then extract all the attributes from the local latest commit object for that group ID. We compare and merge the local attributes with the received group state attributes and then create a new group state. The new group state is now set to the tree ID of the new commit object, and then updating the local reference pointing to the latest commit object.

We've also implemented the above merge function in one of our custom script when a git fetch is made. Since, there are no hooks available when a git fetch is made on remote references, we wrote the merge function logic in one of our custom script after the git fetch was called.

5.3.6 Design Choice

While designing the group chat application, we expected certain behaviors that could occur. Below we illustrate few cases:

⟨1⟩ If an admin wants to exit the group?

CASE 1. Only one admin is present inside the group. ($m = 0, a = 1$)

If an admin exits the group, the ownership of being an admin is also taken away. The exited admin cannot enter the group again. The group is non-functional, and no one can enter the group again.

CASE 2. Multiple admins + other members in the group. ($m \geq 1, a > 1$)

If an admin exits the group, the ownership of being an admin is also taken from the group. Exited admin can re-enter the group by requesting to other current admins present inside the group.

CASE 3. Only one admin + other members in the group. ($m \geq 1, a = 1$)

If an admin exits the group, the ownership of being an admin is also taken away. Now there are no admins inside a group. In this case, we promote all other members of the group to admin. We chose this design choice because we don't want the group to be semi-functional, where there are no admins present inside the group. We could randomly pick a single member from the group to promote to an admin. But we don't know if the selected person is a regular active member of the group. If a non-active member is promoted to an admin, then the group is still considered semi-functional, which is not the choice we want to make.

CASE 4. Only two admins are present inside the group. ($m = 0, a = 2$)

If both admins exit the group or try to remove each other concurrently, then they are not part of the group. The group is non-functional.

⟨2⟩ If an admin wants to self-demote?

CASE 1. Only one admin is present inside the group. ($m = 0, a = 1$)

If the admin self-demotes, the admin is now a regular member of the group. The group is semi-functional. The member that was an admin before can only send messages (3.2) or exit (3.2) the group. Can't perform any other group operations.

CASE 2. Multiple admins + other members in the group. ($m \geq 1, a > 1$)

If the admin self-demotes, the admin is now a regular member of the group. The group is still fully functional since other members, plus admins, are present inside the group.

CASE 3. Only one admin + other members in the group. ($m \geq 1, a = 1$)

If the admin self-demotes, it is now a regular member of the group. Now there are no admins inside a group. We then follow a similar case from [⟨1⟩. CASE 3.] (13)

CASE 4. Only two admins are present inside the group. ($m = 0, a = 2$)

If both the admins self-demote at the same time and if the push updates are made to each other, they again become the admin of the group. See *Chapter 3* (7).

From [⟨1⟩. CASE 3.] and [⟨2⟩. CASE 3.], we mentioned that all the normal members are promoted to become an admin. But this happens only when the self-exited/self-demoted admin makes a push to at least one group member. But here is the catch: the admin could be a byzantine group

member. The admin never makes a push. For this case, we leave this as a future implementation. For now, we assume all the members are trustworthy/non-byzantine (5.1). In P2P, networks do not assume trust, and it is more likely to expect Byzantine behavior.

5.4 Programming Languages Used

All the APIs (3.2) are implemented both in *Bash* and in *Rust*. The *Bash* script contains low-level(plumbing) git commands. Bash is more flexible, where you can directly run Linux commands inside the scripts, and can automate all your tasks. We've used a few low-level git commands to create our chat application. Those commands are *git hash-object*[19, p. 419], *git update-index*[19, p. 311], *git write-tree*[19, p. 423], *git commit-tree*[19, p. 425], *git show-ref*[19, p. 369], *git cat-file*[19, p. 252], *git push*[19, p. 438], *git ls-remote*[19, p. 85], *git fetch*[19, p. 52] and *git log*[19, p. 40].

The *git hash-object* command is used to create blobs. In our group state (5.3.2), we are storing the group name, description, and meta value as blobs. Also, we are storing the tuple values of each member as a blob.

The *git update-index* command is used to modify/update the *git index* file. With additional arguments passed, which are *-add* and *-cacheinfo*, artificially adds the file to the staging area. The file doesn't have to exist in the git repository. After executing this command, you can run *git status*. You will see the file is added to the staging area. We use this command to add all the group attributes (5.3.1) and the members to the staging area.

The *git write-tree* command is used to create a git tree object of all the files and directories that are currently inside the staging area. If no files/directories are found in the staging area, this command will always return *4b825dc642cb6eb9a060e54bf8d69288fbee4904* hash value. We run this command to create a group state, which is a tree object that contains all the group attributes and members. This command returns a SHA1 hash ID, which will later be used for creating a commit object. We then run the *git commit-tree* command to create a snapshot of our current group state.

The *git cat-file* command outputs the contents of the git object, and the *git log* command is used to check all the git logs of a commit-graph. The git logs help to extract the messages exchanged between members or the messages created internally by the APIs.

The *git push* command is used for pushing the messages to the remote members that are present inside the group. We pass the remote name, which is the member of a group, and the reference related to the specific group as parameters to the *git push* command. The git push is made over network operations. Whenever a push is made, it transfers all the git objects of the commit-graph pointed by the git reference from the sender's side to the receiver's side.

The *git ls-remote* gives you a full list of remote references, and *git fetch* fetches the specific branch from the remote repository.

However, the APIs written in bash using git commands gave us poor performance. The reason is because of the *git update-index* command. Each time this command is executed, it will always modify the *.git/index* file. There is an overhead for reading and writing the *.git/index* file. There are no git commands that can load the entire *.git/index* file in the memory. Whenever a group state is modified, we have to add all the content of the group state object in the git staging area to create a new git tree object. This makes it a very expensive operation. E.g., if there are 1000 members inside a group, and we want to modify only a single member. We have to execute *git update-index* 1000 times to add all the members inside the staging area. In addition, we also add the group attributes in the staging area to get a full group state. Executing *git update-index* 1000 times means reading and writing *.git/index* file 1000 times. This process will make all the operations slow.

Therefore, we implemented entire APIs in *Rust* [38] using *libgit2* [20] library for testing purposes.

With the help of this library, we can directly call the functions of the git low-level commands. This gives more flexibility because we can now directly load the entire *.git/index* file in the memory, and we can directly modify the group state from the memory. E.g., if there are 1000 members inside a group, we can directly fetch the group state from the index file and load it into memory, modify the values of a member, and then create a new tree object. This is done in a single operation call. However, Rust is a compiler-based programming language that compiles the source code to machine code that the processor can execute. Due to this, the execution speed of any compiler-based languages are faster compared to scripting-based languages.

6

Evaluation

In this chapter, we will have each section for our two experiments, *Experiment 1*, and *Experiment 2*.

6.1 Experiment 1

In *Experiment 1*, we are testing how does the insert cost grows of the *Git* DB. Since, our chat app is build over the top of *Git* and storing necessary information like group messages, group members/admins, group name etc, information in the *Git* objects (*blobs*, *trees*, and *commits*). This experiment conducts consecutive addition of a member inside a group, which internally creates *Git* objects followed by a push operation to the remote peer (git repository). The insertion cost of this experiment was conducted with both non-GC (non garbage collector) and GC (garbage collector) with different GC intervals (128, 256, 512 inserts), to evaluate the impact on the storage footprint of the *Git* DB.

6.1.1 Methodology

Our setup includes, 2 *git* repositories named *peer1* and *peer2* setup locally on single machine. Our machine specs are

1. **cpu** - Intel i7-6700HQ CPU @ 2.60GHz 64-bit processor. (released in 2016)
2. **ram** - 16 Gb ram size with 2400 MHz speed.

Inside *peer1*, we initiated a group with the group name *hiking*, and description *With family..*. We have written a bash script, which consecutively adds a member inside the group, followed by push to *peer2*, and records the interval times of inserts and push (with merge operation). Note that the insertion and push times are recorded separately (not combined). For each reading, we also record the size of the git db, git object, and git delta object counts using '*git count-objects -v*' command. All these values are then appended into a .csv file after each insertion and push operation.

The table below explains each field of our csv file.

Field	Data Type	Description
add-mem-time	int	Unix time which represents the time of insertion of a member in a group.
diff-add-mem-time	float	The time taken (difference) to add a member inside a group.
push-time	int	Unix time at which the push operation was made.
diff-push-time	float	The time taken to complete the push operation including the merge operation.
git-size	int	The size of the <i>git</i> db.
count	int	Total number of <i>git</i> objects.

Table 6.1: Dataset

The experiment was conducted in a non-GC as well GC environment. For GC, we have 3 experiments for GC interval 128, 256, and 512. For each GC interval means after specific insertions, we run ‘*git gc*’ command.

Each run was conducted 5 times, and then we took the average values of the time difference for insertion, push, git db size, and object counts. In total, 20 runs were conducted for Non-GC, GC-128, GC-256, and GC-512.

6.1.2 Results

Below are the following insights drawn from *experiment 1*, consisting of 4 sections. Out of 3 are line plots, CDF plots, and bar plots. In the final section we discuss our findings.

6.1.2.1 Moving Average Plots for Insert and Push Latency

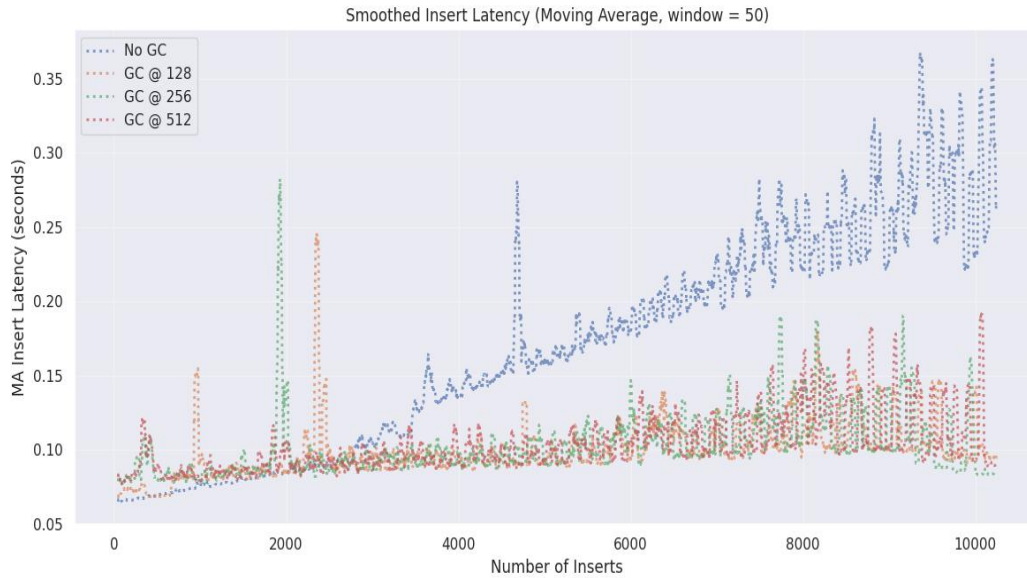


Figure 6.1: Moving Average Inserts vs. Number of Inserts

Figure (6.1) depicts the smooth insert (consecutively adding member inside a group) plot by taking the moving average (window=50) of the raw data. The *No-GC* variant shows a steady

upward slope, indicating a performance degradation over time. Latency increases steadily as the repository grows. In contrast, *GC* variant maintains a steady latency profile exhibiting stationary performance. In *GC*, cost occurs only at certain intervals, and when averaged over time, it becomes stable. *GC-128* and *GC-256* almost looks identical and noisier. We spot 2 large spikes between ($\sim 2000-2400$). *GC-512* exhibits controlled behavior over the time.

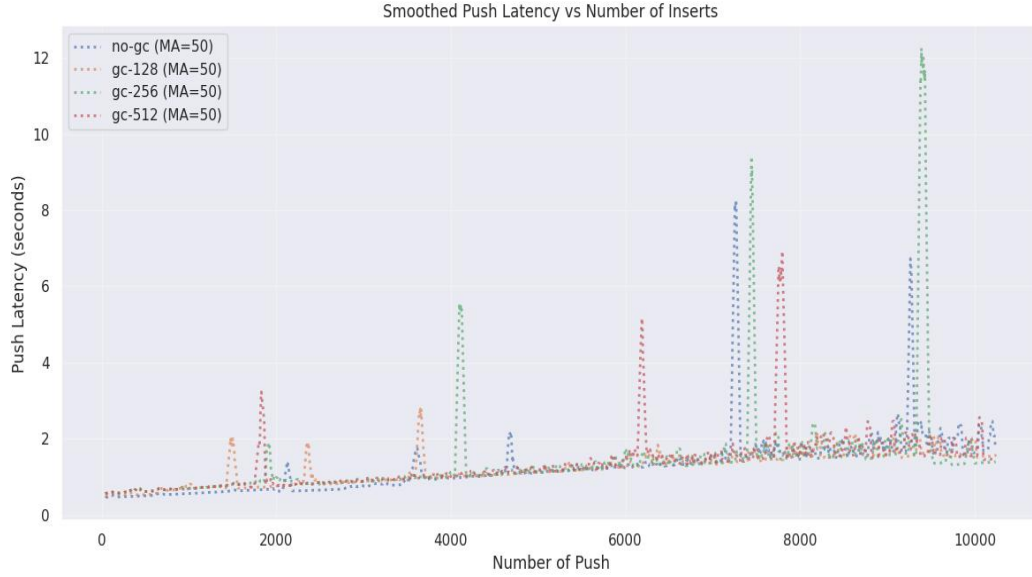
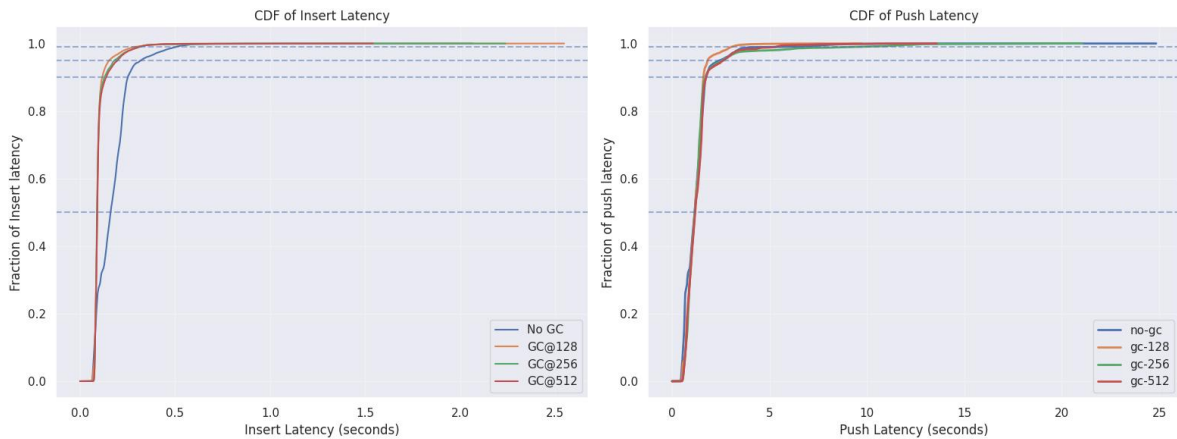


Figure 6.2: Moving Average Inserts vs. Number of Inserts

Figure (6.2) is a smooth push plot. All curves rise gradually. *No-GC*, *GC-256*, and *GC-512* showed anomalous behavior with occasional spikes. *GC-128* has outperformed all with fewer small spikes around ~ 1800 , ~ 2200 and ~ 3800 .

6.1.2.2 CDF of Inserts and Push



(a): CDF of Insert Latency

(b): CDF of Push Latency

Figure 6.3: CDF of Insert and Push Latency

In Figure (6.2(a)), we demonstrate the fraction of insert latency. We observe that the *GC* curves shift to the left while the *No-GC* curve is slightly shifted to right. At median (P50), *GC* latency is around (~ 0.08 - 0.10 s) while *No-GC* is around (~ 0.18 - 0.20 s). *GC* nearly halves the insert latency at the median. At tail latency (P90, P95, & P99), *GC* clearly dominates with lower inserts latency. Overall, *GC* dominates in both median and tail latency.

In Figure (6.2(b)), we demonstrate the fraction of push latency. We observe that all the configurations rise steeply between (~ 0 - 0.25 s). Even at the median(P50) latency, all the curves look the same at around (~ 1.1 - 1.3 s). We don't see any performance improvement from *GC*, nor performance degradation from *No-GC*. But from the region (P90-P99), *GC-128* rises faster and shows a better tail, while *No-GC*, *GC-256*, and *GC-512* have a slow climb with a heavier tail. Overall, *GC-128* improves tail latency for push operations by controlling repository growth.

6.1.2.3 Bar Plots of Repository Size and Git Objects

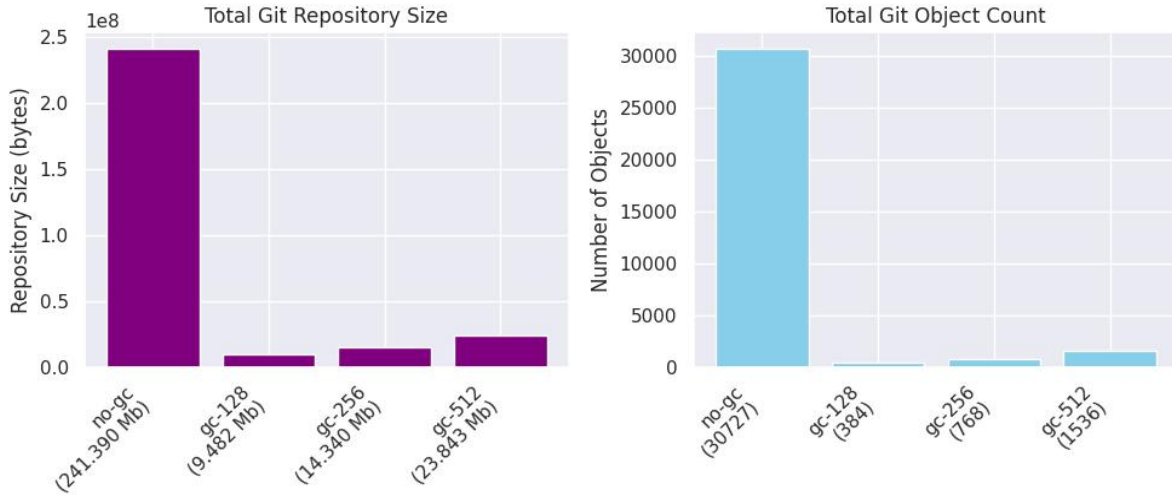


Figure 6.4: DB Size and Object counts

Figure (6.4) depicts two bar plots; the first compares the final repository size, and the second compares total remaining git objects within all the configurations. In both the plots, *No-GC* exhibiting huge storage size with a large number of git objects in the git db. *GC* induces a smaller disk footprint with less number of git objects. *GC-128* reduces storage size by 184.83%, *GC-256* reduces storage size by 177.536%, and *GC-512* reduces storage size by 163.989%.

6.1.2.4 Discussion

From our experiment results, periodic garbage collection (GC) clearly demonstrated an improvement in the long-term performance characteristics of our group chat application. The smooth insert latency plot disclose a clear distinction in the performance between No-GC and GC variants. Without GC, insert latency shows a steady growth over time, with the growth in repository size and object accumulation directly impacting operational cost. This behavior reflects that our app's performance is not stable over time, and the latency keeps increasing because of past workload (previous operations). In contrast, GC configuration maintained nearly a flat moving average latency curve, indicating that periodic compaction prevents unbounded growth in the operational

cost. However, GC exhibited occasional spikes due to compaction events, but the baseline remained stable throughout the experiment. The GC event confirms performance improvement in our chat application, despite having occasional controlled pauses; overall, it keeps the latency stable. However, smooth push latency exhibited a different pattern than smooth insert latency due to its strong dependency on object packing. For all the configurations, the latency increases gradually due to growing data volume. Except for GC-128, all other configurations showed heavier extreme spikes.

The CDF of insert latency further affirms the improvement across the latency distribution. All GC variants significantly shift the distribution leftward relative to No-GC, reducing both median and tail latency (P90, P95, and P99). This proves that the performance by GC is not limited to average-case but guarantees performance at high-percentiles and effectively stabilizes insert performance over time. Although the CDF of push latency increases gradually in all configurations due to growing data volume, GC-128 slightly improves extreme tail behavior by limiting the growth of loose objects. Even though GC performs extra cleanup work, it does not negatively impact normal push performance.

Finally, the bar plots for repository size and object counts provide an explanation for these latency improvements. The No-GC configuration results in dramatic growth, while periodic GC significantly reduces disk footprint and maintains compact object structure. This reduction in storage validates the results of a stable lower insert latency and moderate push tail latency.

Overall, our results demonstrate that the periodic GC not only contributes in the storage optimization but also fundamentally improves the performance of our system. GC prevents uncontrolled object growth, and improves latency percentiles.

6.2 Experiment 2

In *experiment 2*, we test the replication strategy for our chat application using *Susceptible Infected* (SI (epidemic-algo-replication)) *gossip (push)* and *anti-entropy (pull)* protocols (2.5.3) to synchronize our *git* DB.

6.2.1 Methodology

The total number of nodes used was 11 and designed in both *the mesh* and *the tree* topology. Each node is a Docker virtual machine with specs:

1. **cpuset** - Our cpuset limits to cores are 1-2.
2. **memory** - Each virtual machine is assigned 1 Gib of RAM.

Inside each Docker virtual machine, we create a peer (git repository) and arrange these Docker virtual machine in a topology which is discussed below. The idea of the topology design was taken from the referenced paper [54].

Figure (6.5) represents 11 nodes arranged in a ring where each node has a minimum degree of 2 and a maximum degree of 4 nodes. The vertex connectivity is (≥ 4), and the edge connectivity is (≥ 6). This topology is perfect for our test because it almost certainly stays connected, as multiple alternate routes exist. In addition, we get to test the replication strategy with different sets of k neighbors. We tested for each $k = \{1, 2, 3, 4\}$ neighbors selecting neighbors non-deterministically for k size to push/pull the updates.

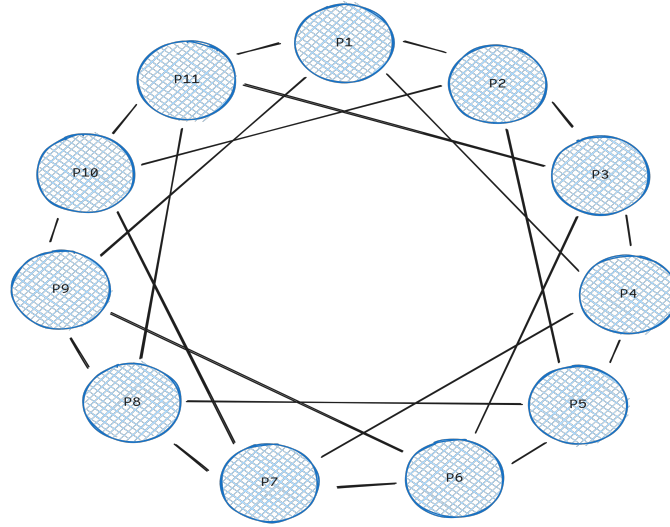


Figure 6.5: Hybrid Mesh Ring Topology

Figure (6.6) represents a binary tree topology where at level 3, each node has a single child. In our tests, each node deterministically chooses the neighbors to push/pull the updates. This topology represents the spanning tree architecture with no cycles.

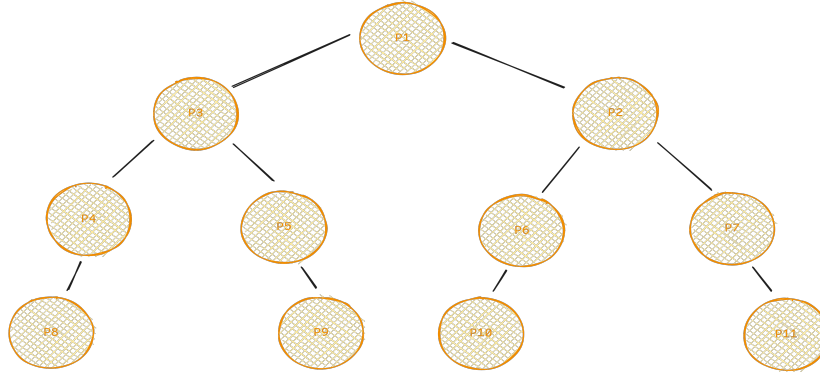


Figure 6.6: Binary Tree Topology

At the start of the experiment, at each node, we run a script that periodically push/pull with the gossip cycle of 5 seconds, and we deterministically stops the script after 700 seconds. In the script, we record the output of the *git push* and/or *git pull* and append the output in a log file with additional information. Figure (6.7) is an illustration of raw push/pull git output.

```
2025-12-06 10:54:06 1764307446207.974064000, from:peer1, to: peer2, tree_id:0c5ffd256d7e8f3d68c3ebc279d0fd25a0708764, group_name: hiking
Enumerating objects: 23, done. Counting objects: 4% (1/23)
Counting objects: 100% (23/23), done. Delta compression using up to 8 threads Compressing objects: 5% (1/20)
Compressing objects: 100% (20/20), done. Writing objects: 9% (2/21)
Writing objects: 100% (21/21), 3.17 KiB | 3.17 MiB/s, done. Total 21 (delta 18), reused 0 (delta 0), pack-reused 0 (from 0) remote: Checking connectivity: 21
, done. remote: post-receive: oldrev:130d56f449b34ef27f1836e05d2f31348a141f82,newrev:9cef1bdc91f29552326e1ac992c9ed78a638281b,push_recv_time:2025-11-28 10:54
:06 1764307446198.310749000, ,push_from:unknown,merge_time:.2847119930,no_mem:7,first:false To ../peer2 130d56f..9cef1bd groupConv/e0ab8903bd2d/e32e8549b699b
584d99a62cc2085d0f5da392c02 -> groupConv/e0ab8903bd2d/e32e8549b699b584d99a62cc2085d0f5da392c02
```

Figure 6.7: Raw Push/Pull Output

We then process and extract the valuable information from the file and store the extracted value in our own custom .csv file. Below is the table that explains each field of our dataset.

Field	Data Type	Description
p_time_from	Date	Stores date, time, and unixtime.
from	String	Local peer name. Inversely for pull operation
to	String	Remote peer name for push. Inversely for the pull operation.
tree_id	String	Newly generated group state tree object ID (5.3.2).
objsize	Int	Size of the objects from push and/or pull
total_objs	Int	Number of objects from push and/or pull.
delta_objs	Int	Number of delta objects from push and/or pull.
commit_id	String	Newly generated commit object (group messages (5.1)).

Table 6.2: Dataset for Replication Tests

We conducted 3 experiments for *single writer* with and without GC, and with *three writers* only without GC. Writers that perform operations are stated below (10).

Algorithm 10 Group Operation Summary

```

1: // Randomly generate peers in range peer1..116
2: // Add generated peers to the group
3: // Insert 1–3 random messages between each add operation
4:
5: // Promote 20 randomly selected existing members
6: // Insert 1–3 random messages between each promote operations
7:
8: // Add 28 members from range peer116..134
9: // Insert 1–3 random messages between each add operation
10:
11: // Remove 30 randomly selected existing members
12: // Insert 1–3 random messages between each remove operation
13:
14: // Demote 30 randomly selected existing members
15: // Insert 1–3 random messages between each demote operations
16:
17: // Again add 121 members from range peer134..255
18: // Insert 1–3 random messages between each add operation

```

In an average, 1078 operations were performed, and a total of 255 peers were added inside the group.

With *single writer*, node **P1** from both the topology performs the above operations, and with *three writers*, nodes **P1**, **P4**, and **P9** performs the above operations. The above script is run immediately after the push/pull script is run from all the nodes.

For GC (garbage collector) in a single writer, the GC command is executed after 70 gossip cycles in the push/pull scripts. It is ran only once whole the push/pull operations.

Besides the above operations, all nodes run periodic push (gossip) and pull (anti-entropy) script which are discussed below:

1. **Push (Gossip)** - In the push script, after every 5 seconds, depending on the *ks*, a random peer is selected among the neighbors, and the *git push* command is executed by pushing the latest updated local(head) git reference of our group. The receiving peer then performs the merge operation via *git hooks*. Every time a push is received, a merge operation is performed by creating a new *git commit* at the remote peers.
2. **Pull (Anti-Entropy)** - In the pull script, *git fetch* is run every after 5 seconds. Since, there

are no *git hooks* for *git fetch*, we had to write our merge algorithm in the pull script. We fetch the references that are available at the remote peers (neighbors). Before fetching, we first list out the references through the *git ls-remote* command and then fetch the references. Not every time the merge operation is performed (and creating a new git commit). From the list of references, we first check if the remote refs are present in the local peer or not. If not, we fetch, and a new merge is created. If remote ref is already present and has a different commit ID, then the group information(remote commit) is fetched via *git fetch*. We first extract the tree id (group state information) from the fetched commit and then compare it with the local tree id from the local latest group commit message. If the tree IDs are equal no merge is performed else, a merge is performed, and a new group commit is created for the group.

Metrics used are,

1. *Residue* - Number of nodes that do not receive updates for each gossip cycle. A lower residue indicates faster and more effective state dissemination.

Residue at cycle t is defined as:

$$R(t) = \frac{N - N_{\text{updated}}(t)}{N}$$

Where N denote the total number of nodes in the system and $N_{\text{updated}}(t)$ denote the number of nodes that received new state information during gossip cycle t .

For each gossip period, we measured how many peers have seen the *tree_id* and *commit_id*. For each *tree_id* and *commit_id*, we apply the above formula to compute the residue. These residue values are then sum which gives us the final residual value.

2. *Traffic* - Number of messages sent from a typical site for each gossip cycle. Higher traffic indicates higher communication cost.

The total traffic at cycle t is defined as:

$$T(t) = \frac{1}{N} \sum_{i=1}^N M_i(t)$$

Where $M_i(t)$ denote the number of messages sent by node i during gossip cycle t .

For each gossip period, for each peer, we count how many of them have exchanged data. Whether it was pushed to or pulled from. After summing we normalized it by dividing by the number of peers.

3. *Convergent* - Nth gossip cycle that tells, how many gossip cycles are required for all the nodes to converge.

To compute the convergence, we took the difference between the adjacent residual values. We have a constant value 1e-3 and compared it with the differences. If the difference between the residual values are less than the constant, we then early stop and note the index value.

However, a few of the metrics are already present in the dataset. Each run was conducted 3 times, and then we took the average values of the above metrics.

6.2.2 Results

Below are the following insights drawn from *experiment 2*, consisting of 4 sections. Most of the plots are bar plots, and the last one is a tradeoff scatter plot. In the final section, we discuss our findings.

6.2.2.1 Single Writer

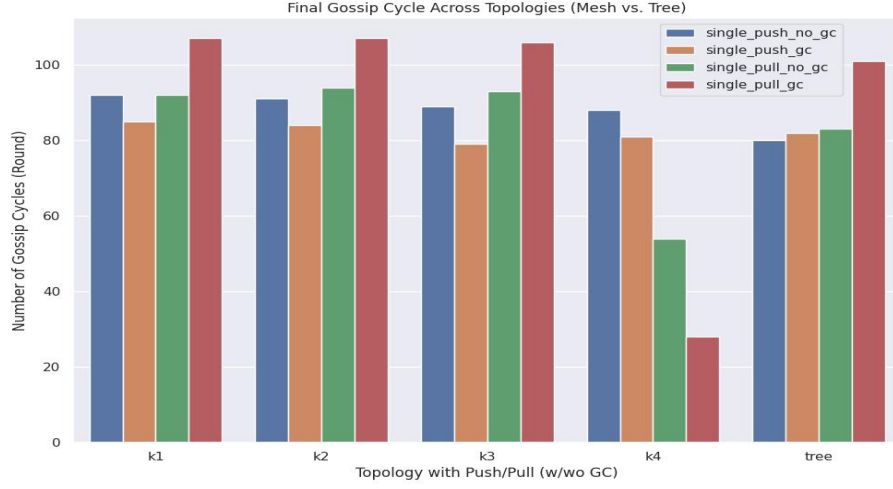


Figure 6.8: Convergence Across Topologies

In figure (6.8), for all ks , we see that push (gossip) with *No-GC* has the same convergence except $k3$ & $k4$, which has slightly performed better. Surprisingly, *tree* performed better than all the ks converging around 80.

For push with *GC*, push (gossip) has performed better than *No-GC*, and out of all the ks , $k3$ has a better convergent rate, which is below 80. For *tree*, it performed slightly better than $k1$ & $k2$ but required little more rounds than $k3$ & $k4$.

For pull with *No-GC*, out of all the ks , $k2$ & $k3$ took the largest number of rounds to converge around ~ 90 . $k1$ required a little less number of rounds compared with $k2$ & $k3$. But $k4$ illustrated even better convergent rate around ~ 55 and has outperformed all the configurations for pull with *No-GC*. *Tree* performed better than $k1$, $k2$, & $k3$ requiring less number of rounds to converge around ($\sim 82-83$).

For pull with *GC*, $k1$, $k2$, & $k3$ required more number of rounds approximately > 105 . Following that, *tree* also required more than 100 rounds to converge. What surprises is that $k4$ is 3.5 times faster at converging all the nodes and requires only 29 rounds.

In figure (6.9), for mesh, push with *No-GC*, the total data transfer for each ks is continuously increasing from ($\sim 10-17$) Mb, whereas the delta ratio is decreasing for each ks . For tree, push with *No-GC*, the total data transfer is less compared to all the ks , which is around ~ 9 Mb, also having a lower delta ratio. $k1$ having the highest delta ratio and $k4$ having the largest data transfer.

For mesh, push with *GC*, we again see the total data transfer for each ks is increasing from ($\sim 9-15$) Mb, whereas the delta ratio is decreasing. Comparing the total data transfer with push with *No-GC*, there is an approximately 2Mb difference, which is not that huge. Also, we don't see a significant amount of difference between the delta ratios. For tree, push with *GC*, again the total data transfer is less compared to all the ks and also having less delta ratio. Again $k1$ has the highest delta ratio, while $k4$ has the largest data transfer.

For mesh, pull with *No-GC*, we again see the same pattern for all the ks where total data transfer is continuously increasing from ($\sim 6-13$) Mb, but for *tree* we see a significant drop around ~ 700 Kb for *No-GC* and ~ 400 Kb for *GC*. We also see a significant drop in the delta ratio compared to mesh *w/wo GC*.

For mesh, pull with *GC*, the size of data transfer increased from $k1$ to $k2$ and then slightly

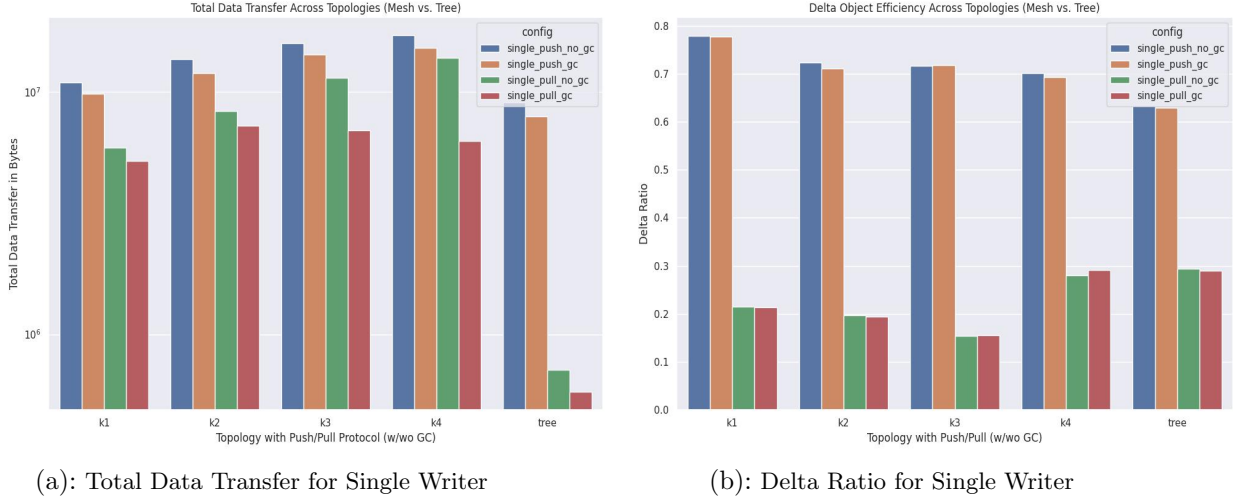


Figure 6.9: Data Transfer and Delta Ratio for Single Writer

dropped for $k3$ and $k4$. For *tree*, the data transfer was less too compared with ks . We observe a similar pattern for the delta ratio when compared with pull with *No-GC*.

6.2.2.2 Three Writer

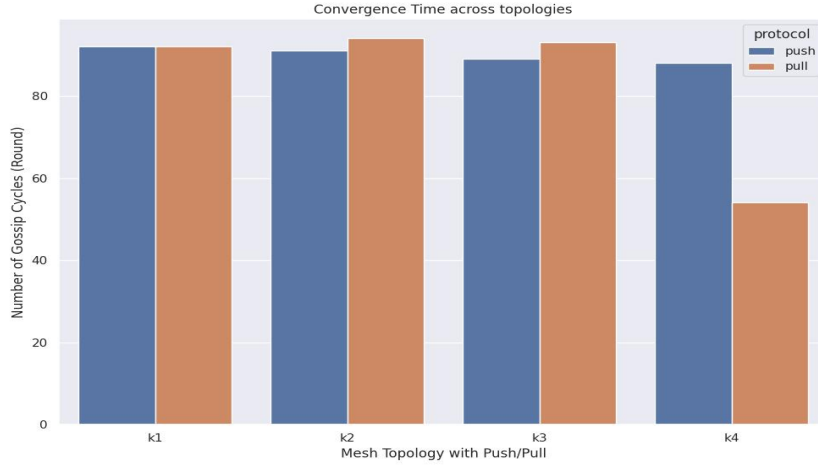


Figure 6.10: Convergence Across Topologies

In figure (6.10), for push, all the ks require approximately the same convergent rate. The difference between the convergent rates for all the ks is 1 to 2.

For pull, $k2$ & $k3$ took little more rounds to converge following that $k1$ took less 2-3 rounds than $k2$ & $k3$. $k4$ outperformed all took 54 rounds to converge.

In figure (6.11), for push, we see the size of data transfer is increasing, which is around (~ 20 - 27) Mb for all the ks . $k1$ having the highest delta ratio following a slight decrease in delta ratio for $k2$, $k3$, & $k4$.

For pull, for $k1$, we observe a significant drop in the data transfer, which is around ~ 9 Mb. For $k2$, $k3$, & $k4$, the data transfer is approximately half of the data transfer of push. The delta ratio for pull is also less compared to the delta ratio of push.

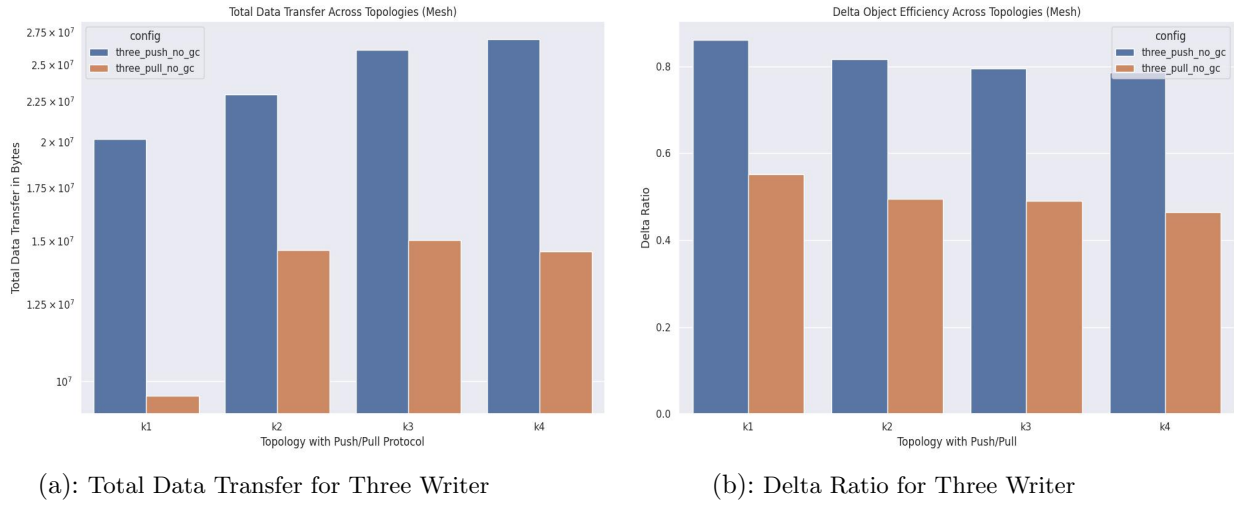


Figure 6.11: Data Transfer and Delta Ratio For Three Writers

6.2.2.3 Performance Tradeoff Scatter Plot

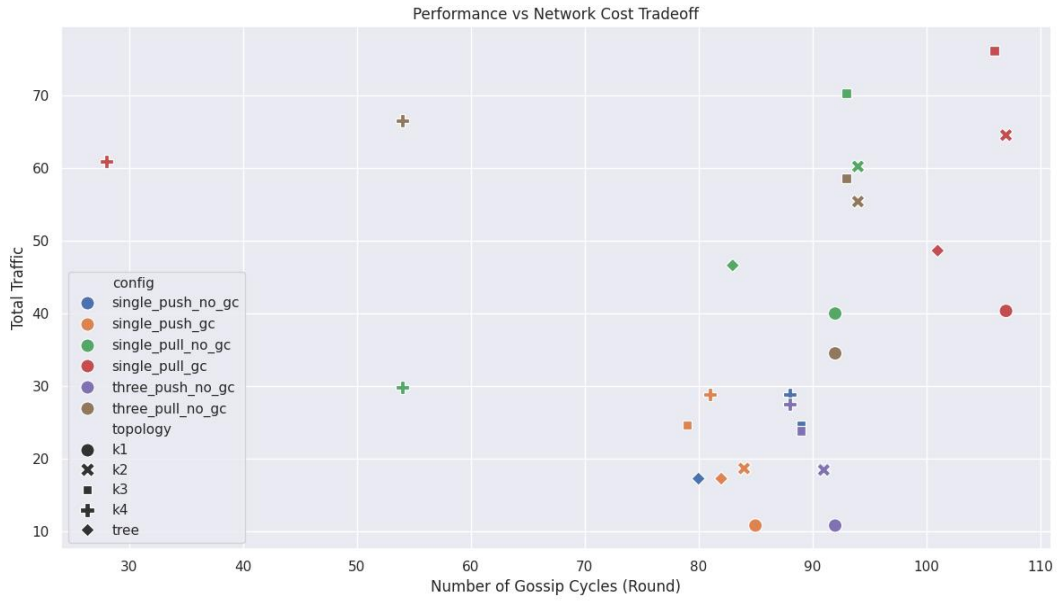


Figure 6.12: Performance Tradeoff Across Topologies

Figure (6.12) shows a direct relationship between convergence rounds and total traffic. We observe that,

1. **Single Writer - Pull with GC** for k_4 converges faster, taking less than 30 gossip rounds, but the cost is too high as the amount of required traffic is high, more than 60. *Pull with No-GC* for k_4 shows an optimal performance with less traffic around 30, and convergence is around ~ 55 . For *push with No-GC* for both *trees* & *ks*, the traffic generated is less between 10-30, but it requires a high number of convergence rounds between 80-95, whereas for *push with GC* (for both *trees* & *ks*) it requires slightly less number of convergence rounds, around 78 to 85 when compared with *push with No-GC*. The amount of traffic generated is the same as in *push with No-GC*.

For *pull w/wo GC* except for k_4 , the amount of traffic is proportional to the number of convergence rounds. For *pull with No-GC*, the traffic is moderately high between 40 and 72, with convergence rounds around 80 to 95, whereas *pull with GC* required a larger number of convergence rounds between 100 and 110. The amount of traffic generated is high, too.

2. **Three Writers** - *Push* generated less traffic same as in *push w/wo GC*, but required a larger number of convergence round around 85-95. *Pull* also required a larger number of convergence rounds with a high amount of traffic between 30-60, except k_4 required fewer than 55 convergence rounds. But the amount of traffic generated for k_4 was too high.

Overall, *single writer* for *pull with GC* for k_3 generated the highest traffic, requiring the highest number of convergence rounds. The *single writer* for *the pull with No-GC* for k_4 gave optimal performance with less traffic generated and exhibited moderately fewer convergence rounds.

6.2.2.4 Discussion

Our experimental results clearly exhibited a behavioral distinction between gossip-based push dissemination and anti-entropy pull synchronization.

In the single-writer scenario, when a peer locally creates updates with the gossip protocol, the newly created git objects are spread quickly. These updates are spread among other peers, relaying them to others hop by hop. What we observed is that the increase in the number of connections (fanout) has shown a reduction in the convergent rounds, but the improvement is small, marginally (92 to 88 rounds). This clearly shows that after a certain point, adding more connections does not significantly speed up synchronization. For such a small network, moderate connectivity is sufficient for rapid dissemination. For our tree topology, the convergent rate is faster at around 80. This suggests that the deterministic propagation is better than the randomized gossip protocol.

In the pull-based (anti-entropy) approach, we clearly see that the number of gossip rounds is higher, which is due to the reconciliation process, except for k_4 . For lower k s, the probability of hitting the most updated node is low, and therefore the spread is low. But for k_4 , the convergence is faster as if the mismatched nodes are almost immediately discovered, and due to this repair happens before the divergence is spread. In our tree topology, the pull-based protocol didn't exhibit any improvements due to uneven connections (not all peers have an equal number of neighbors) across the topology. Reconciliation takes time in a spanning tree-type architecture.

In terms of total data transfer, we observe that Push-based synchronization transferred more total data than pull in our experiment because similar updated data is being transferred to the neighbors. Within a gossip cycle, there are possibilities that the same updated data is being transferred to the neighbors that the local peer is communicating with, which increases the total number of bytes transferred. However, in git, before pushing the objects [50], the local node ask remote about the missing git objects, which are then these missing objects are then compressed into a pack-file and transmitted from your local to the remote node. Sometimes, git push extra data because it cannot confirm what the remote already has. To avoid missing updates, sometimes, git prefers to send more rather than risk sending too little data. This reason also justifies that push-based synchronization has a high delta ratio because more deltas (compressed differences between files) are sent. In pull, total data transfer is less compared to push-based sync. Since many peers share most of the same history, the receiving peer first checks what data it already has, and then it requests the missing data. The delta ratio is significantly lower compared to push-based synchronization because mostly full objects are packed into the pack-file rather than delta-compressed objects. We see a significant drop in total data transfer for *tree* topology as the information flows in a straight, organized path, unlike mesh, where information spreads in many directions, causing extra communication.

GC [49, Performance Optimizations] has slightly improved in terms of convergent rate, and this is due to fewer loose objects. These objects are packed into pack-files, which are indexed and optimized for efficient access. This makes fewer object checks and reduces file system lookups. GC also accelerates commit graph traversal during common ancestor computation in the negotiation phase. This leads to faster synchronization.

In a multi-writer scenario, we tested how concurrent writers achieve eventual consistency when there is a CRDT-like system. We observe a slight increase in the convergent rate. We observed that the concurrent writers were immediate neighbors, i.e., **p1**, **p4**, & **p9**, therefore the convergent rate was slightly increased. If the concurrent writers were not immediate neighbors, then there would be a significant rise in the number of convergence rounds. In terms of total data transfer, there is a rise in both push and pull compared single writer because of multiple writers creating more data objects.

From our tradeoff analysis, the traffic generated by pull-based (anti-entropy) is comparatively higher than push-based (gossip). Supposedly, the traffic generated should be much lower as discussed in [53]. In push-based (gossip), the sender transfers data to other nodes without checking if the receiver is already in a consistent state, whereas in pull-based (anti-entropy), only metadata is exchanged, like hashes (hash of the DB state), rather than sending the whole state. These hashes are compared; if unequal, then the nodes request the entire state. As discussed here (6.2.1), our pull script fetches all available remote references from known peers. The script was intentionally written this way to reduce synchronization delay and achieve faster global consistency. As the nodes learn about more references over time, each node not just effectively synchronizes with immediate neighbors, but with all other peers. Due to this, pull generates more traffic.

In general, our results show that increasing network density (higher fan-out) increases traffic much more than it improves convergence. More connections do not necessarily mean much faster synchronization, but they do increase communication cost. Moderate connectivity is sufficient for small peer-to-peer groups.

Overall, for our chat system, push(gossip) is effective for quickly spreading updates but the data transfer cost is too high, whereas pull(anti-entropy) is communication efficient but takes time for convergence. The choice between the two strategies depends on whether the system prioritizes fast dissemination or controlled network usage.

6.3 End of Chapter

We ran 2 experiments and showed how much time a single device can take to add a member (create git objects), and how our data is being replicated and converged across multiple nodes with different sets/sizes of neighbors.

Note that the experiments were not exclusively running on our hardware. Other applications like browsers, PDF files, bash terminals, network connections (internet), etc., ran simultaneously. Not accurate results are obtained. However, we can consider this experiment a real-life example, like they are running on a smartphone, where multiple tasks or processes are running in the background.

For completeness, additional plots for both experiments 1 and 2 are presented in the Appendix B. These additional figures are to corroborate the trends discussed in this chapter.

6.3.1 Limitations

For *experiment 2*, all the nodes were in a single system, and due to this, we couldn't have better scalability due to the limitation of resources. Better results could've been produced if each node

were a separate machine and located across different parts of a specific zone. With this, we could've achieved infrastructure realism. Each node would be perfectly isolated, having dedicated resources. Our experiment lacked network realism. In a distributed deployment, our nodes would experience network latency, asymmetric bandwidth, congestion, and the probability of packet loss.

We also tried testing pull-push and multiple writers (with GC), and unfortunately, the periodic push/pull script would halt and sometimes couldn't generate *git push* or *git pull* output. There are multiple reasons why this problem has occurred. *First*, git uses lock files like *.git/index.lock*, *.git/packed-refs.lock*, etc.; due to concurrent updates on these files, the lock might not have removed, making it a stale lock. *Second*, GC is CPU-heavy, memory-heavy, and disk I/O heavy. *Third*, all the containers shared the same physical disk, and git operations are disk-heavy during object writing, pack creation, delta compression, etc.

These are a few probable reasons why we couldn't output the results for pull-push and multiple writers (with GC). Also, while testing the *experiment 2*, we had the same problem, but were somewhat able to record the push/pull output.

7

Related Work

In this chapter, we will present a detailed review of the ongoing research and the literature related to this thesis. We will also discuss a few decentralized / P2P group messaging applications that closely resemble our app model. We will solely rely on the *internet* and *crdt.tech* [15] to present the information of our related work.

We would first discuss a few P2P/Decentralized chat applications. A recent offline messaging app that came into the limelight, which was co-created by Jack Dorsey name *BitChat* [4]. *BitChat* is an open source, secure, and private P2P and decentralized messaging app designed for infrastructure-less wireless networks where peers/nodes connect directly and dynamically. No need for central servers. All messages are stored locally at the peer node. Communication takes place directly over transports like Bluetooth Low Energy (BLE) or on a private WiFi network. Once the Bluetooth is on, the app automatically scans for and connects to nearby users. To make communication over the internet, it uses the Nostr protocol, which indeed makes use of Nostr relay nodes to relay messages over the internet. They utilize the Noise Protocol Framework to establish an end-to-end encrypted session between peers, thereby creating a mutual authenticated connection. There is no concept of group chats [5]; either peer sends a private message to another peer or a peer broadcasts to all other peers available in a network. Although when sending a private message to other peers, the message is still broadcast among the peers. The peers who have exchanged the correct Noise session keys with the sender can decrypt the packet.

Briar [6] is another open source, fully P2P messaging app designed for users who need censorship-free or private communication that is safe and easy to use. It doesn't require any central servers to store messages; they are directly synchronized between peers. Communication takes place over the internet using the Tor network [44]. The Tor network provides secure, end-to-end encrypted, and anonymous communication where eavesdroppers are not aware of peers' communication. Communication can also take place via Bluetooth or WiFi when there is no internet access. To add contacts, a user can share a link *Briar://* followed by a hash value, which is the address of the peer over email or other messenger apps. To connect nearby users, one can show their QR code from the app and get connected. *Briar* supports both private and group messaging [7]. The group membership model is simple. There is only one admin in the group, who is the creator of the group. The admin can add or remove members from the group, and members can self-exit from the group. If the admin/creator leaves the group, then the group is dismissed or closed. However, the sources of *Briar* neither provided the information about the group size nor the data structure or algorithm to manage groups.

Session [39], another open source decentralized messaging app designed for human rights ac-

tivists, protesters, or journalists to have safe and private communication with others. Like our chat design, each peer is identified by a public key. To add contacts, each peer needs to share its account ID, which is the public key. *Session* uses two ways to store the data of the chat application; first, the data is stored permanently locally on the peer's device. Second, the data is temporarily stored on a small group of session nodes known as swarms. *Session* nodes are like storage nodes that store the data temporarily. These session nodes also help in message routing. The network these session nodes use is Arbitrum [1], which is an Ethereum Layer 2 blockchain. So when the peer comes online, the job of a swarm is to deliver those undelivered messages or data. *Session* supports both one-to-one and group communication [57]. In a group, there can be multiple group admins in a group. Admins and/or members can exit the group. Members can delete their own messages, and admins have the privilege to delete anyone's message in a group. If there is only one admin inside the group, then the group becomes orphaned. There is a limit in group size which can be upto 100. However, the sources of the session didn't provide the internal architecture to manage groups. But, then mentioned on [57, p. 15] that concurrent removal of members by multiple admins is a complicated process, but never discussed the internal algorithm of the process of removal of a member from the group.

Manyverse [25] is another offline P2P messaging app built using *Secure Scuttlebutt (SSB)* protocol [42]. *SSB* is a decentralized network protocol designed for offline-first social networking. The app data is stored using append-only logs, where each data point is signed cryptographically to ensure its authenticity. This data structure is known as a feed. It represents a user's activity within the *Secure Scuttlebutt (SSB) network*. *SSB* follows the offline-first design and syncs by pulling feeds from its followers (peers) using a gossip-based replication protocol. *Manyverse* does support group messaging [26], where each group is like a feed. Groups are opaque to outside peers as the messages in the group feed are encrypted and are only accessible to the members of the group. However, the existence of the group is still visible to everyone, even the non-members of the group. Non-members of the group can get invited or request to be a part of the group. The group membership model is slightly different. There is no admin concept; everyone inside a group is like an admin [27]. A peer creates a group with a group key/secret, which is a symmetric key and is distributed among other peers who are invited to join the group. The peer who holds the group secret is aware of the group and starts replicating the group feed from the creator of the group. Members can send messages or communicate by appending their messages to the group feed. These messages are signed by the public key of the group key. A member of the group who wants to remove/exclude a member from the group has to create a new group key/secret [28] and share it with only those whom the member wants to be part of the group. However, we didn't find any source about members leaving/exiting the group. One can simply stop posting inside the group feed. Also, there is no source for the group size limit. Messages of the app can't be deleted since a feed is immutable, append-only logs.

Berty [2] is another offline P2P messaging app that functions even without an internet connection. Communication can take place over Bluetooth, WiFi, or the internet. No email id or phone number is required to create an account in the *Berty Messenger app*. *Berty* uses the Wesh protocol [3] originally developed by *Berty Labs*. Like our chat app, each peer has an identity key pair. To store and share the data over the internet, Wesh protocol uses IPFS [24]. *Berty* supports both one-to-one and group communication. All the messages are stored in *Berty's DB*. *Berty* uses OrbitDB, which is a Merkle-CRDT. This database is similar to our git implementation discussed in Chapter 5.

Merkle-CRDT [66] is essentially embedding CRDTs as events within a Merkle-DAG, where Merkle-DAG [66, p. 2] is a graph having direction in the edges with no cycles. Each node is a cryptographic hash of its contents and its parent nodes. Merkle-DAG can act as a logical clock known as Merkle-Clocks (Merkle-Clock-DAG). Each node in Merkle-Clock is known as a Merkle-

Clock Node (each node represents an event). New events create a new root node and point to its previous parent nodes, just like git commits. These events are causally ordered. Concurrent events are merged by creating a new root node and pointing its parent to these concurrent events. This paper [66, p. 6] proved that the behavior of Merkle-Clocks is like a state-based Grow-Only Sets (G-Sets), proven to converge as CRDTs. Merkle-CRDT is a Merkle-Clock where each node carries CRDT payload/data. Each node in Merkle-CRDT is triple (α, P, C) , with α being the node CID, P as an opaque CRDT data object, and C being the set of children identifiers(CID). Merkle-CRDT operations are similar to Merkle-Clocks. Whenever a replica/peer issues a new CRDT payload or modifies/adds data in the CRDT object, a new root event is created pointing to its old / parent events. Merging of Merkle-CRDT DAGs is similar to git fetch/push. When a replica sends its Merkle-DAG (the root node with all its parents) with CRDT payload, the receiver replica will go through all the event nodes of the received DAG, simultaneously fetching those event nodes that are not present.

However, in *Berty*, they did not disclose the underlying CRDT used in their database to sync chats and metadata. In group chat, there is no concept of admins. Other members can invite or add other non-members of the group. The removal is similar to *Manyverse 7*, a new key is generated and distributed among a few members of the group. However, the source didn't provide the information about the group size, deletion, or editing of the messages.

We discussed a few P2P chat applications that support group messaging. Each of them has a different group membership model. Now talking about apps that use CRDT data structures, the list is huge of applications [15, crdt.tech/implementations]. There are a bunch of distributed databases, text editors, and other applications built using CRDT. Popular databases like Riak uses state based CRDTs to build a KV pair [37]. Redis, another popular in-memory database, uses CRDT to synchronize data between geo-replicated servers [36]. The list is too huge, and we won't further discuss all the apps that use CRDTs.

Now we discuss a few applications that are built on top of git using plumbing commands and how they leverage the git objects to store data. The *Delta-GOC-Ledger* [16] is implemented on top of Git. This is an MSc thesis by Jannick at the University of Basel, CH. The goal was to introduce delta types to the existing state-based *GOC-Ledger* [61] to reduce the communication overhead. *GOC-Ledger* is a ledger that implements crypto tokens, designed for a local community that can exchange crypto tokens without the need for blockchain infrastructure. *GOC-Ledger* is a state-based CRDT combining multiple grow-only counters, which makes it consensus-free, unlike blockchains that require consensus among the peers to approve a valid transaction. The *GOC-Ledger* itself is implemented on top of Git using its low-level (plumbing) commands. In *GOC-Ledger*, the entire state object of the CRDT is exchanged among the peers to achieve eventual consistency. Depending on the number of transactions, the size of the state can become large. Therefore, *Delta-GOC-Ledger* was introduced to exchange only small states while guaranteeing correct convergence on unreliable communication channels. In the git implementation, each commit [16, p. 31] represents the delta account operations. Each commit points to the tree object that contains the actual data of the delta state. The tree object stores the delta mutator values [16, p. 15]. Git reference is used to point to the latest commit to track the current ledger state.

The *2P-BFT-Log* [60] is also implemented on top of Git using low-level (plumbing) commands. The idea of this paper is to detect and repair the forks created by a Byzantine node from an append-only log and still preserve the eventual consistency when forks are pushed/fetched from the malicious author.

A P2P Catan boardgame [33] MSc thesis by Tim at the University of Basel, CH, was built on top of Git. Catan [8] is a multi-player strategic boardgame developed by Klaus Teuber. The board

consists of hexes of the island of Catan, and roads and settlements constructed on it. Players can build roads, settlements, and cities on the island as they settle on the island. This game is usually played with a dice. The number of rolls activates certain hexes on the board. Players can trade or negotiate with other players for making deals, any counteroffers, or temporary alliances with each other. The game was built using Python using the GitPython library, which means the author of this thesis decided to work use git high-level (porcelain) commands. Each git branch represents a player of Catan. The state of players is organized into folders and files, which we can assume as a Git tree and blobs. Whenever the player state (files) changes, a new git commit is created to the player's branch. These branches are exchanged among the peers for game synchronization to obtain the current status of the boardgame.

8

Conclusion

In this thesis, we have implemented an offline P2P group messaging app for a small community group residing in a location that lacks infrastructure or resources. Our thesis addressed the challenge of how it operates without central servers (for coordination) that require a dedicated infrastructure running 24/7, and persistent internet connectivity. Our core part of the thesis was to focus on the group membership model, designing group operations APIs (3) like adding or removing a member, how to promote/demote an admin, how group descriptions are stored, and how the history of messages is preserved. With no central servers, in P2P architecture, it is difficult to coordinate or replicate group state data objects among the peers, especially when concurrent modification takes place among the peers. To tackle this problem, a non-consensus approach was proposed by using CRDTs to manage the state of the groups. The app was implemented on top of *git* using the use of *git* objects to store group information; moreover, commit objects were used to preserve the causality of group messages. Our idea of implementation is similar to *Berty Messenger* 7, which uses Merkle-CRDTs [66] as an underlying database. We did not build our own Merkle-DAG from scratch; instead, we used Git. Comparing our app's group membership model with popular group messaging apps discussed in (2.1), we implemented most of the APIs, but a few ones, like deleting and/or editing of messages was not possible due to *git*'s inherent limitations. This could be a part of a future extension. Also, comparing our group messaging apps with other P2P and decentralized applications which were discussed in *chapter 7* (7), none of them used CRDTs as an underlying data structure except for *Berty*, but *Berty* never disclosed the type of CRDTs they are using.

We evaluated our chat system in 2 experiments. In the *first* experiment, we tested how the insertion cost grows and how the storage footprint increases when there are large number of *git* objects. We added 10000 members in a group that occupies ~ 240 Mb of storage, which is huge, even though our group state only contains a few metadata and history of messages, let alone photos and videos. 10 groups with 10,000 members each would occupy 2.4 Gb of disk. If GC (Garbage Collection) is enabled and is run periodically, this could significantly improve storage, taking less space around ~ 4 -9 Mb. In the *second* experiment, we tested how we replicate the groups of our chat model across a set of peers using a replication strategy like push (gossip) vs. pull (anti-entropy). We observe that push generates more total data transfer, while pull generates less total data transfer across the network but requires more time to converge. Due to H/W limitation we were only able to scale 255 members. We already discussed the limitations of our evaluation in this section 6.3.1.

Overall, our work contributes to the understanding of building a P2P system and replicating DB objects across peers without coordination. For this, we used CRDTs to replicate data objects among peers when there are concurrent modifications among the peers. We learned how to design

a P2P system using CRDTs and proved the correctness of our design. However, challenges remain, which are discussed in the next section.

8.1 Future Work

In this section, we will discuss a few possible extension that can be implemented in the future.

1. **Unique Members** - We overlooked the case where there can be multiple members with the same name. There can be 2, 3, or N members with the name *alice*. In the implementation chapter in section (5.2.3), we have stated that each member is identified by their public key. We can take the hash value of the public-key and concatenate the member name with the hash value, which will ensure uniqueness in the member names. We don't think this should be a part of the future extension, but a simple fix is needed in the implementation part.
2. **Byzantine nodes** - In Git, one can internally inspect the contents, type, or size of the Git objects. For example, in our P2P group messaging app, a normal group member can view the blob values from the tree objects 5.3.2. These blob values are simply integers and are transparent to others. Assuming a non-admin group member is Byzantine, it can internally create a new commit object by changing the blob values, making itself an admin, then creating a new tree object which is associated with a new commit object. Now this Byzantine node has full control over the group. This Byzantine node can now push the newly created commit message to other peers associated with the group, eventually making them believe that it's now an admin of the group. Therefore, we either need a way to secure our CRDTs or detect forks from byzantine nodes. To secure CRDTs, we employ homomorphic encryption. *Homomorphic encryption* [64] is a technique where one can perform certain mathematical operations [69] directly on the encrypted data with the help of a server key, which is often public. Since we are only storing positive integer values in our blobs, we can simply store the serialized, encrypted values in our blob objects and share the server key with all members and admins of the group. Note that for each group, we need to generate the key pair and then only share the public key (server key) with the members and admins of the group. The reason to share the public key (server key) with other members/admins is because of the merging algorithm (math operations) that is performed when a push/pull is called. We can share the secret key with other admins to view the actual data by decrypting it, but it is sufficient to share only the public key (server key) since it is only used for encrypting the data as well as to perform computations. We also need to abstract the homomorphic encryption mathematical algorithm (like add, sub, mul, div, mod) inside the group operation APIs. So whenever these APIs are called, they should be able to perform math operations over the encrypted blob values. Although the content of the blobs is stored in an encrypted, serialized way, the attacker would likely be able to find a pattern and distinguish between the blob values, and could create a new commit. To detect forks, we should make our CRDTs *Byzantine Fault Tolerant* [60]. In Appendix (C), we have presented a vague idea about how we can secure it.
3. **Pruning** - Due to decentralization, all the messages are stored in peers. And there is high possibility that the storage size would increase. Therefore, we can add an extension to this project to prune old messages of our chat application.
4. **Storing Group Chats** - If, in the future, our app is extensible to have communication between multiple users across the world, then it would not be feasible to push/fetch the messages that have not been fetched or received for a long period of time. For that, one can store git commits on an IPFS node [56] somewhere between the user's location. However, there

is a limitation in Git where one cannot fetch half-commits from a branch. If one wants to push messages to a newly added member, then sending the entire payload would be inconvenient, might expect data loss due to network failure. And sending fewer of the latest commits(based on timestamps) from the group chat is not possible.

5. **UI** - We have only built backend APIs for our chat application. There is no support for a UI interface. It could be a bachelor's project to give front-end support to these group operations APIs.

Bibliography

- [1] Arbitrum. <https://arbitrum.io/>. Accessed: 2025-12-24.
- [2] Berty messenger. <https://berty.tech/messenger>, . Accessed: 2025-12-26.
- [3] Wesh protocol by berty. <https://berty.tech/docs/protocol>, . Accessed: 2025-12-26.
- [4] Bitchat. <https://github.com/permissionlesstech/bitchat>, . Accessed: 2025-12-23.
- [5] Bitchat whitepaper. <https://github.com/permissionlesstech/bitchat/blob/main/WHITEPAPER.md>, . Accessed: 2025-12-23.
- [6] Briar. <https://briarproject.org/>, . Accessed: 2025-12-23.
- [7] Briar manual. <https://briarproject.org/manual/>, . Accessed: 2025-12-23.
- [8] Catan. <https://www.catan.com/>. Accessed: 2025-12-29.
- [9] Challenges of building a reliable realtime chat service. <https://www.infoq.com/articles/challenges-realtime-chat-service-pusher/>, . Accessed: 2024-02-25.
- [10] Chat app system design: Messaging architecture. <https://trueconf.com/blog/reviews-comparisons/chat-app-system-design>, . Accessed: 2025-11-22.
- [11] Design a chat app. <https://bytebytego.com/courses/system-design-interview/design-a-chat-system>, . Accessed: 2024-02-25.
- [12] Design a chat application. https://algomaster.io/learn/system-design-interviews/design-whatsapp?utm_source=chatgpt.com, . Accessed: 2025-11-22.
- [13] Whatsapp ui in react native. <https://dev.to/jacksbridger/whatsapp-ui-in-react-native-part-1-4nbm>, .
- [14] What is ci/cd? <https://about.gitlab.com/topics/ci-cd/>.
- [15] crdt.tech. <https://crdt.tech/resources>. Accessed: 2024-05-29.
- [16] Delta-goc-ledger: Incremental checkpointing and lower message sizes for grow-only counters ledgers with delta-crdts by jannick heisch. https://cn.dmi.unibas.ch/fileadmin/user_upload/redesign-cn-dmi/pubs/theses/master/Heisch-Delta-GOC-Ledger.pdf. Accessed: 2025-12-28.
- [17] ed25519 algorithm. <https://en.wikipedia.org/wiki/EdDSA>.
- [18] Geopolitical risk dashboard. <https://www.blackrock.com/corporate/insights/blackrock-investment-institute/interactive-charts/geopolitical-risk-dashboard>. Accessed: 2025-12-07.

- [19] Git. <https://git-scm.com/book/en/v2>, .
- [20] libgit2 bindings for rust. <https://github.com/rust-lang/git2-rs>, .
- [21] List of distributed chat applications. <https://www.practicalecommerce.com/group-messaging-mobile-apps>, .
- [22] Groupme.
Info: <https://groupme.com/>, <https://en.wikipedia.org/wiki/GroupMe>;
FAQ: <https://support.microsoft.com/en-us/office/groups-messaging-a3ad1855-ab4d-4658-a86c-4b276aed43b7>,
<https://groupme.com/blog/groupme-just-made-messaging-smarter-with-the-ability-to-edit-messages-after-sending-lea>.
. Accessed on 2025-12-02.
- [23] imessage.
Info: <https://en.wikipedia.org/wiki/IMessage>;
FAQ: <https://support.apple.com/en-in/messages>. Accessed on 2025-12-02.
- [24] Ipfs. <https://ipfs.tech/>. Accessed: 2025-12-26.
- [25] Manyverse. <https://www.manyver.se/>, . Accessed: 2025-12-25.
- [26] Manyverse blogs - private groups. <https://www.manyver.se/blog/2022-11-25>, . Accessed: 2025-12-25.
- [27] Manyverse blogs - private groups user experience. <https://www.manyver.se/blog/2023-05-19>, . Accessed: 2025-12-25.
- [28] Manyverse blogs - private groups member exclusion. <https://github.com/ssbc/ssb-group-exclusion-spec>, . Accessed: 2025-12-25.
- [29] Add descriptions to whatsapp group chats to coordinate discussions better. <https://smartphones.gadgethacks.com/how-to/add-descriptions-whatsapp-group-chats-coordinate-discussions-better-0183581/>.
- [30] Microsoft teams.
Info: <https://www.microsoft.com/en-in/microsoft-teams/group-chat-software>, https://en.wikipedia.org/wiki/Microsoft_Teams;
FAQ: <https://support.microsoft.com/en-us/teams>, <https://learn.microsoft.com/en-us/microsoftteams/limits-specifications-teams>. Accessed on 2025-12-02.
- [31] Do unix timestamps change across timezones? <https://stackoverflow.com/questions/23062515/do-unix-timestamps-change-across-timezones>, . Accessed: 2025-11-04.
- [32] Is epoch time the same on the same machine with different scripts/programs? <https://stackoverflow.com/questions/13232389/is-epoch-time-the-same-on-the-same-machine-with-different-scripts-programs>, . Accessed: 2025-11-04.
- [33] Modelling and implementing the "catan" boardgame as a replicated state machine for peer-to-peer systems by tim matter. https://cn.dmi.unibas.ch/fileadmin/user_upload/redesign-cn-dmi/pubs/theses/master/Matter-Catan-P2P-TLAplus-Git.pdf, . Accessed: 2025-12-29.

- [34] P2p group messaging app based on crdt. <https://github.com/abhilashmendhe/P2P-Group-Chat-App>, . Accessed: 2026-01-03.
- [35] Quorum consensus. <https://web.mit.edu/6.033/2005/wwwdocs/quorum`note.html>. Accessed: 2025-11-22.
- [36] Active-active geo-distribution (crdts-based). <https://redis.io/active-active>. Accessed: 2025-12-26.
- [37] Developing with riak kv. <https://docs.riak.com/riak/kv/latest/developing/data-types/index.html>. Accessed: 2025-12-26.
- [38] Rust. <https://www.rust-lang.org/>.
- [39] Session. <https://getsession.org/>. Accessed: 2025-12-24.
- [40] Signal.
Info: <https://signal.org/>, <https://en.wikipedia.org/wiki/Signal`software>;
FAQ: <https://support.signal.org/hc/en-us/articles/360007319331-Group-chats>, <https://support.signal.org/hc/en-us/articles/360050427692-Manage-a-group>. Accessed on 2025-12-02.
- [41] Threatened small island states. <https://archive.ipcc.ch/ipccreports/tar/wg2/index.php?idp=671>. Accessed: 2025-11-22.
- [42] Scuttlebutt (ssb). <https://scuttlebutt.nz/>. Accessed: 2025-12-25.
- [43] Telegram.
Info: <https://telegram.org/>, <https://en.wikipedia.org/wiki/Telegram`software>;
Groups Info: <https://telegram.org/tour/groups>;
FAQ: <https://telegram.org/faq>, <https://telegram.org/faq`channels#:~:text=Q:%20What%20can%20administrators%20do,as%20delete%20the%20channel%20completely>. Accessed on 2025-12-02.
- [44] Tor. <https://www.torproject.org/>. Accessed: 2025-12-23.
- [45] Facebook–cambridge analytica data scandal. <https://en.wikipedia.org/wiki/Facebook%E2%80%93Cambridge`Analytica`data`scandal>. Accessed: 2024-09-01.
- [46] Unix time. <https://en.wikipedia.org/wiki/Unix`time>.
- [47] Wechat.
Info: <https://www.wechat.com/>, <https://en.wikipedia.org/wiki/WeChat>;
FAQ: <https://help.wechat.com/oshelpcenter>. Accessed on 2025-12-02.
- [48] Whatsapp messenger.
Info: <https://www.whatsapp.com/>, <https://en.wikipedia.org/wiki/WhatsApp>;
Groups Info: <https://www.whatsapp.com/groups>;
FAQ: <https://faq.whatsapp.com/>, <https://faq.whatsapp.com/441425088132574/?cms`platform=web>. Accessed on 2025-12-02.
- [49] Git 2.11 has been released, performance optimizations. <https://github.blog/open-source/git/git-2-11-has-been-released/#performance-optimizations/>, 04-01-2019.

- [50] Git’s database internals iv: distributed synchronization. <https://github.blog/open-source/git/gits-database-internals-iv-distributed-synchronization/>, 21-09-2022.
- [51] Hesam Nejati Sharif Aldin, Hossein Deldari, Mohammad Hossein Moattar, and Mostafa Razavi Ghods. Consistency models in distributed systems: A survey on definitions, disciplines, challenges and applications, 2019. URL <https://arxiv.org/abs/1902.03305>.
- [52] Annette Bieniusa, Marek Zawirski, Nuno Preguiça, Marc Shapiro, Carlos Baquero, Valter Balegas, and Sérgio Duarte. An optimized conflict-free replicated set, 2012. URL <https://arxiv.org/abs/1210.3368>.
- [53] Alan Demers, Dan Greene, Carl Hauser, Wes Irish, John Larson, Scott Shenker, Howard Sturgis, Dan Swinehart, and Doug Terry. Epidemic algorithms for replicated database maintenance. In Proceedings of the Sixth Annual ACM Symposium on Principles of Distributed Computing, PODC ’87, page 1–12, New York, NY, USA, 1987. Association for Computing Machinery. ISBN 089791239X. doi: 10.1145/41840.41841. URL <https://doi.org/10.1145/41840.41841>.
- [54] Vitor Enes, Paulo Sérgio Almeida, Carlos Baquero, and João Leitão. Efficient synchronization of state-based crdts, 2019. URL <https://arxiv.org/abs/1803.02750>.
- [55] Heidi Howard and Richard Mortier. Paxos vs raft: have we reached consensus on distributed consensus? In Proceedings of the 7th Workshop on Principles and Practice of Consistency for Distributed Data, EuroSys ’20. ACM, April 2020. doi: 10.1145/3380787.3393681. URL <http://dx.doi.org/10.1145/3380787.3393681>.
- [56] Interplanetary Filesystem (IPFS) Git Remote Helper. <https://github.com/dhappy/git-remote-ipfs>, 2019. Accessed: 2026-01-05.
- [57] Kee Jefferys, Maxim Shishmarev, and Simon Harman. Session: End-to-end encrypted conversations with minimal metadata leakage, 2024. URL <https://arxiv.org/abs/2002.04609>.
- [58] B. Peabody K., Davis and Leach P. Universally unique identifier. <https://datatracker.ietf.org/doc/html/rfc9562>.
- [59] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system, page 179–196. Association for Computing Machinery, New York, NY, USA, 2019. ISBN 9781450372701. URL <https://doi.org/10.1145/3335772.3335934>.
- [60] Erick Lavoie. 2p-bft-log: 2-phase single-author append-only log for adversarial environments, 2023. URL <https://arxiv.org/abs/2307.08381>.
- [61] Erick Lavoie. Goc-ledger: State-based conflict-free replicated ledger from grow-only counters, 2023. URL <https://arxiv.org/abs/2305.16976>.
- [62] Erick Lavoie. State-based ∞ p-set conflict-free replicated data type, 2023. URL <https://arxiv.org/abs/2304.01929>.
- [63] Carlos Baquero Marek Zawirski Marc Shapiro, Nuno Preguiça. A comprehensive study of convergent and commutative replicated data types. Inria – Centre Paris-Rocquencourt; INRIA, 2011.

- [64] Monique Ogburn, Claude Turner, and Pushkar Dahal. Homomorphic encryption. Procedia Computer Science, 20:502–509, 2013. ISSN 1877-0509. doi: <https://doi.org/10.1016/j.procs.2013.09.310>. URL <https://www.sciencedirect.com/science/article/pii/S1877050913011101>. Complex Adaptive Systems.
- [65] Fernando Pedone, Matthias Wiesmann, André Schiper, Bettina Kemme, and Gustavo Alonso. Understanding replication in databases and distributed systems. pages 464–474, 01 2000. doi: 10.1109/ICDCS.2000.840959.
- [66] Hector Sanjuan, Samuli Poyhtari, Pedro Teixeira, and Ioannis Psaras. Merkle-crdts: Merkle-dags meet crdts. CoRR, abs/2004.00107, 2020. URL <https://arxiv.org/abs/2004.00107>.
- [67] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In Xavier Défago, Franck Petit, and Vincent Villain, editors, Stabilization, Safety, and Security of Distributed Systems, pages 386–400, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg. ISBN 978-3-642-24550-3.
- [68] Weihai Yu and Sigbjørn Rostad. A low-cost set crdt based on causal lengths. In Proceedings of the 7th Workshop on Principles and Practice of Consistency for Distributed Data, PaPoC '20, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450375245. doi: 10.1145/3380787.3393678. URL <https://doi.org/10.1145/3380787.3393678>.
- [69] ZAMA FHE Computation - Arithmetic operations. <https://docs.zama.org/tfhe-rs/fhe-computation/operations/arithmetic-operations>, 2022. Accessed: 2026-01-04.



– Concurrency Diagrams

Based on our design 3, we would like to present a few concurrent diagrams about our group chat application.

A.1 Add Same Member

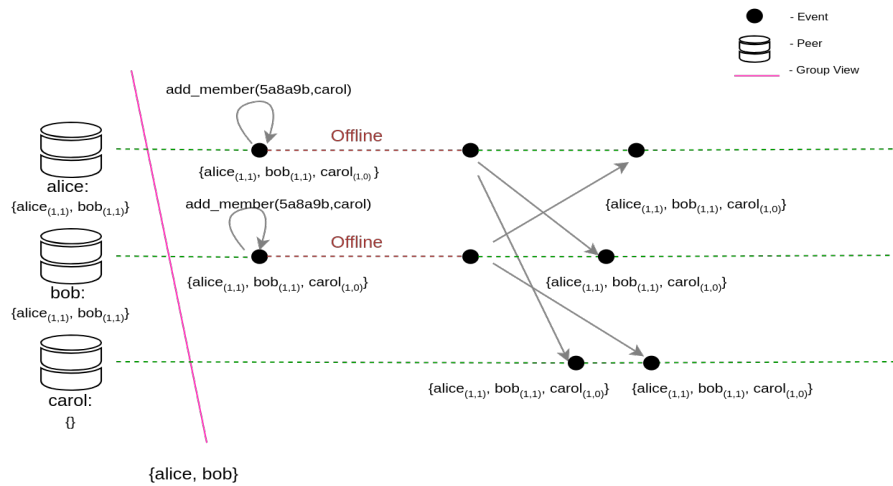


Figure A.1: Adding Same Member

In fig A.1, there is a group with id *5a8a9b* that has two admins *alice* and *bob*. Both are concurrently adding *carol* to the group and pushing their updates to everyone inside the group. By algo. 7, *alice* and *bob* have no effect of the push update since their states were already equal. Carol gets the full state of the group.

A.2 Add Two Different Members

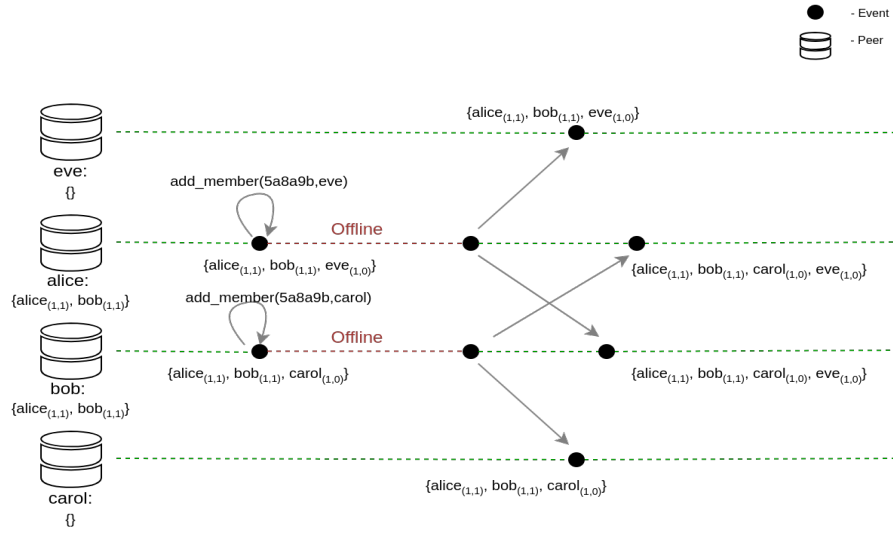


Figure A.2: Adding Two Different Members

In fig A.1, *alice* and *bob* are the admins of the group with id *5a8a9b*. *alice* adds *eve* where as *bob* adds *carol* concurrently and their states are updated. Both of them push their updated states to everyone. *eve* and *carol* learns about *alice* and *bob* are already present inside the group. By algo. 7, *alice* and *bob* get the fully updated state and learn about new members that were added concurrently. To achieve Strong Eventual Consistency (SEC) among the members, one of the admins needs to push their latest states to everyone.

A.3 Remove Same Member

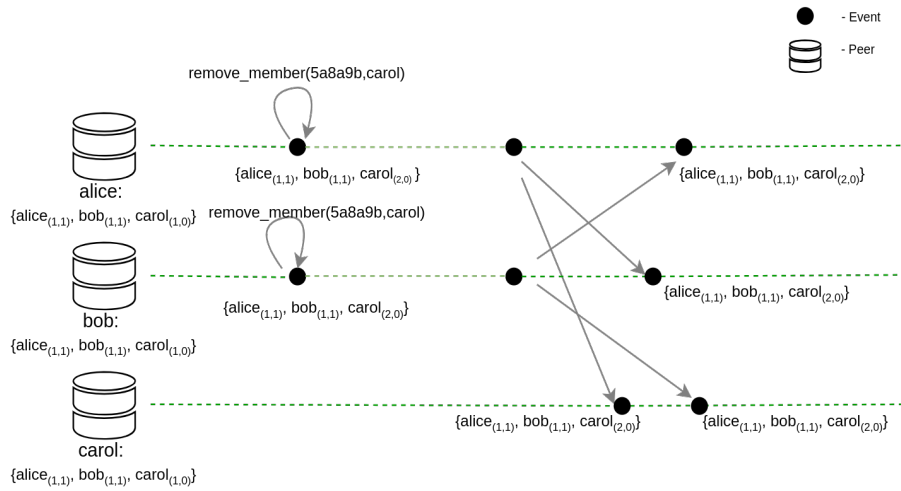


Figure A.3: Removing Same Member

In fig A.4, both *alice* and *bob* are concurrently removing *carol* from the group and pushing their updates accordingly. However, both admins are removing the same member from the group, there is no effect on the state of the group.

A.4 Remove Two Different Members

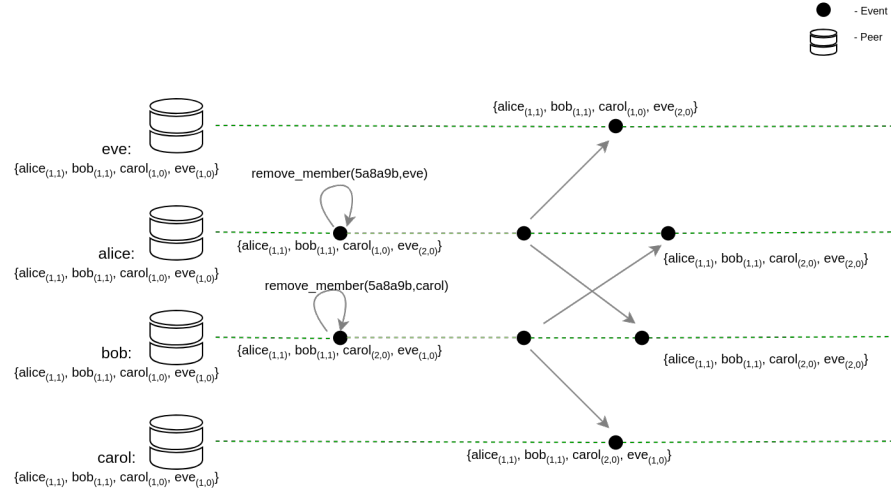


Figure A.4: Removing Two Different Members

In fig. A.4, both *alice* and *bob* are concurrently removing two different members and pushing their updates accordingly. *eve* learns about her removal from the group, whereas *carol* learns about her removal from the group. *alice* and *bob* get the fully updated states and learn about the removal of both *eve* and *carol*.

A.5 Remove Member + Member Exit

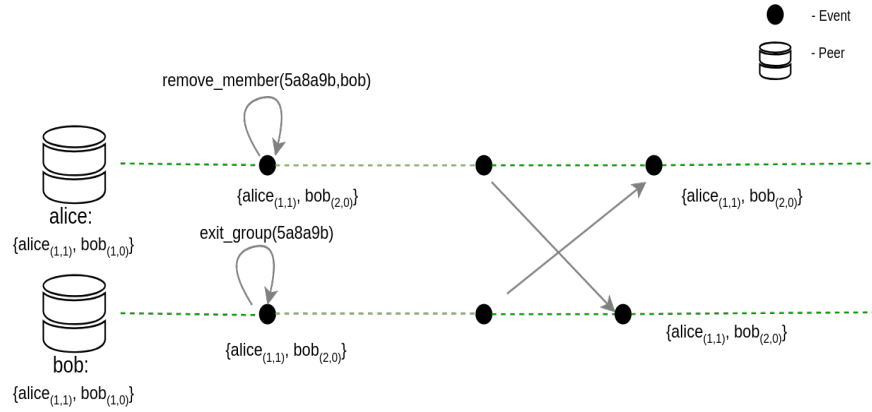


Figure A.5: Removing Member and Same Member Exits

In fig. A.5, the scenario depicts the removal and exiting of members concurrently. *alice* removes *bob* and pushes her updates, while *bob* exits the group and pushes his updates. By algo. 7, both have the same group state, having no effect on their group states.

A.6 Promote Same Member

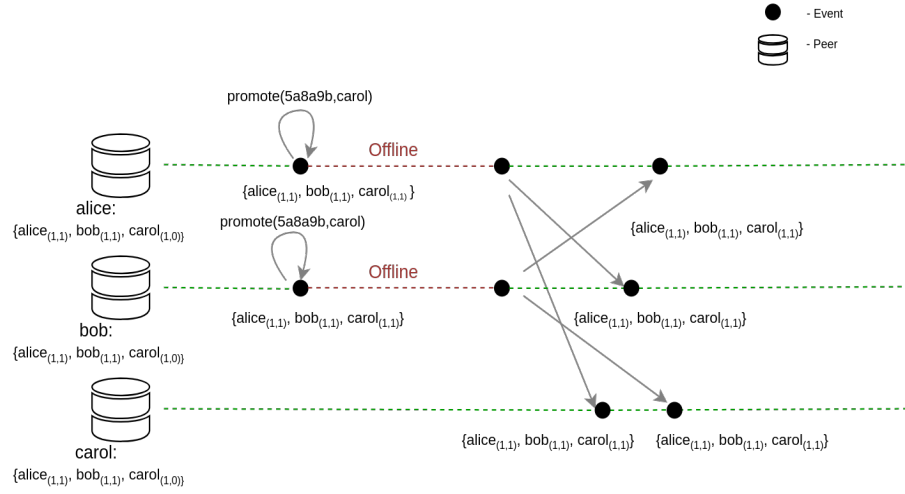


Figure A.6: Promote Same Member

In fig. A.6, *alice* and *bob* are concurrently promoting *carol* as an admin of this group and pushing to everyone in the group. Their respective current state changes. After receiving the states from other group members, there is no effect on the final group states since they are both promoting to the same member.

A.7 Promote Two Different Members

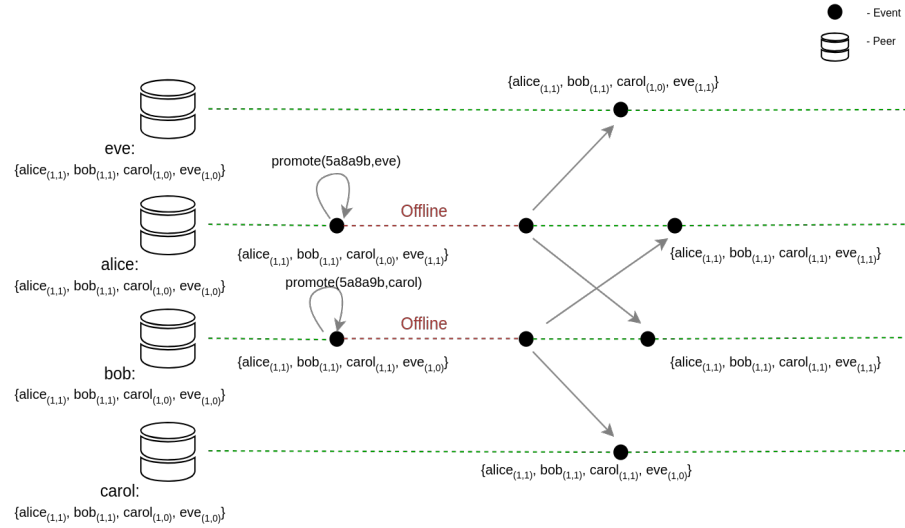


Figure A.7: Promote Two Different Members

In fig. A.7, *alice* and *bob* are concurrently promoting two different group members and pushing their respective states to all other members. *eve* notifies that she is also an admin of the group from *alice*, and *carol* notifies that she is also an admin of the group from *bob*. The fully final states are received at *alice* and *bob*'s side. Both learn that *eve* and *carol* are now the admins of the group. *eve* and *carol* can get the final state by getting push from either *alice* or *bob*.

A.8 Demote Same Member

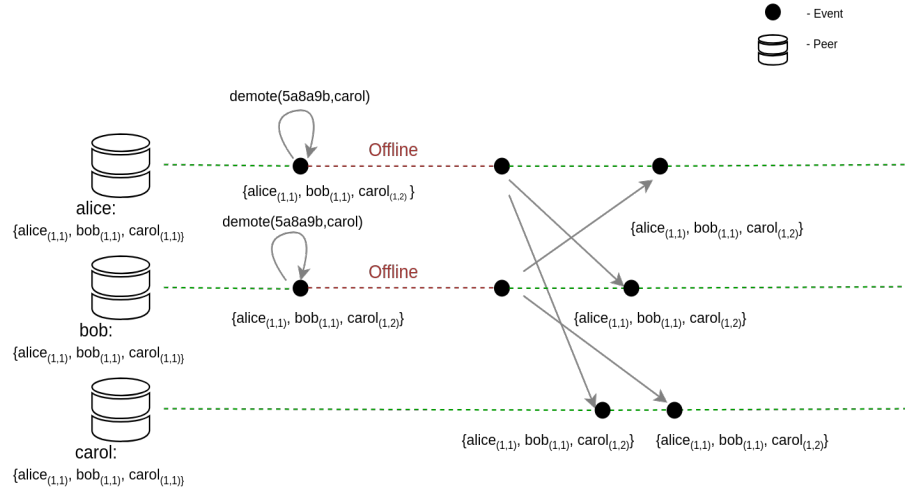


Figure A.8: Demote Same Member

In fig. A.8, *alice* and *bob* are concurrently demoting *carol* from the group and updating their local state. After pushing the states from *alice* and *bob*, everyone gets the updated state and merges with their local state. *carol* state is changed while there is no effect on *alice* and *bob*'s state.

A.9 Demote Two Different Members

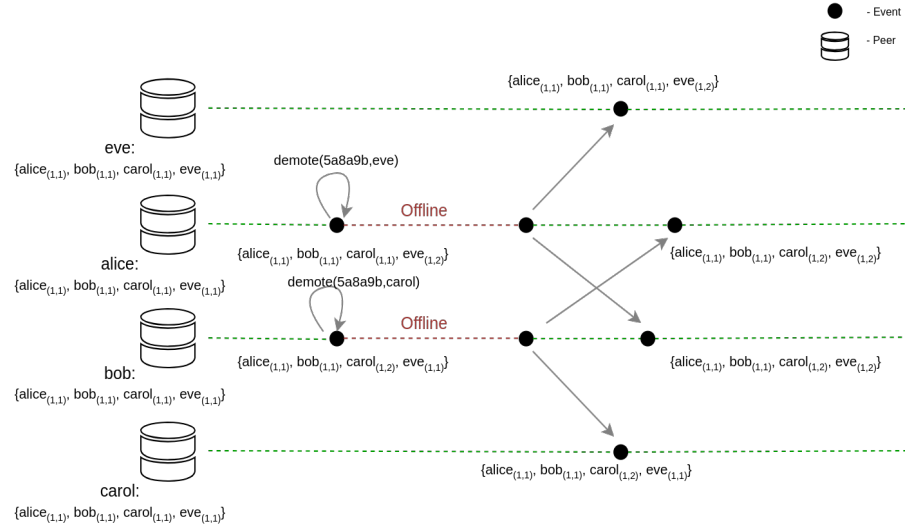


Figure A.9: Demote Two Different Members

In fig. A.9, *alice* and *bob* are concurrently demoting two different members (*eve* and *carol*) and updating their local state. After pushing the states from *alice* and *bob*, everyone gets the updated state and merges with their local state. *eve* and *carol* are being notified that they are no longer the admin of the group. *alice* and *bob* gets the full updated state and are notified about *eve* and *carol* are no longer the admin of the group.

A.10 Demote Each Other + Members

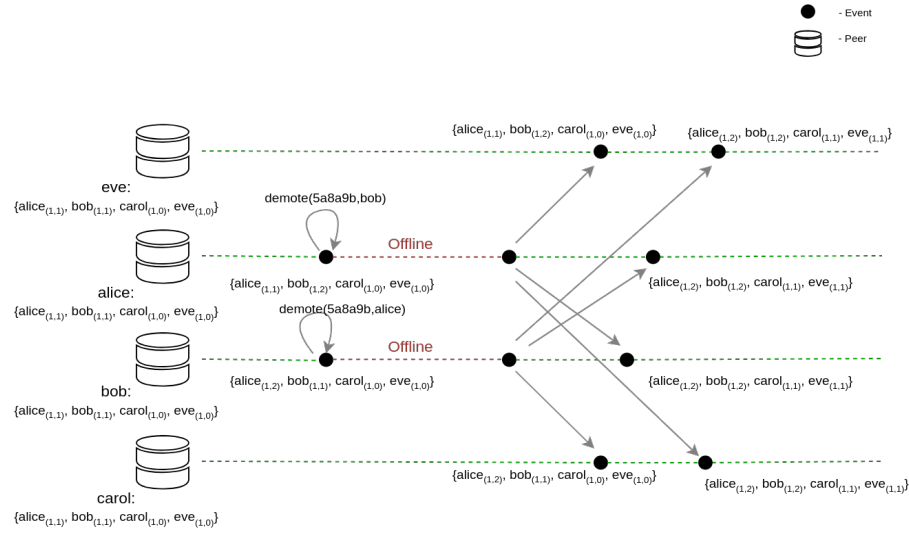


Figure A.10: Demote Each Other

In fig. A.10, both admins *alice* and *bob* are concurrently demoting each other and updating their local state. Both *alice* and *bob* push their updated local state to everyone. Everyone learns that *alice* and *bob* are no longer the admins of the group. In this case, everyone becomes an admin since there are no other admins left inside the group.

A.11 Demote Each Other + No Members

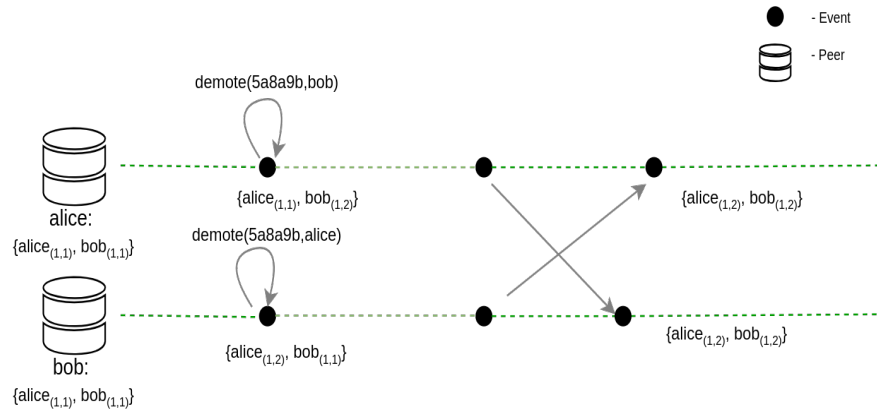


Figure A.11: Demote Each Other

In fig. A.11, there is a similar case from the fig. A.10 but with no additional members. Both admins are notified that they are no longer the admins of the group. The group now has only 2 normal members with no admins. The group is semi-functional, where the members can only send messages but can't add, remove, promote, or demote any other members.

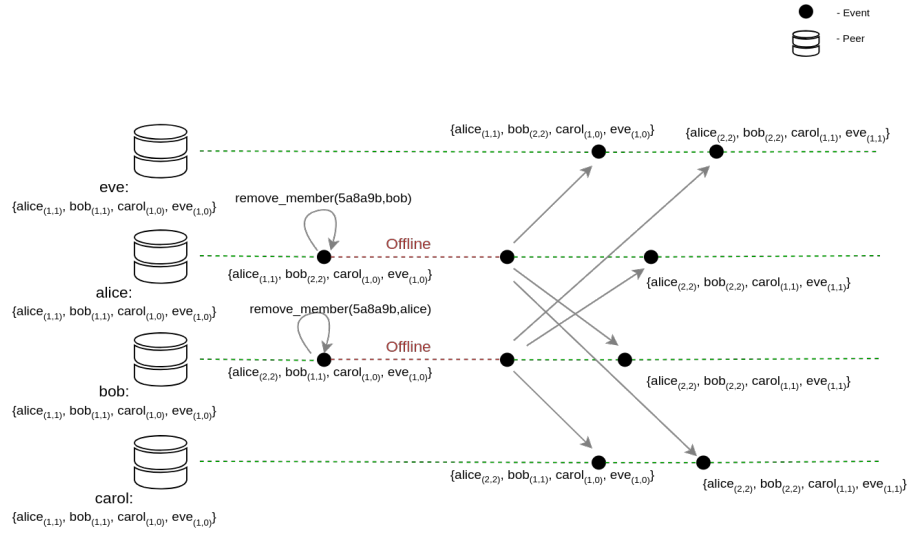


Figure A.12: Remove Admins

A.12 Remove Admins + Members

In fig.A.12, there is a similar case from fig. A.10, but the only difference is that the admins are removed from the group. This case is analogous to admins exiting the group concurrently.

A.13 Remove Admins + No Members

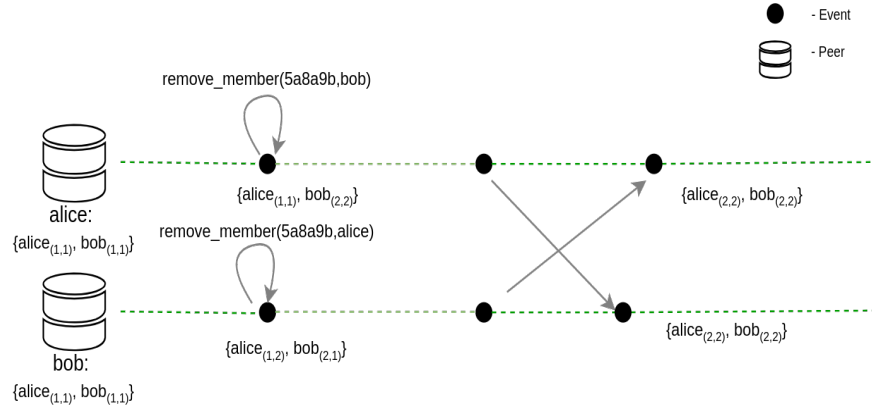


Figure A.13: Remove Admins

In fig. A.13, there is a similar case from fig. A.11 but there are few differences. When admins are removed from the group, the group is non-functional. No one can enter the group again, and sending messages is not possible either.

B

– More Evaluation Plots

In this chapter, we provide additional plots to get clearer insights into our experiment.

B.1 Experiment 1

B.1.1 Insert Latency vs Number of Inserts

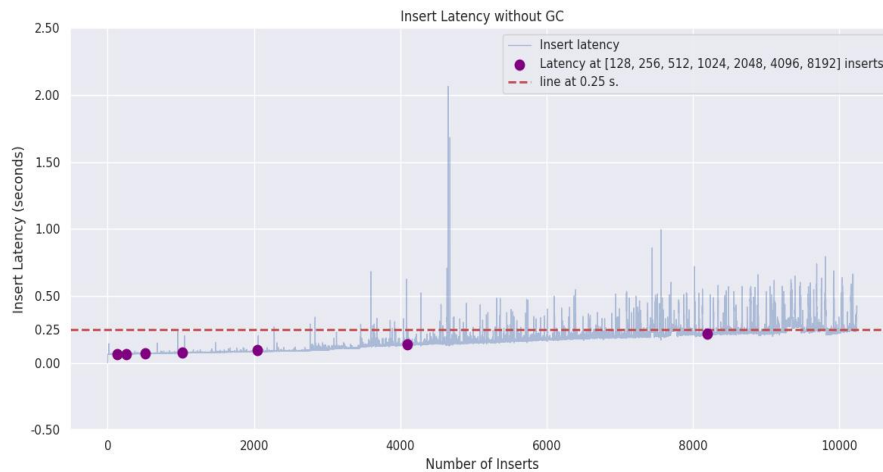


Figure B.1: Insert latency vs. Number of Inserts (No GC)

Figure (B.1) depicts a line plot of consecutively adding members inside a group. Adding a new member creates new git objects, which results in the increase of insert cost. The band (variance) started rising after a 0.125s latency, approximately after adding 3000 members to the group. After adding 4000 members, the variance explodes. We see occasional spikes up to 1–2s latency, and this could happen due to git packing files while pushing git objects or due to OS noise.

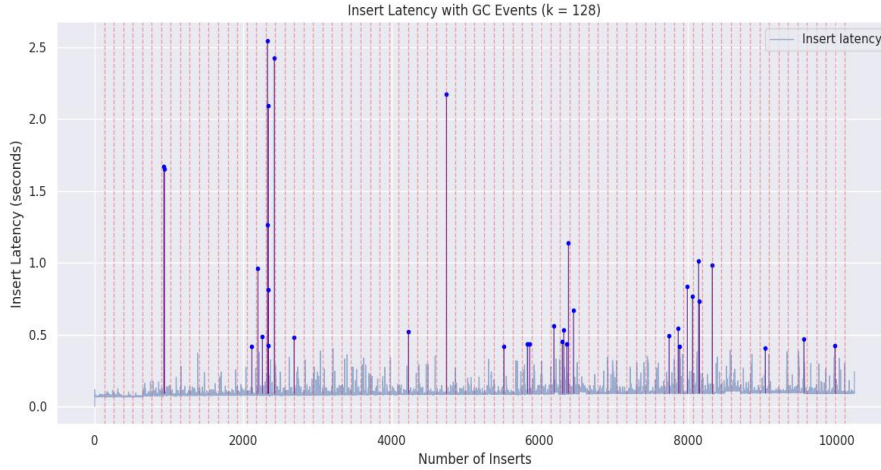


Figure B.2: Insert latency vs. Number of Inserts (GC - 128)

Figure (B.2) depicts a line plot of consecutively adding members inside a group with periodic GC after every addition of 128 members. We observe 6-7 lines that have a minimum of 1s latency to a maximum of 2.4s latency, and this is due to frequent GC events. The variance is between 0.25s to .4s latency. However, we see a stable baseline.

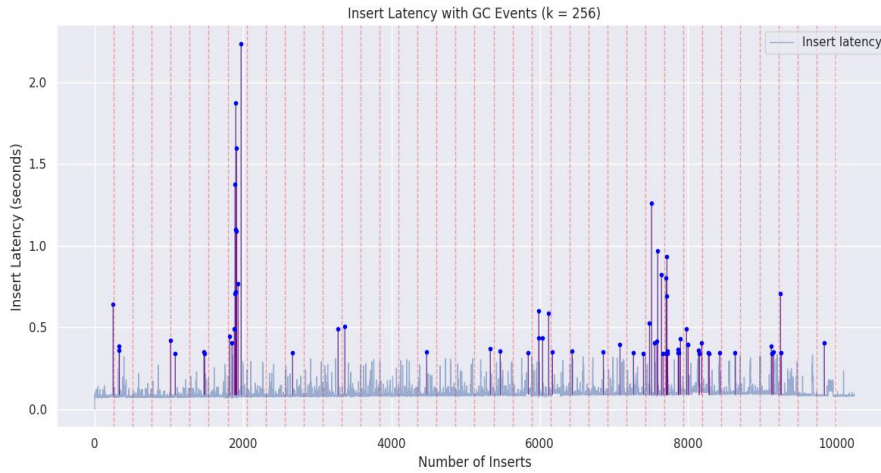


Figure B.3: Insert latency vs. Number of Inserts (GC - 256)

Figure (B.3) depicts a line plot of consecutively adding members inside a group with periodic GC every after addition of 256 members. Again, we observe 6-7 lines that have a minimum of 1s latency to a maximum of 2.4s latency, and this is due to frequent GC events. The variance has increased slightly compared to (GC-128) and is between 0.3s to .4s latency. However, we see a stable baseline, but it is noisier than (GC-128).

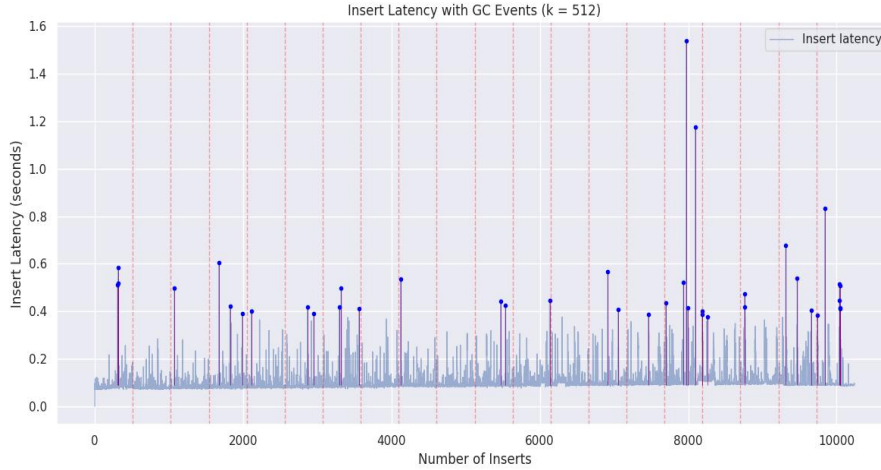


Figure B.4: Insert latency vs. Number of Inserts (GC - 512)

Figure (B.3) depicts a line plot of consecutively adding members inside a group with periodic GC after the addition of 512 members. We observe 2-3 lines that have a minimum of 1s latency. Compared to (GC-128) and (GC-256), the maximum latency is lower, at 1.5s, and this is because of rarer GC events. The variance has increased slightly compared to (GC-258) and is between 0.35s to .4s latency. However, we see a stable baseline, but it is noisier than (GC-256).

B.1.2 Push Latency vs Number of Inserts

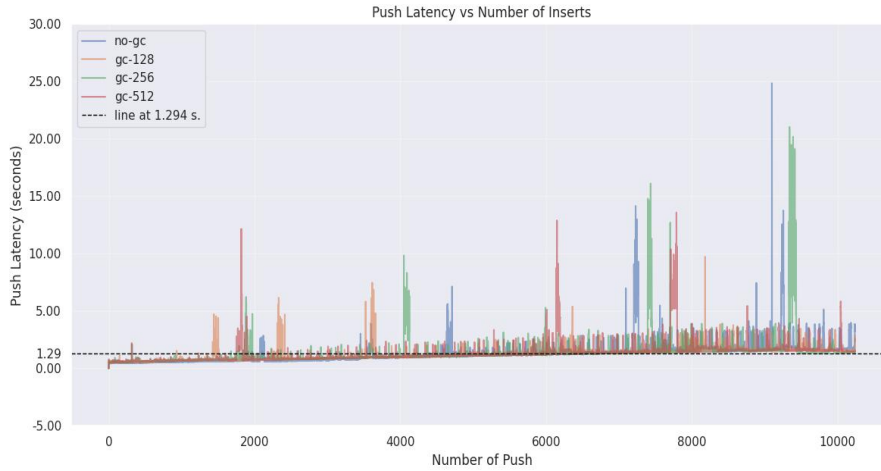


Figure B.5: Push Latency vs. Number of Inserts

Figure (B.5) depicts the line plots of push latency for both GC (128,256,512) and non-GC events. We observe that push cost increases over time for all the configurations. We see a massive push latency between 20-25s for non GC event. However, we observe that both GC-256 and NO-GC tend to produce the worst extreme spikes. Following that, GC-512 exhibits a higher latency of around 10s. GC-128 has spiked around 5s latency, performing better than others. Overall, the push latency trend for all the configurations is increasing, exhibiting large variance.

B.2 Experiment 2

B.2.1 Raw Residual Swift Plots

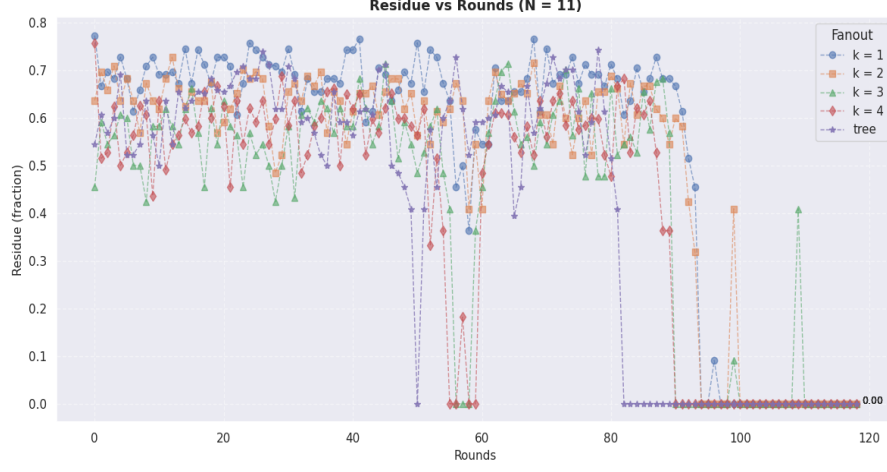


Figure B.6: Residue vs. Number of Rounds (Single Writer)

Figure (B.6) is a representation of an instantaneous plot for different ks (*mesh*) and *tree* topology with a single writer. The residual variance lies between 0.4-0.8, and after 80 gossip cycles, we see convergence where the residue goes to 0.

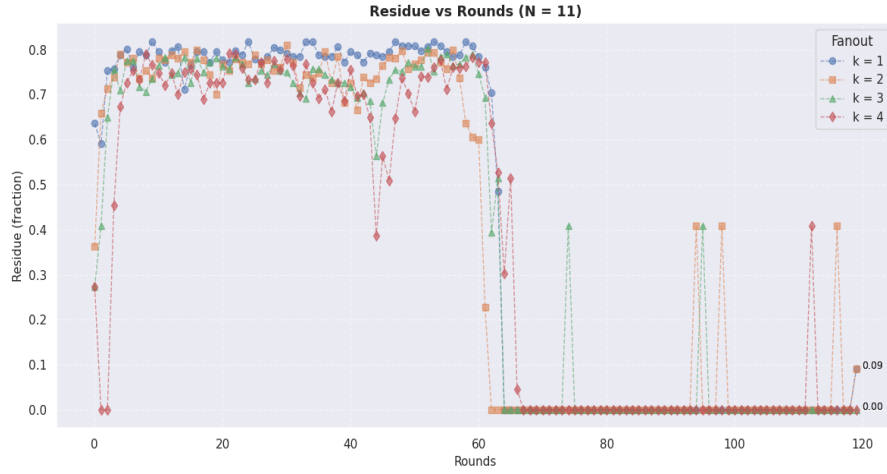
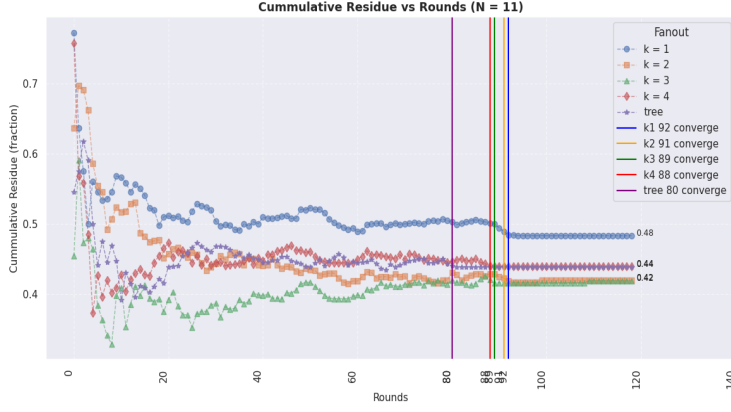


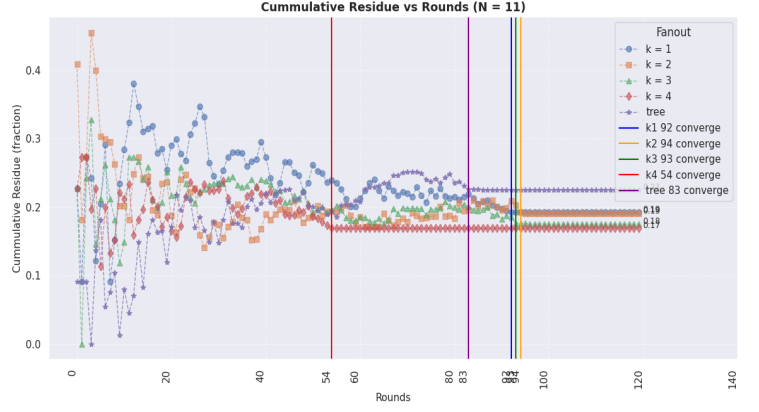
Figure B.7: Residue vs. Number of Rounds (Three Writers)

Figure (B.7) is similar to figure (B.6) but with a single writer. The residual variance lies between 0.6-0.8, and after 60 gossip cycles, we see convergence where the residue goes to 0.

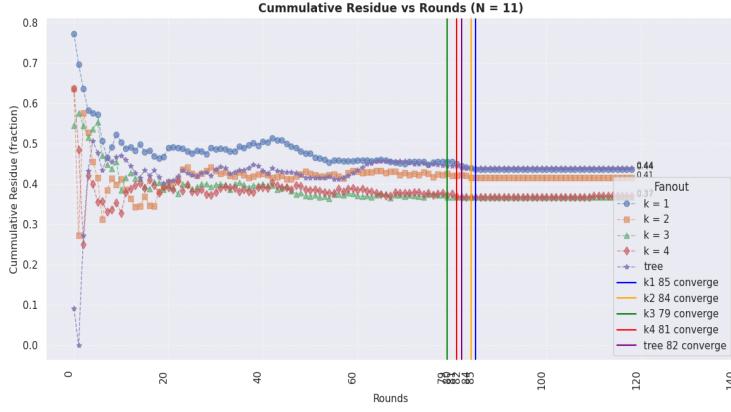
Both plots show temporal behavior as the data is exchanged among the nodes and simply exhibit how convergence happens. There are more plots for different configuration that exhibits temporal behavior. We intend to keep only these two.



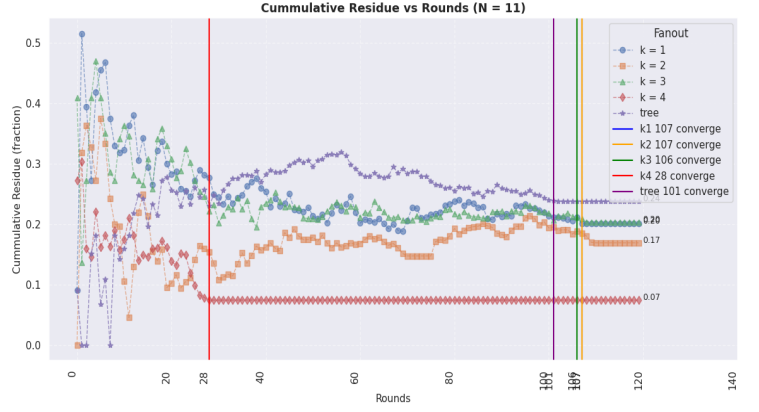
(a): Push



(b): Pull



(c): Push (GC)



(d): Pull (GC)

Figure B.8: Cumulative Residual vs. No. of Rounds (Push/Pull)

B.2.2 Cumulative Residual Plots - Single Writer

In figure (B.7(a)), before 25 gossip rounds, we observe high variance (for the residue) between 0.2-0.8. Many of the nodes didn't see all the data and therefore exhibit high residue. After 40 gossip rounds, the plot gradually leads to convergence and stabilizes. We can clearly see that *tree* topology performs competitively converging at round 80. Out of *mesh* topology, higher fan-out with $k=4$ wins, converging at round 88.

In figure (B.7(b)), before 25 gossip rounds, we observe high variance (for the residue) between 0.0-0.45. Many of the nodes didn't see all the data, and a few of the nodes didn't see any data at all, and therefore exhibit high residue. After 40 gossip rounds, the plot quickly stabilizes and leads to convergence. We can clearly see that the mesh topology with higher fan-out $k=4$ performs competitively, converging at round 54, following that *tree* topology performs the best among other configurations. For $k=1$, the convergence rate is similar to *Push (No-GC)*, and $k=2$ and $k=3$ is slightly slower than *Push (No-GC)*.

In figure (B.7(c)), before 10 gossip rounds, we observe high variance (for the residue) between 0.0-0.8, and after 20 gossip rounds, the plot quickly stabilizes and leads to convergence. We observe that $k=3$ performs the best, converging at gossip round 79, following $k=4$, which quickly converges at round 81. The convergence rate for all the configurations was better in *Push (GC)* than *Push (No-GC)*.

In figure (B.7(d)), approximately before 20-25 gossip rounds, we observe high variance (for the residue) between 0.0-0.5. We observe that $k=4$ quickly converges at round 28. We still observe variance in the residue until round 80-100. *Pull (GC)* performed worst among others, where the convergence radius is more than 100, except for $k=4$ configuration.

B.2.3 CDF Residual Plots - Single Writer

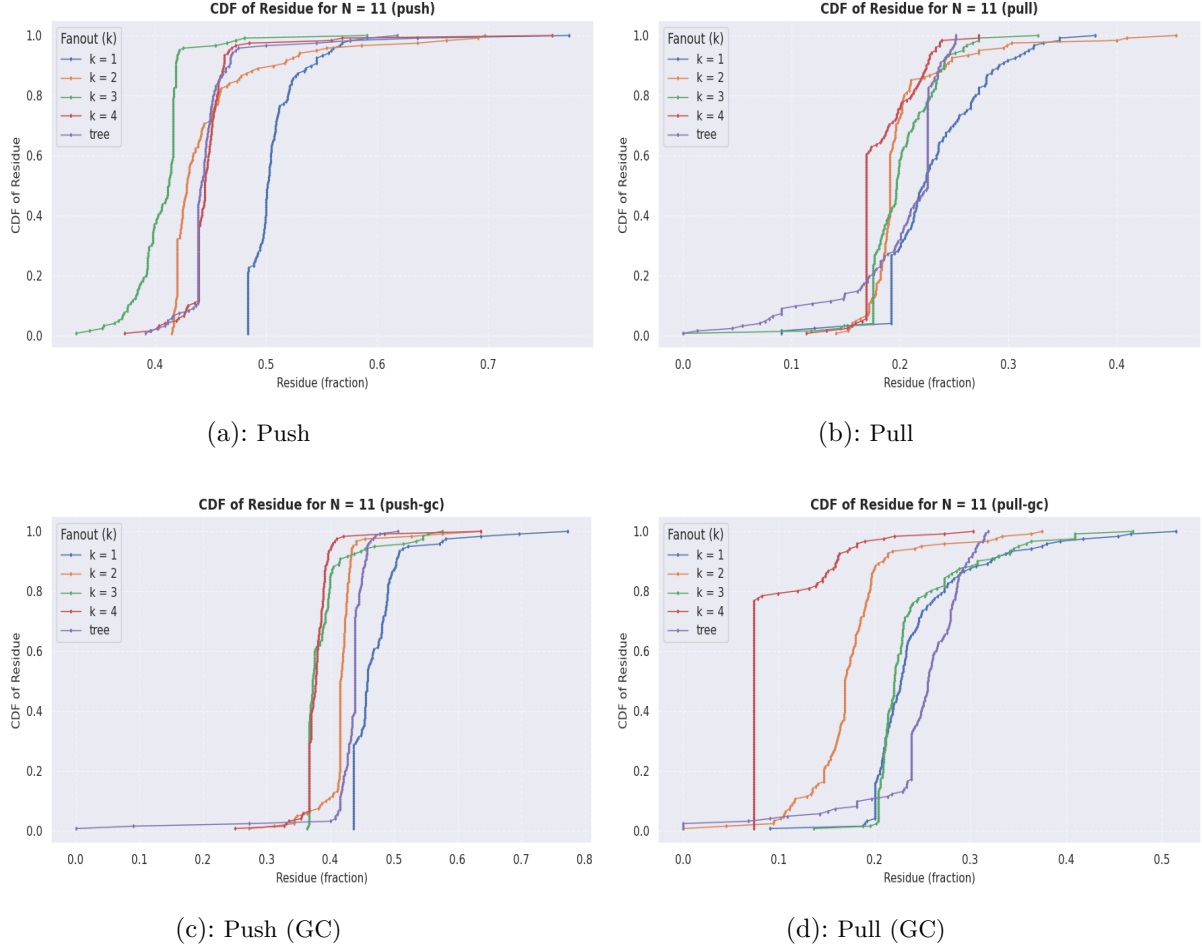


Figure B.9: CDF Residual vs. No. of Rounds (Push/Pull)

In Figure (B.8(a)) for Push (No GC), we observe that increasing k moves the curve to the left side, as higher fanout reduces residue because messages spread faster. $k=1$ is clearly worst shifted to the right side, around ($\sim 0.48-0.7$). $k=3$ outperforms all the configurations and has a steady residue around ($\sim 0.4-0.42$). Initially $k=2$ exhibited stable residue around (~ 0.42), and then we observe increase in residue. *tree* and $k=4$ are tightly clustered and show a controlled propagation (low variance).

In Figure (B.8(c)) for Push (GC), we observe a little residual shift to the left side compared to Push (No GC). $k=3$ and $k=4$ are tightly clustered with low variance and exhibit better propagation and outperform all the configurations. $k=2$ also improves significantly where as $k=1$ showed an increase in residue. *tree* outperforms $k=1$ having residue around ($\sim 0.4-0.45$). GC introduces residual shrinks due to packed redundant loose objects.

In Figure (B.8(b)) for Pull (No GC), we observe low residue compared to Push (No GC and GC). $k=4$ stabilizes early compared to all the configurations. $k=2$ and $k=3$ are steep, showing stable behavior, while both $k=1$ and *tree* performed worst and see a gradual rise in the residue. However, even without GC, pull is more efficient than push in terms of residue.

In figure (B.8(d)) for Pull (GC), the residue is much lower overall, around (~ 0.18 - 0.3). Surprisingly, $k=4$ showed a constant residue for a longer period of time, and outperforms all the configurations. $k=2$ showed lower residue compared to $k=3$. *tree* performed worst among others.

B.2.4 Cumulative Residual Plots - Three Writers

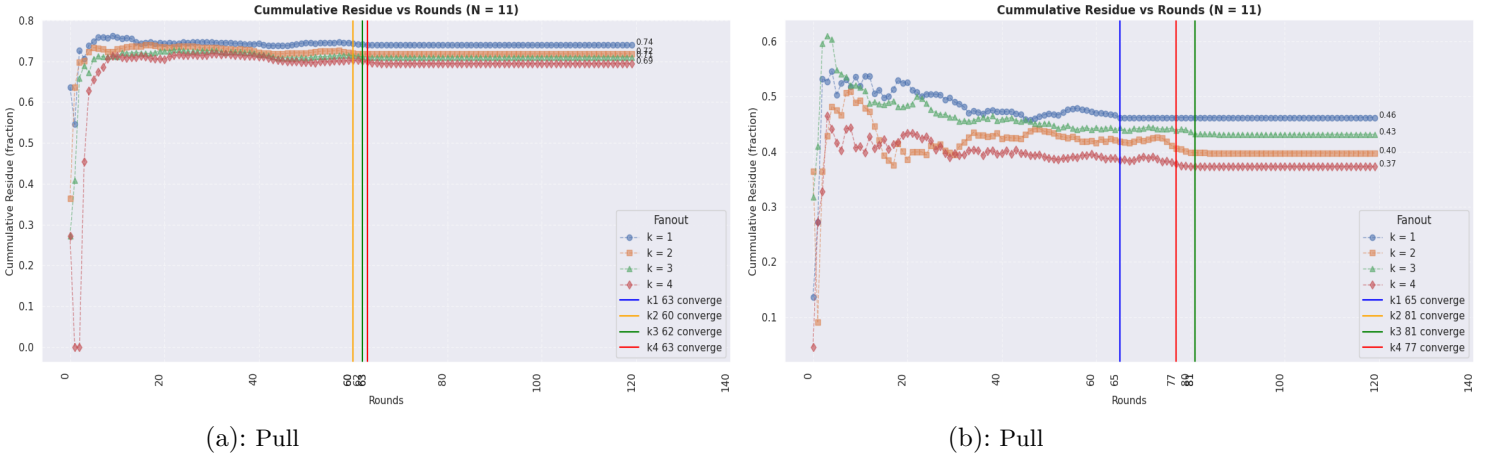


Figure B.10: Cumulative Residual (No-GC) vs. No. of Rounds (Push/Pull)

In figure (B.9(a)), we observe a smooth trend where residual variance lies between (0.6-0.8) for all the nodes (with all the configurations) converging around 60-64 gossip rounds. Fan-out with $k=2$ converges faster.

In figure (B.9(b)), we observe a smooth trend after 40 gossip rounds where $k=1$ converges early. Variance in residual exhibits before 20 gossip rounds, ranging between 0.3-0.6.

B.2.5 CDF Residual Plots - Three Writers

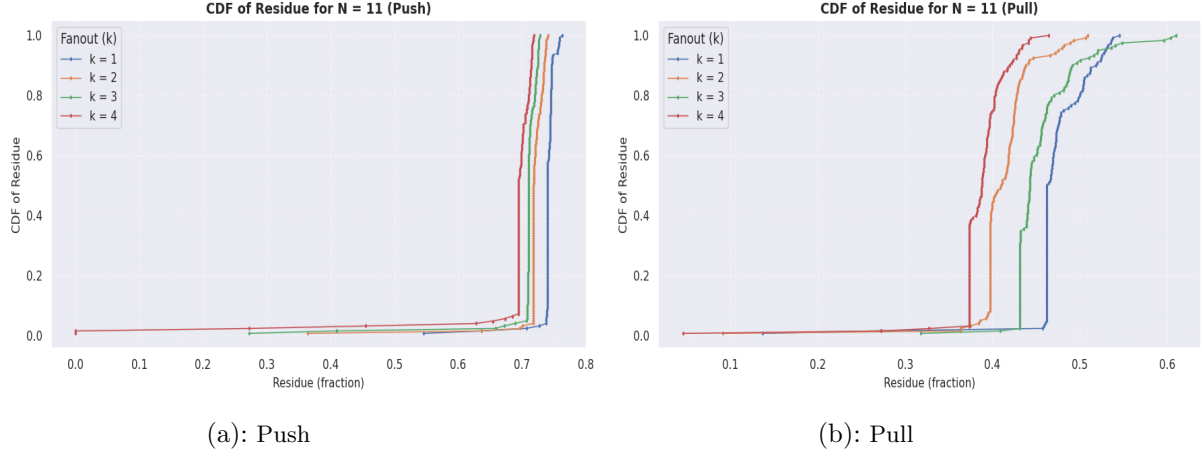


Figure B.11: CDF Residual vs. No. of Rounds (Push/Pull)

In figure (B.10(a)), we observe a dramatic jump in the residue around $\sim 0.7+$. All curves shift to the right side. Due to concurrent writes, we see an increase in duplicate propagation. Concurrent writes induce redundancy. $k=4$ performs the best. Not all k s are tightly clustered.

In figure (B.10(b)), the residue is much lower than Push around (~ 0.38 – 0.50) and reduces redundancy. $k=4$ performs the best among all and not all k s are tightly clustered. Distribution remains consistently lower than push-based dissemination



– Securing Group State and Operations

In *Chapter 8 - Conclusion*, we presented the future extension for our Byzantine nodes (2) and talked about how we can secure it. Here, we will give a brief idea about how we can make use of homomorphic encryption [64] to secure our CRDTs used in our group model design, and also present how one can detect the forks from a malicious node/peer.

C.1 Approach I

In our first approach, we will present a high-level idea of our design, and we'll just present toy data about the homomorphic encryption algorithm for clarification. We also won't specify the types of homomorphic encryption algorithms like PHE, SHE, and FHE.

Assume that our homomorphic encryption algorithm generates **2** keys,

1. **secret-key** (*private*) - which is kept on the client side to decrypt the encrypted text.
2. **public-key** (*public*) - which is shared among the servers to perform computation over the encrypted data. This key can also be used for encryption.

$$secret_key = Ab12Zc, \quad public_key = Qw87D1 \tag{C.1}$$

For illustration, we present dummy key pairs. In practice, these are large random numbers or text generated by cryptographic algorithms.

Also, our homomorphic algorithm only performs computation on encrypted positive integers. It can perform *add* (addition), *sub* (subtraction), *mul* (multiplication), *div* (division), and *mod* (modulus) operations. For now, assume that each API (3.2) from our design abstracts this mathematical computational algorithm.

We also present dummy data in a table form that maps integers to encrypted values. Assume that these values are encrypted from the generated **public-key**. We use the table below as a reference.

Input	Function	Output
0	enc(0)	UU109r
1	enc(1)	axsd9k
2	enc(2)	qlP3xA
3	enc(3)	mZ81we
4	enc(4)	T7saQ2
5	enc(5)	kxL90b
6	enc(6)	Jp4Wz8
7	enc(7)	aR92Km
8	enc(8)	vE6sQ1
9	enc(9)	ySx3D9
10	enc(10)	P0Lwa7
11	enc(11)	N8sK2M
12	enc(12)	ZrA41x
13	enc(13)	bW7mQe
14	enc(14)	sYx90T
15	enc(15)	ysc8A2

Table C.1: Mapping of inputs to encrypted outputs.

C.1.1 Encrypting Blob Values

From *Chapter 3* (5.3.2), our group state member values are a tuple (size 2) of positive integers. From our group state, we now encrypt the data of our member blob values.

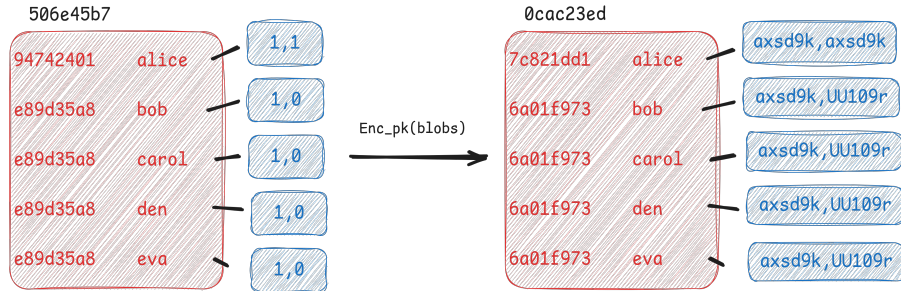


Figure C.1: Encrypted Blobs of the Group State

The above figure, C.1 is the group state tree object where the blob values are in both plain and encrypted text. With the encrypted blob values, the real positive integer values are opaque. But from the encrypted values, we definitely see a pattern.

Let us assume that *holly* is a Byzantine node/peer that stores the chats of other groups as well. Since it's a P2P application, other peers can or have the capability to store one-to-one/group chats of other peers, even if they are not members of those stored groups. With the help of Git plumbing commands, *holly* can now view everyone's blob values and compares it. *holly* can infer that there is a match of similar encrypted blob values to everyone except *alice*, and with that she gets to know that *alice* is an admin. *holly* can easily get into the group by copying *alice*'s blob values and then creating a new commit (forking) and pushing the latest group state to everyone. Now everyone believes that *holly* is a new member of the group with admin privileges.

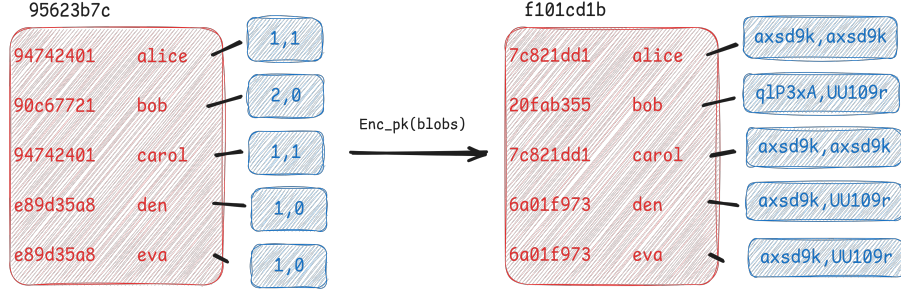


Figure C.2: Encrypted Blobs of the Group State

In the above fig. C.2, now with the help of homomorphic computation, *alice* removes *bob* by incrementing the encrypted first tuple value with the help of the *public-key* without the need of decrypting it, and promotes *carol* by incrementing the second encrypted tuple value, making *carol* an admin. *holly* can still somehow determine the admin values and create a fork (creating a new commit object with the new group state).

C.2 Approach II

From C.1, we presented how we can hide the blob values by encrypting them. But somehow, a malicious node can still figure out who is a regular member and/or an admin. In this approach, we present a non-deterministic way to assign blob values (positive integers) and then encrypt them. This approach will only work when adding a new member to a group (not the already removed member from the group) and for the creator of the group. For this, we will refer to table C.1.

Let $k \in \mathbb{N}_0$ and let $i \in \{0, 1, \dots, k\}$. For every new member or creator of the group, we need to randomly pick i_1 and i_2 within the given range.

For *creator* of the group, both i_1 and i_2 should be a random odd value, because the odd value indicates that it is present within a group and an admin of the group.

For adding a new *member*, only the first tuple value i_1 should be a random odd, indicating the presence of a member inside a group. And the second tuple value i_2 should always be a random value, even indicating a non-admin of the group.

Note that after randomly choosing values for i_1 and i_2 , we need to encrypt and store them inside the blob object.

Now we will present an example of the above approach.

C.2.1 Example i

Let $k = 15$. Now we need to randomly pick the values between $0 \dots 15$. From the fig. C.3, we mapped the randomly picked values from the range $0 \dots 15$ and assigned to the creator as well when adding a new member to the group.

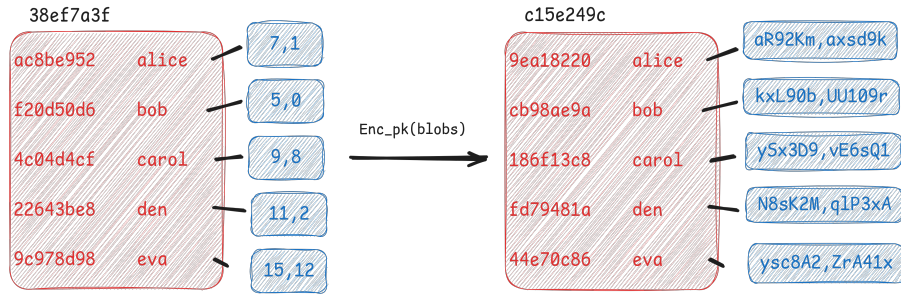


Figure C.3: Encrypted Blobs of the Group State

alice is the creator and admin of the group, and all others are normal members of the group. With the encrypted values, it would be difficult for a malicious node/peer to distinguish between admins and members of the group, and it would not be able to get inside the group and fork it.

But does this conform with the laws/rules of CRDTs? What would happen with concurrent events?

C.2.2 Example ii

We now present a concurrency table where two admins, *alice*, and *bob*, are trying to add *carol* in the group. *bob* also removes *carol* after adding it to the group. Here, we only present the plain integer values, not the encrypted ones. Let k be 15.

Time	Alice's Event	Bob's Event
t=0		Bob adds Carol Carol = (7,4)
t=1	Alice adds Carol Carol = (13,2)	
t=2		Bob removes Carol Carol = (8,4)

Table C.2: Concurrent operations.

In the above table, at $t=0$, *bob* adds *carol* with a random tuple value within k 's range, which is (7, 4). Then at $t=1$, Alice adds *carol* with a random tuple value which is (13, 2). Finally, at $t=2$, *bob* removes *carol*, incrementing the first tuple value by 1, and that makes (8, 4).

If we try to merge the tuple values by taking the max of both *alice* and *bob*, we get (13, 4).

alice sees a valid state but for *bob*, it is an abnormal behavior. It is conforming with the rules of CRDTs because after merging the values by taking the max, they are monotonically increasing. But it doesn't follow the order of events.

Let us interpret the above table (C.2) in a concurrency diagram.

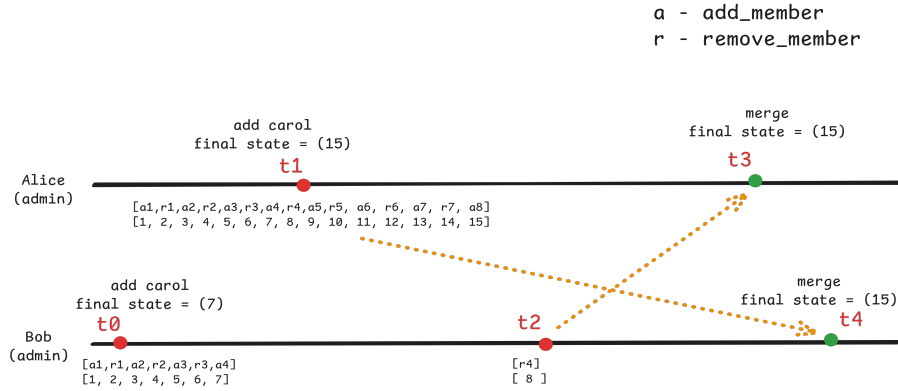


Figure C.4: Concurrent Diagram: Add & Remove For Homomorphic Values

From the fig. (C.4), let us assume the events t_0 and t_1 as a bunch of add and remove member events. At t_0 , *bob* adds and removes *carol* consecutively until the final value is 7. At t_1 , *alice* adds and removes *carol* consecutively until the final value is 15. At t_2 , *bob* removes *carol* and increments the value, making it 8. Now the states are exchanged among the peers, resulting in the final merge value 15. We see different orders of events with *carol* being a member of the group. *Does this case look familiar?* We have already stumbled upon such a case and tried to sketch out a solution for this situation in (3.5) and discussed the existing flaw about the nodes being Byzantine.

Choosing a larger k value could give more advantage to the malicious peer. If $k=1000$, and a randomly selected value to add a member to a group is near 1000, then the malicious peer has full control over that added member. However, we can choose a larger k value, but it should be in a sub-range. All peers should agree on that specific sub-range.

With immense research and design choices, we can opt for this idea to secure our group state object.

C.3 Approach III

In this section, we will show how we can detect forks. What happens if a malicious node/peer somehow finds the admin values and then tries to create a fork by adding itself to the group? Or, what about Byzantine members who are already present inside a group? They clearly have the knowledge about the admins present inside the group.

From our implementation chapter, our commits are signed by the peer's private key. We know that a few group operations, like (add, remove, promote, demote) are only performed by admins. So to detect the forks, we can check if that commit has been signed by any one of the admins present inside the group.

Note, we should be able to distinguish the signed commits between the removal of a member and self-exiting by a member.

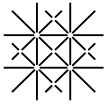
We earlier talked about the key-pair for each group. To add additional security, we can have an additional sign the commit with the group's secret-key.

C.4 Conclusion

All the presented approaches are just vague information based on my distributed systems knowledge and can be researched thoroughly. We can combine both homomorphic encryption and detect-

ing forks to add better security to our group chat application.

With in-depth research, we can test this idea of homomorphic encryption over *cl-crdt*[68] or *infy-crdt*[62]. There is a possibility to come up with a new research paper, *maybe* or *maybe not*.



Declaration on Scientific Integrity

(including a Declaration on Plagiarism and Fraud)

Translation from German original

Title of Thesis: _____

Name Assessor: _____

Name Student: _____

Matriculation No.: _____

I attest with my signature that I have written this work independently and without outside help. I also attest that the information concerning the sources used in this work is true and complete in every respect. All sources that have been quoted or paraphrased have been marked accordingly.

Additionally, I affirm that any text passages written with the help of AI-supported technology are marked as such, including a reference to the AI-supported program used. This paper may be checked for plagiarism and use of AI-supported technology using the appropriate software. I understand that unethical conduct may lead to a grade of 1 or "fail" or expulsion from the study program.

Place, Date: _____ Student: _____

Will this work, or parts of it, be published?

No

Yes. With my signature I confirm that I agree to a publication of the work (print/digital) in the library, on the research database of the University of Basel and/or on the document server of the department. Likewise, I agree to the bibliographic reference in the catalog SLSP (Swiss Library Service Platform). (cross out as applicable)

Publication as of: _____

Place, Date: _____ Student: _____

Place, Date: _____ Assessor: _____

Please enclose a completed and signed copy of this declaration in your Bachelor's or Master's thesis.